JOINT INTERDISCIPLINARY PROJECT 2026

Financial Fraud Detection via Quantum Reservoir Computing

Final Report

TEAM 6.8.1

Linh Dang
Jules Dupont
Radu Cosmin Dumitru
Ioanna Giannakopoulou
Prisha Meswani
Julian Ruiz Avella
Lola Serrano Carrasco

COMPANY COACHES

Alain Champenois
Karla Baumann

Contents

1 Introduction **4**

2 Problem definition **6**

3 Business context **7**
3.1 Stakeholder analysis . . . 7
3.1.1 Potential adopters . . . 7
3.1.2 Power-interest matrix . . . 7
3.2 User requirements . . . 8
3.3 Use case . . . 9

4 State of the art **10**
4.1 Existing methods for fraud detection . . . 10
4.1.1 Real-time authorization . . . 10
4.1.2 Asynchronous post-authorization processing . . . 11
4.1.3 Anti-fraud systems benchmark . . . 11
4.2 Existing literature on QML applied to quantitative finance . . . 11
4.3 Classical and quantum reservoir computing . . . 12

5 Solution concepts and selection **14**
5.1 Scope: which history the model sees . . . 14
5.2 Design constraints . . . 14
5.3 Concepts . . . 14
5.4 Selection . . . 15
5.5 Sliding-window QRC protocol . . . 16

6 Methodology **19**
6.1 Data . . . 19
6.1.1 Dataset choice . . . 19
6.1.2 Transaction generation . . . 19
6.1.3 Chronological split . . . 20
6.1.4 Features, preprocessing and windows . . . 20
6.2 Implementation of the sliding-window QRC . . . 20
6.2.1 Circuit construction . . . 20
6.2.2 Input encoding . . . 21
6.2.3 Quantum reservoir . . . 21
6.2.4 Observables . . . 21
6.2.5 Simulation backends . . . 22
6.2.6 Classical readout . . . 22
6.3 Baselines and benchmarking methods . . . 22
6.3.1 Direct logistic regression . . . 23
6.3.2 Echo state network . . . 23
6.3.3 Long short-term memory network . . . 23
6.3.4 Information processing capacity . . . 23
6.4 Evaluation methods . . . 24
6.5 Software and reproducibility . . . 25

7 Results **26**
7.1 Memory retention of the QRC . . . 26
7.1.1 Delay and window length . . . 26
7.1.2 Observable choice and shot precision . . . 27
7.2 Untuned QRC performance on fraud detection, against classical baselines . . . 28

7.3 Equal-budget tuning of the models . . . 31
7.4 QELM with engineered features . . . 33
7.5 Multi-objective optimization (Pareto front analysis) . . . 34
7.6 QRC performance under realistic conditions . . . 36
7.6.1 The reservoir on a spin-qubit chip . . . 36
7.6.2 Memory under hardware noise . . . 36
7.6.3 Effect of different noise types on memory . . . 37
7.6.4 Fraud detection on the chip . . . 38
7.7 Cost of running the QRC: time and energy . . . 39

8 Discussion **40**
8.1 Interpretation . . . 40
8.2 Quantum vs classical, under realistic conditions . . . 41
8.3 Limitations . . . 41
8.4 Validation of the minimum viable product . . . 42

9 Impact assessment **43**

10 Recommendations and future work **45**

11 Team reflection **46**
11.1 Team process during project . . . 46
11.2 Interdisciplinary work . . . 46

12 Conclusion **48**

Appendix **53**

A Model performance metrics **53**

B Dataset selection and chronological partitioning **55**
B.1 Assessment of candidate datasets . . . 55
B.2 Split strategies considered . . . 55
B.3 Implemented split and leakage control . . . 56

C Raw-feature signal by fraud category **57**
C.1 Objective and procedure . . . 57
C.2 Results . . . 57
C.3 Implications and limitations . . . 58

D Data generation, preprocessing and evaluation implementation **59**
D.1 Generation procedure and transaction schema . . . 59
D.2 Chronological partitioning and configuration precedence . . . 60
D.3 Historical feature construction . . . 60
D.4 Preprocessing and customer-specific windows . . . 61
D.5 Evaluation categories and ranking metrics . . . 61
D.6 Validation decision threshold . . . 61
D.7 Daily Card Precision@K . . . 62
D.8 Reproducibility, validity and interpretation . . . 62
D.9 Configuration search space for multi-objective optimization . . . 62

E Mathematical formulation of IPC implementation **63**
E.1 Normalized Legendre polynomials . . . 63
E.2 Target functions by degree . . . 63
E.3 Capacity calculation and aggregation . . . 64

F Reservoir architectures and Hamiltonians **65**
F.1 Hamiltonian models . . . 65
F.2 Random circuit reservoir . . . 65
F.3 Hamiltonian evolution . . . 66

G Operational metrics under a false-positive budget **67**

H Classical fixed random feature maps **68**
H.1 Tanh random feature map . . . 68
H.2 Random Fourier features . . . 68

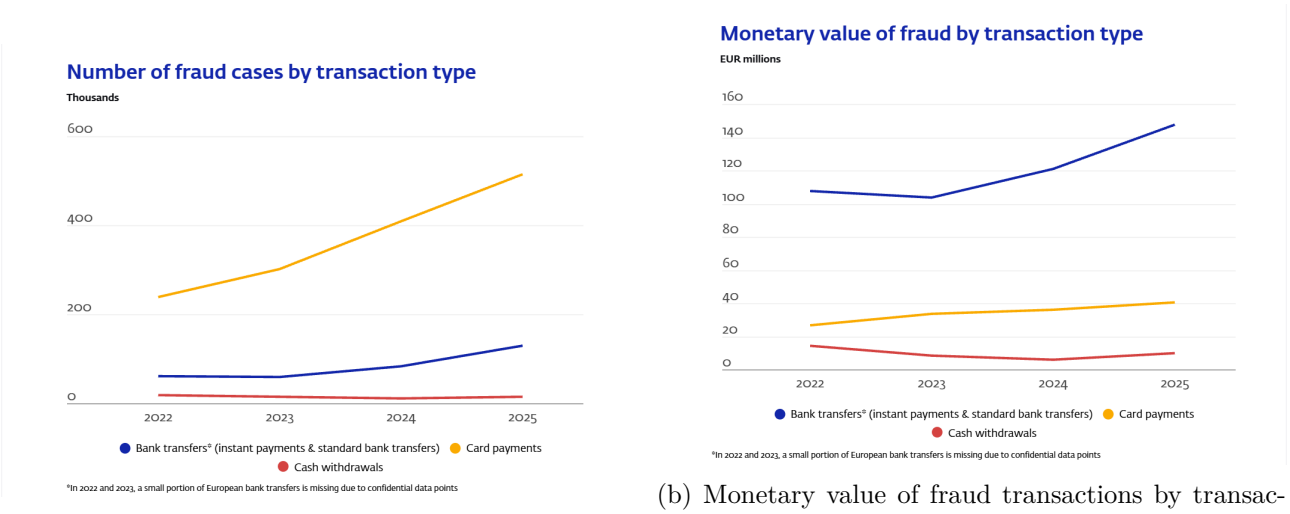
I Full IPC results across varying parameters **69**
I.1 Window lengths and maximum delays . . . 69
I.2 Observables and precision . . . 69
I.3 Observables and precision: absolute IPC . . . 70
I.4 Observables and precision under noise . . . 72

J Input qubits reset ablation **73**

1 Introduction

Financial fraud, according to the USA Bureau of Justice Statistics, can be defined as the intentional misrepresentation of information or identity to deceive others, the illegitimate use of a credit or debit card, or the use of electronic means to transmit deceptive information in order to obtain monetary income (Bureau of Justice Statistics, 2017). In the Netherlands, fraudulent payment transactions amounted to €198 million in 2025 (De Nederlandsche Bank, 2026). This criminal activity has been evolving thanks to digital technologies. Nowadays, perpetrators have a wide range of techniques to reach and defraud potential victims through social media, websites, and many other online platforms.

De Nederlandsche Bank reported a total monetary value of €41 million in the Netherlands for fraud cases related to in-person and online card payments. Although it is not the fraud type with the largest monetary value, it is the most common one, with 514,000 fraud cases reported in 2025 (see Figure 1a and Figure 1b).



(a) Number of fraud cases by transaction type.

(b) Monetary value of fraud transactions by transaction type.

Figure 1: Fraud cases and monetary value by transaction type. Source: De Nederlandsche Bank (2026)

This project focuses on card payment fraud, which refers to situations in which "an individual other than the cardholder unlawfully conducts transactions using a card that is stolen, lost, or otherwise misused" (Popova and Gardi, 2025). Victims' card information is commonly obtained through phishing attacks, social engineering, or massive data leakages.

Criminals are continuously looking for ways to cover up their activities using more sophisticated methods, which makes it increasingly difficult to differentiate a legitimate transaction from a suspicious one. Machine learning has emerged as a powerful tool against these criminal actions due to its data processing capabilities, adaptability, and especially its ability to recognize complex patterns (Pan, 2024). The currently implemented models learn the user's usual behavior and flag any transaction that is an outlier with respect to the learned pattern. In that case, the model triggers security measures that instantly try to identify if the legitimate cardholder is behind the suspicious transaction. The main objective of these systems is to protect cardholders and retailers by automatically blocking any fraudulent transaction before any money is moved.

Since fraud detection systems rely heavily on temporal memory, paradigms such as Reservoir Computing (RC) are gaining relevance in this context, as they reduce training complexity and therefore the energy cost, which is one of the biggest downsides of the currently used models. Quantum Reservoir Computing (QRC) is an extension of RC that exploits the characteristics of quantum systems to implement highly expressive nonlinear mappings with a relatively small quantum setup (Dos Santos, 2026).

This project was carried out under the guidance of Quobly, a quantum computing company based in Grenoble, France, which develops silicon-based quantum computers using proven semiconductor

manufacturing processes. At Quobly, financial fraud detection is seen as a promising near-term application for QRC. To assess the relevance of this technology in the financial industry, they decided to work with TU Delft through a Joint Interdisciplinary Project. The main objective of the project is to evaluate how effectively gate-based QRC can perform against existing classical benchmarks in identifying high-dimensional, time-dependent fraud patterns.

To achieve this objective, the team and the company evaluate whether spin-qubit silicon architectures can capture non-linear temporal dynamics in financial transactions. With this project, we intend to contribute to the study of QRC in the financial sector and strengthen QRC's applicability to other similarly complex dynamical systems.

The rest of the report is structured as follows. Section 2 defines the problem, lists the research and design questions, and compares the initial scope with the work achieved. Section 3 analyses the business context: stakeholders, user requirements, the use case and an industry benchmark. Section 4 reviews current fraud detection methods, the literature on QML and QRC in finance, and the basics of classical and quantum reservoir computing. Section 5 presents the three solution concepts considered and the selected one, sliding-window QRC, together with its protocol. Section 6 describes the methodology: data, implementation of the QRC, classical baselines and evaluation. Section 7 presents the results, and Section 8 discusses these results. Section 9 assesses the impact and ethical implications of the project, and Section 10 presents the business case and recommendations for the company. Section 11 reflects on the team process and interdisciplinary work, and Section 12 concludes.

2 Problem definition

Credit card fraud causes significant financial losses. Detecting fraud is an important and challenging task, as it requires models that capture nonlinear, time-dependent patterns in highly imbalanced datasets. Quantum reservoir computing has been proposed as an innovative approach to capture such complex patterns. However, it is still unclear whether QRC offers a practical advantage over established fraud detection methods. Therefore, the problem addressed in this project is to investigate how a gate-based, sliding-window QRC pipeline can be implemented for credit card fraud detection, and to evaluate how it performs compared to existing classical benchmarks in identifying high-dimensional, time-dependent fraud patterns. Some of the research and design questions that guided us were:

1. Which evaluation metrics are meaningful for fraud detection?
2. What is the relevant transaction history the model should see?
3. How can we best map high-dimensional financial data onto a quantum register?
4. How does QRC compare to classical baselines?
5. How to model realistic silicon spin-qubit hardware conditions?
6. Can QRC improve financial fraud detection performances under realistic hardware conditions?

During the initial phase of the project, the scope of the project was defined as follows:

> Developing a QRC pipeline for temporal credit card fraud detection using a financial dataset, Qiskit (Javadi-Abhari et al., 2024), SpinPulse emulator (Vermersch et al., 2026), and the Optuna optimizer (Ozaki et al., 2026).
>
> The model will cover feature selection and encoding, quantum reservoir evolution, measurement, classical readout, parameter optimization, and performance evaluation.
>
> The main variables to be investigated include feature-to-qubit mapping, number of qubits, circuit depth, observables, readout parameters, hardware noise, number of evaluations, and the acceptable performance threshold defined through a classical baseline.
>
> The intended outcomes are a reproducible QRC implementation, a quantitative understanding of model's accuracy, temporal memory, and resource requirements, and an optimized multi-dimensional QRC circuit.

3 Business context

Analyzing the external business context for the development of a QRC based fraud detection system is key to determining how this new technology could be used in a real setting. To achieve this, we defined and analyzed the most relevant stakeholders involved in the context of this exploratory project. We focused mainly on identifying prospective customers and their needs to set the baseline and establish where QRC could fit in the financial industry.

3.1 Stakeholder analysis

The stakeholders for this project were identified using the Project Management Institute (PMI) guidelines. According to PMI, a stakeholder is an individual, group, or organization that has an interest in the project and is capable of mobilizing resources to affect its outcome to some extent (Smith, 2000). For a quantum-powered fraud detection system, we defined several roles based on The Open Group Architecture Framework (TOGAF) stakeholder map to distinguish foundational technology vendors and the execution team (Ewig, 2024). Additionally, innovation diffusion theory was used to analyze potential commercial adopters within the financial technology ecosystem.

3.1.1 Potential adopters

To identify potential adopters of this technology, we began by identifying the actors liable for credit card fraud. In most cases, cardholders are not responsible for fraudulent transactions, since card issuers offer voluntary zero-liability policies to protect their customers. As a result, the cost of fraud is typically absorbed by either the card issuer, or the merchant involved. Depending on the circumstances, the party that authorizes the fraudulent transaction bears the associated losses (Patel, 2026).

Therefore, to protect themselves in the event of a dispute, each of these actors must implement its own fraud detection systems. Contrary to the major card issuers, most of the merchants cannot afford to develop their own anti-fraud systems. They contract third-party software from anti-fraud SaaS vendors or a payment processor who manages the entire collection process (Dataintelo, 2026).

Payment processors also play an important role in credit card fraud detection, it is easier for these actors to detect suspicious patterns thanks to their access to a high volume of data through their network of customers. As they compete (e.g. Visa vs. Mastercard), card issuers would prefer the service which helps them to catch fraudulent patterns more effectively, anti-fraud systems play the role of a value-added service (Visa Contributor, 2025).

The main potential customers of a quantum powered fraud detection system would be card issuers, payment networks, anti-fraud SaaS vendors, and payment processors. These actors could implement a quantum machine learning model into their systems to enhance them.

3.1.2 Power-interest matrix

To characterize the stakeholders and define an engagement strategy, a power-interest matrix was used. The detailed stakeholder analysis can be found in Table 1.

In this context, not all of the most influential stakeholders show a high level of interest in QRC due to its novelty. For example, financial regulators, anti-fraud SaaS vendors or payment processors. These actors could hinder the commercial adoption of the system during the exploratory phase. Poor management of their expectations regarding QRC could result in them discarding it as a potential technology to enhance their operation. Keeping them satisfied enables those responsible for the technical and research execution to continue developing QRC for credit card fraud detection without friction. At this stage, it is important not to overwhelm high power/low interest stakeholders with excessive consultations or detailed project updates. Providing high-level visibility of the project's progress is the most effective way to keep them comfortable with the technology, build stronger relationships, and support future adoption and funding opportunities.

For highly interested stakeholders, such as investors, it is essential to keep them engaged by establishing structured and regular communication, co-creating goals, defining clear expectations, and

involving them in key decision-making milestones. These stakeholders should be treated as strategic partners, whose expertise and resources can contribute to the project. Having proactive and collaborative engagement reduces the risk of potential roadblocks or resistance.

Table 1: Comprehensive stakeholder analysis matrix for QRC-based fraud detection

Stakeholder	**Role**	**Power / Interest**	**Description**	**Engagement Strategy**
Quobly	Foundational Vendor	High power / High interest	Hardware/software provider exploring applications for its Quantum Reservoir Computing (QRC) technology and building a commercial business case.	Manage closely
JIP Team	Technical Execution	Moderate power / High interest	Group of TU Delft master's students developing a financial fraud detection algorithm using QRC technology.	Manage closely
Investors	Sponsors	High power / High interest	Venture capital firms or corporate investors seeking investment opportunities in early-stage QRC development.	Regularly engage
Card Issuers	Adopters	High power / Low interest	Financial institutions providing payment cards. Potential contractors of fraud detection services to mitigate loss exposure.	Keep satisfied
Academic & R&D Translators	Technical Execution	Low power / High interest	Academic institutions and research labs interested in verifying algorithmic feasibility and scientific rigor.	Maintain interest
External Financial Regulators	Regulatory Bodies	High power / Low interest	Government and international bodies monitoring financial operations and AI auditability (e.g., SEC, FCA, FinCEN) with enforcement powers.	Keep satisfied
Payment Networks	Adopters	High power / Low interest	Infrastructure entities connecting card issuers, acquiring banks, and merchants (e.g., Visa, Mastercard). Prioritize system performance over underlying methods.	Keep satisfied
Credit Card Holders	Impacted Parties	Low power / Low interest	End-consumers requiring secure transactions and protection against unauthorized account access.	Keep informed
Merchants & E-Commerce	Impacted Parties	Low power / High interest	Businesses accepting digital payments, focused on reducing chargebacks, fraudulent orders, and false-positive declines.	Keep informed
Enterprise Anti-Fraud SaaS Vendors	Adopters	High power / Low interest	Enterprise platforms delivering real-time risk scoring, chargeback guarantees, and behavioral analytics.	Keep satisfied
Payment Processors	Adopters	High power / Low interest	Entities managing online transaction processing (e.g., Stripe, Adyen) using anti-fraud infrastructure as a core differentiator.	Keep satisfied

3.2 User requirements

Stakeholders who use anti-fraud systems aim to block all unlawful transactions ideally before any money moves between organizations to avoid dealing with chargebacks and further costs. Therefore, the models must comply with service level agreements (SLAs) regarding the time they have to make a decision. These depend on the fraud detection phase the system is implemented in and it could vary from less than 100 ms for real-time detection systems (Abemon Engineering, 2026) to several days for asynchronous post-authorization processes (PayCompass, 2025).

The users expect these systems to primarily maintain the approval rate while managing the false positive rate, and, to a lesser extent but importantly, to limit the fraud loss rate. In this context,

the false positives are among the most expensive mistakes that the model can incur. More than 40% of customers abandon their carts if the system declines their payment attempt rather than retrying the purchase. This not only prevents the retailer from gaining potential revenue but also diminishes the reputation of other stakeholders, such as the card issuer, payment network, or payment processor. Flagging a genuine transaction as fraudulent can result in customer churn (Ravelin, 2024).

In the case of false negatives, the real cost of a dollar lost in a successful fraudulent transaction is between $4.36 and $5.75, considering the administrative cost of dealing with the situation, which includes chargeback fees and replacing physical cards or credentials. Nevertheless, this cost stops there; losing a customer because of a false positive implies wasted customer acquisition costs, lifetime value destruction, support overhead, and more hidden costs. According to industry estimates, the actual fraud losses hover around $48 billion per year, whilst false decline losses are between $200 billion and $440 billion per year (Mohanty, 2026).

3.3 Use case

Considering the user requirements and the compliance with SLAs, a useful, realistic application of a QRC based machine learning model to detect credit card fraud would be in the asynchronous phase of the process. Based on discussions with our coaches and our own expertise with this technology, it is unlikely that an implementation of QRC would take less than 100 ms to analyse the data and make a decision; the system needs time to encode the features into the quantum system and allow the reservoir to evolve to generate a proper response.

At early commercial stages, a QRC circuit could compete with other machine learning models used in the industry, thanks to its efficient resource consumption. Due to its intrinsic quantum properties, this type of system has the potential to process information and discover subtle relationships between features with a smaller setup, in terms of physical components and hardware footprint, compared to the currently used methods (Dos Santos, 2026).

Nevertheless, it is important to note that an industrial application of QRC is not feasible in the near term due to the high upfront costs associated with developing quantum hardware. This use case is therefore based on theoretical expectations, and its validity depends on future advancements in the technology.

4 State of the art

In this section, we present the existing fraud detection methods, as well as an overview of the existing literature on quantum machine learning (QML) and QRC applied to quantitative finance. Additionally, we include a brief theoretical description of reservoir computing and its quantum variation. The description of a quantum reservoir system is further explained in more detail in Section 5.

4.1 Existing methods for fraud detection

Currently, to address fraudulent transactions, card issuers, payment networks and processors rely mostly on classical machine learning, deep learning and real-time stream processing. As we mentioned in the previous section, automatic fraud detection is divided into two sequential phases: in-line real-time authorization and asynchronous post-authorization processing.

4.1.1 Real-time authorization

This first phase needs to be completed quickly; the authorization decisions have between 100 ms and 250 ms to be committed. Exceeding this time would cause checkout abandonment or a violation of the payment network SLAs (Abemon Engineering, 2026).

A modern real-time fraud detection pipeline is a multi-stage system composed of machine learning models and/or rule-based checks which return an approve/decline outcome (Jiang, 2026). Most firms use a hybrid approach which combines both systems to improve accuracy (Nowak, 2026).

The rule-based engine uses deterministic logic to approve transactions. The outcome of this system is based on a mix of decision tables, threshold rules, deny lists and behavioral baselines (Nowak, 2026). For instance, a rule-based engine would immediately block a card-present transaction made in Asia after another made in America with only a couple minutes of difference between each other. This method helps to screen suspicious movements faster than other more sophisticated methods that could overlook these logical rules. On the other hand, machine learning methods are used to score the transactions' risk using contextual features such as the last transactions done by the cardholder, merchant's data or IP address. If the result does not comply with the established threshold the transaction is flagged as suspicious and this activates the firms' security systems.

The most commonly used machine learning methods for live fraud detection are:

- **Gradient Boosted Decision Trees** (XGBoost, LightGBM, CatBoost): due to their short latency of 5 to 15 ms, decision trees are the most commonly used models for this application. The tree ensembles handle missing data and non-linear relationships without heavy preprocessing. It is also useful to explain why the transaction had a specific score (Abemon Engineering, 2026).
- **Isolation forest and autoencoders** (unsupervised learning): they come handy for anomaly detection when labeled fraud data is sparse, for example, for new merchants onboarding, new geographies or even for a novel fraud pattern. These models are recommended for new users or as supplementary methods for the supervised model (Abemon Engineering, 2026).
- **Transformer-based sequence models**: this model is used to analyze the session behavior behind a transaction. The model considers the sequence of clicks, page views, and API calls that lead to a transaction. It is mainly used in card-not-present operations where the latency budget allows, as it can take from 50 to 150 ms (Abemon Engineering, 2026).

The input of these models is provided by a feature store that mixes the transaction data with other relevant sources and transforms it into significant features such as the number of transactions on the card in the last hour, the number of transactions from this IP address in the last 5 minutes or days since the device fingerprint was first observed on the platform.

4.1.2 Asynchronous post-authorization processing

In this phase, fraud detection methods access a richer dataset and, due to banks being able to hold authorization for several days, have a longer time span (minutes or hours) to analyze the transactions (PayCompass, 2025). This second layer of protection aims to catch sophisticated fraud and intercept high-risk transactions before money is moved. The methods used in this step analyze behavioral and pattern anomalies, network relationships, and identity resolution. The most used machine learning models in the industry for this phase are:

- **Graph Neural Networks** (GNNs): evaluates graph topology across several nodes in order to reveal fraud rings, account takeover farms, and sophisticated fraud networks. These models are normally too slow for real-time authorization and are used by institutions with massive data scale such as Visa, Mastercard, Chase, or Amazon (Rees and Liu, 2025).
- **Gradient Boosted Decision Trees** (XGBoost/LightGBM): although its main task is to re-score individual transactions with enriched feature vectors, it can be used together with GNNs to improve the system's scalability and explainability (Rees and Liu, 2025).
- **Recurrent Neural Networks** (Long short-term memory, Gated Recurrent Unit): instead of only analysing the current transaction to generate predictions, these models process transactions as an ordered time-series sequence to track the evolving story of the customer or the account (Branco et al., 2020).

These models are used by financial institutions and large retailers with different purposes. For the former, their main objective is to ensure their customers' account safety, while the latter aims to avoid more financial losses by cancelling suspicious orders before the goods leave their warehouses.

4.1.3 Anti-fraud systems benchmark

The work by Ma et al. (2026) discusses some key performance metrics for various machine learning models used for credit card fraud detection. The authors consider that detecting this criminal activity is a rare-event classification problem as fraudulent transactions account for only a tiny proportion of all transactions. The metrics for the best model among those presented in the article, in terms of false positives, are shown in Table 2.

Table 2: Performance metrics for the best model (XGBoost-Synthetic Minority Oversampling Technique-Edited Nearest Neighbor-RandomSearch) (Ma et al., 2026). Note these figures are given for reference only, and should not be directly compared with the results we present in this report.

Model	**Precision**	**Recall**	**FPR**	**FNR**	**MCC**	**FP**	**FN**
XGBoost-SMOTE-ENN	0.922	0.716	0.0105%	28.40%	0.812	9	42

This model could be considered a benchmark for a basic machine learning anti-fraud system that heavily penalizes customer friction. The Matthews correlation coefficient proves that the model's high performance is not a product of the imbalanced dataset, it maintains a reliable correlation between its predictions and the real result. It is exceptionally safe for the user experience thanks to its very small FPR, it rarely blocks legitimate customers. Nevertheless, checking the FNR, it misses almost 30% of fraud cases the model adopts a conservative threshold. This could be considered a trade-off of having almost zero false positives. More details about these metrics and how they are calculated can be found in the Appendix A.

4.2 Existing literature on QML applied to quantitative finance

There is broad academic literature on potential applications of QML techniques to finance, with quantitative evidence on the added value they could have over classical methods. Nevertheless, the

proposals remain potential use cases rather than techniques adopted by the financial industry. The authors have an optimistic stance on the potential role of quantum computers in quantitative finance. Still, they "caution the reader that a number of experimental breakthroughs will be necessary before we can construct a universal quantum processor capable of surpassing present-day supercomputers" (Orús et al., 2019).

In the paper "Quantum computing for finance: overview and prospects", Orús et al. (2019) review "ways in which quantum computing could disrupt finance". The paper discusses quantum optimization, quantum machine learning, and quantum amplitude estimation. Some of the applications discussed are optimal trading trajectories, optimal arbitrage opportunities, optimal feature selection in credit scoring, risk analysis, and pricing of financial derivatives. The quantum machine learning techniques explored are quantum Boltzmann machine, quantum principal component analysis (PCA), quantum support vector machine (SVM), quantum reinforcement learning, and quantum amplitude estimation (QAE).

Egger et al. (2020) extend the previous summary, and additionally include demonstrations of the quantum algorithms using Qiskit. The mentioned QML techniques with applications on finance discussed are: QAE in the context of credit risk to determine the economic capital requirement, and optimization algorithms such as variational quantum eigensolver (VQE) and quantum approximate optimization algorithm (QAOA), in the context of portfolio optimization. The paper also discusses technical challenges in quantum computing, including loading data to quantum computers, quantum error correction, precision, and sample complexity.

In a review paper, Mironowicz et al. (2026) provide an overview of potential quantitative finance use cases for quantum algorithms, and for hybrid quantum-classical methods. The use cases mentioned are portfolio optimization, market prediction and trading, option pricing, and risk management. For each application, the authors present current classical approaches and alternative quantum machine learning methods. The expected advantages and existing limitations are also indicated. The attitude of the authors is among the same lines as in Orús et al. (2019); they highlight the competitive performance of QML algorithms in finance, while recognizing that scalability, hardware limitations, and algorithmic complexity are remaining challenges that limit the full potential of QML in finance.

We note that none of these review papers mentions quantum reservoir computing. Yet there is existing literature on QRC applications in finance. For example, the paper "Quantum Reservoir Computing for Realized Volatility Forecasting" (Li et al., 2026). In their QRC approach to realized volatility forecasting, the model outperformed benchmark models across various metrics. Another example is the paper "A Quantum Reservoir Computing Approach to Quantum Stock Movement Forecasting in Quantum-Invested Markets" Otieno et al. (2026) which presents a QRC framework for time-series forecasting. The model is applied to predict the future daily closing volumes of publicly traded companies. To the extent of our research, these were the only papers found where QRC is applied in a quantitative finance context.

4.3 Classical and quantum reservoir computing

Classical reservoir computing is a machine learning approach that transforms input data into a feature representation that captures relationships between inputs over time (Nakajima and Fischer, 2021). A separate readout layer is trained on said feature representation to make predictions, while the reservoir itself remains unchanged. In this context, features are attributes of a transaction (such as its amount, timestamp, and terminal ID) while the reservoir acts as a memory that captures relationships between these inputs over time. A high-level diagram of the reservoir computing workflow is seen in Figure 2.

Quantum reservoir computing follows the same principle but uses a quantum system as the reservoir to transform input data into a feature representation (Nakajima and Fischer, 2021). A QRC model consists of three main components: an input encoding method, a quantum reservoir, and a classical readout. The Input encoding maps the input data to quantum states. The quantum system then processes these states through its dynamics, before being measured. The measurement outcomes are converted into features for a classical readout, such as linear or logistic regression, which is trained to make predictions.

QRC can be applied to time-dependent tasks because the reservoir can retain information about

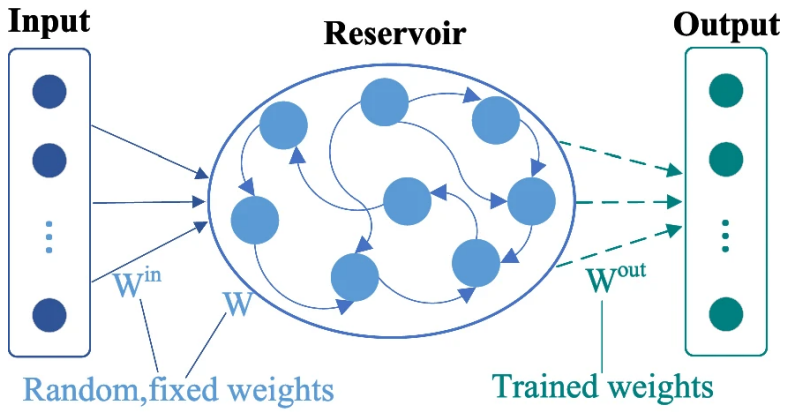


Figure 2: Diagram of classical reservoir computing. Reproduced from Wu et al. (2024), Fig. 1.

previous inputs. Its dynamics can also transform inputs into outputs that may capture nonlinear relationships in the data, allowing the readout to model patterns that may not be linearly separable in the original input space. This makes it applicable to time-series prediction, nonlinear system modeling, and classification (Nakajima and Fischer, 2021). However, its performance depends on whether the reservoir dynamics, input encoding, and resulting measurements capture information relevant to the task. Any advantage over classical reservoir computing must therefore be demonstrated through empirical comparison on the target task.

5 Solution concepts and selection

The use of QRC for financial fraud detection was a requirement from Quobly, so the concepts considered are not competing technologies but different ways of implementing QRC for binary prediction. They differ mainly in where the memory of past transactions is kept: in a quantum state that persists over time, or in a classical buffer. In this section we first state which history the model should see, and the constraints that any concept has to satisfy. We then describe the three solution concepts considered. We state the selected concept, explain the rational for choosing it, and present the resulting protocol. This section refines the selection presented during the midterm review.

5.1 Scope: which history the model sees

Transactions arrive as a single chronologically ordered stream, so consecutive transactions generally belong to different cards and terminals: the history of any one card or terminal is interleaved with that of many others. Fraud can show up at different stages in this stream: in the current transaction alone (e.g. an unusually large amount), in the recent history of a terminal (a compromised terminal), or in the recent history of a card (a compromised card). Any temporal model, quantum or classical, must therefore decide which past transactions form the relevant context for a given transaction.

We chose to build this context from the customer's own recent transactions. This follows the question at the heart of the problem posed by the company: given the recent history of this card, does the new transaction look fraudulent? It is a choice, not a necessity. Windows keyed by terminal would target compromised terminals instead, and classical engineered features can summarize both histories at once. The choice has two consequences:

1. Any memory the quantum reservoir provides can only help on card-level fraud. Fraud that is visible in the current transaction does not need memory, and terminal-level fraud is invisible in a card's history. For this reason, we report performance separately for each fraud category (Section 5).

2. The relevant history is short. In our data, a compromised card stays compromised for 14 days (Section 6), so the transactions that matter are at most the last few tens of the same customer, not the whole stream.

5.2 Design constraints

In addition to this scope, three constraints drove the selection.

1. **Near-term hardware.** Current quantum processors, including Quobly's spin-qubit devices, offer few qubits with limited connectivity, imperfect gates and finite coherence times. Every extra layer of gates costs fidelity, and mid-circuit measurements with fast classical feedback are costly. Shallow circuits on a few qubits, without mid-circuit operations, are therefore preferred.

2. **Expectation values are estimated from repeated measurements.** The features given to the readout are expectation values of observables. Estimating each one to precision ε takes on the order of $1/\varepsilon^2$ shots, and every projective measurement destroys the state. Each shot therefore has to prepare the state again from scratch, including every input that state is supposed to remember.

3. **A quantum advantage has to be attributable.** If the QRC model detects fraud, we must be able to tell whether the gain comes from the quantum dynamics (in particular their memory) or simply from giving the readout more inputs. A concept is more useful when it comes with natural controls.

5.3 Concepts

Three concepts were identified to solve our problem. Each one is detailed below, and then compared in Table 3.

Concept 1: persistent-state QRC. This is the textbook formulation (Mujal et al., 2021). A single quantum state is driven continuously by the incoming time series and read out after every input. Fading memory comes from dissipation, from partial resets or from weak or mid-circuit measurements (Mujal et al., 2023). In principle the memory can reach arbitrarily far back. In our setting, however, this concept runs into the following problems:

- Driving one register with the interleaved stream would mix unrelated customers. Keeping one register per customer would mean keeping thousands of quantum states coherent for unreasonable times.
- Because every shot destroys the state (constraint 2), producing the features for transaction t means replaying the whole history up to t in every shot. The cost per prediction then grows with the length of the stream, unless fast mid-circuit measurement with feedback is available (constraint 1).

Concept 2: quantum extreme learning machine (QELM). The quantum system has no recurrence. Each sample is encoded once, evolved under fixed dynamics and measured, and the measured expectation values act as an untrained nonlinear feature map for a classical readout (Mujal et al., 2021; Kawanabe et al., 2026). All memory is classical, but two variants of this concept can be identified:

(2a) Only the current transaction is encoded, with engineered features (rolling statistics of the customer and the terminal) carrying the history, or

(2b) The whole window of L transactions is encoded at once, on separate qubits. QELM is shallow and cheap per shot, which fits constraints 1 and 2 well. It cannot, however, provide any quantum memory, which is what the company hoped QRC would contribute. This variant also needs a number of qubits that grows with L, which quickly exceeds what near-term hardware offers.

Concept 3: sliding-window QRC (selected). For each transaction t, the customer's last L transactions are encoded *one after another* into a quantum register, which evolves under fixed dynamics between encodings. The register is measured once, at the end of the window, and fully reset before the next window. Within a window, the quantum state carries the memory of earlier inputs, as in Concept 1. Across windows, the model is feed-forward, as in Concept 2.

	1. Persistent-state QRC	**2. QELM**	**3. Sliding-window QRC**
Where the memory lives	Quantum state	Classical (features or window)	Quantum state, bounded by the classical window
Per-customer history	Needs one register per customer	Yes	Yes, via per-customer windows
Circuit depth per shot	Grows with the stream	One layer	L layers
Qubits	Fixed	Fixed (2a), or growing with L (2b)	Fixed
Mid-circuit operations	Required for fading memory	None	Optional
Gain attributable to quantum memory	Hard to isolate	No quantum memory	Yes, with Concept 2 as control

Table 3: Comparison of the three QRC concepts.

5.4 Selection

We selected Concept 3 as the main solution, for the following reasons.

- It is the only concept that keeps a *quantum* memory of past transactions while remaining feasible on near-term hardware. The circuit depth is bounded by L instead of by the length of the stream, so the error budget of a window can be controlled by choosing L.

- It matches our scope. Windows are built per customer, so the reservoir only ever sees one card's history, and L can be chosen to cover the horizon of card-level fraud and no more.
- It fits shot-based estimation. Each window is an independent circuit of fixed depth, so every shot starts from the default state, which is the between-shot reset that the hardware performs anyway. Windows can be batched and executed in parallel.
- It does not depend on mid-circuit measurements. Resetting part of the register between encodings is an option that we evaluate, not a requirement (See Section 5.5).

Concept 2 was not discarded; it became the control that makes the results of Concept 3 interpretable (constraint 3). Concept (2a), QELM on engineered features, tests whether a quantum feature map adds anything on top of strong classical features. Concept (2b), QELM on the whole window, receives exactly the same inputs as Concept 3 but has no recurrence. If Concept 3 improved with L only as much as variant (2b) does, its gain would come from seeing more transactions, not from its quantum memory. Concept 1 was rejected for the reasons given in Section 5.3.

5.5 Sliding-window QRC protocol

We refer to the implementation of Concept 3 as *sliding-window QRC*. It turns a stream of transactions into fraud predictions in four steps, described below and illustrated in Figure 3.

1. Windowing. Let $F \in \mathbb{N}$ be the number of real-valued features describing a transaction, $T \in \mathbb{N}$ the total number of transactions, and $\mathbf{x}_t \in \mathbb{R}^F$ be the vector of features describing the transaction t, for $t \in \{1, \dots, T\}$. Note that each $\mathbf{x}_t$ is scaled to $[0,1]^F$. Each transaction has a label $y_t \in \{0,1\}$, where $y_t = 1$ means that the transaction is fraudulent.

$$\text{Transaction data set: } \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_T^\top \end{pmatrix} \in \mathbb{R}^{T \times F}, \qquad \text{Label vector: } \begin{pmatrix} y_1 \\ \vdots \\ y_T \end{pmatrix} \in \{0,1\}^T$$

For each transaction $t \in \{1, \dots, T\}$, we build the window $W_t = (\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t)^\top$ of the customer's last L transactions, ordered from oldest to newest and ending with the transaction to classify.

$$\text{Window of size } L\text{: } W_t = \begin{pmatrix} \mathbf{x}_{t-L+1} \\ \vdots \\ \mathbf{x}_t \end{pmatrix} \in \mathbb{R}^{L \times F}$$

If $t < L$, the customer does not yet have L past transactions to fill the window. In that case we pad the window at the start by repeating the customer's oldest transaction until it reaches length L.

2. Quantum feature map. A register of n qubits is split into n_{in} input qubits, which receive the data, and n_{mem} memory qubits, which never receive it. Naturally, we have $n = n_{\text{in}} + n_{\text{mem}}$. The register starts in the default state $\rho_0 = (|0\rangle\langle 0|)^{\otimes n}$. Given a window $W_t = (\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t)^\top$, each vector $\mathbf{x}_l$ (for $l \in \{t-L+1, \dots, t\}$) is then processed by one layer, involving three steps:

1. *Input reset. (Only for $l > t - L + 1$.)* The input qubits are reset to $|0\rangle$, and the memory qubits are left untouched. We denote this channel as $\mathcal{R}$.
2. *Encoding.* A unitary $E_{\mathbf{x}}$ acting only on the input qubits writes the features of $\mathbf{x}_l$ into their state.
3. *Evolution.* The whole register evolves under a fixed Hamiltonian H for a time τ, i.e. under $U = e^{-iH\tau}$. This step is where information flows from the input qubits to the memory qubits.

Therefore, one layer is the channel

$$\Phi_{\mathbf{x}}(\rho) = U E_{\mathbf{x}}\, \mathcal{R}(\rho)\, E_{\mathbf{x}}^\dagger U^\dagger,$$

and the state at the end of the window is

$$\rho^{(t)} = \Phi_{\mathbf{x}_t} \circ \cdots \circ \Phi_{\mathbf{x}_{t-L+1}}(\rho_0).$$

Finally, the expectation values of a fixed set of M observables $\{O_k\}_{k=1}^{M}$ form the feature vector of transaction t, denoted by $\mathbf{m}^{(t)} = (m_1^{(t)}, \ldots, m_M^{(t)})$ and defined as

$$m_k^{(t)} = \mathrm{Tr}\left(O_k\, \rho^{(t)}\right), \qquad k = 1, \ldots, M.$$

On hardware, each $m_k^{(t)}$ is estimated by preparing $\rho^{(t)}$ and measuring it a finite number of times (shots).

3. Classical readout. A logistic regression is trained on the feature vectors $\mathbf{m}^{(t)}$ of the training set. It gives each transaction a continuous score

$$\tilde{y}_t = \sigma\left(\mathbf{w}^{\mathsf{T}}\mathbf{m}^{(t)} + b\right) \in [0,1], \qquad \sigma(a) = \frac{1}{1+e^{-a}},$$

where the weights $\mathbf{w} \in \mathbb{R}^M$ and the intercept b are the only trained parameters of the model.

4. Decision. A decision threshold, chosen on the validation set, turns each score $\tilde{y}_t$ into a binary prediction $\hat{y}_t \in \{0,1\}$.

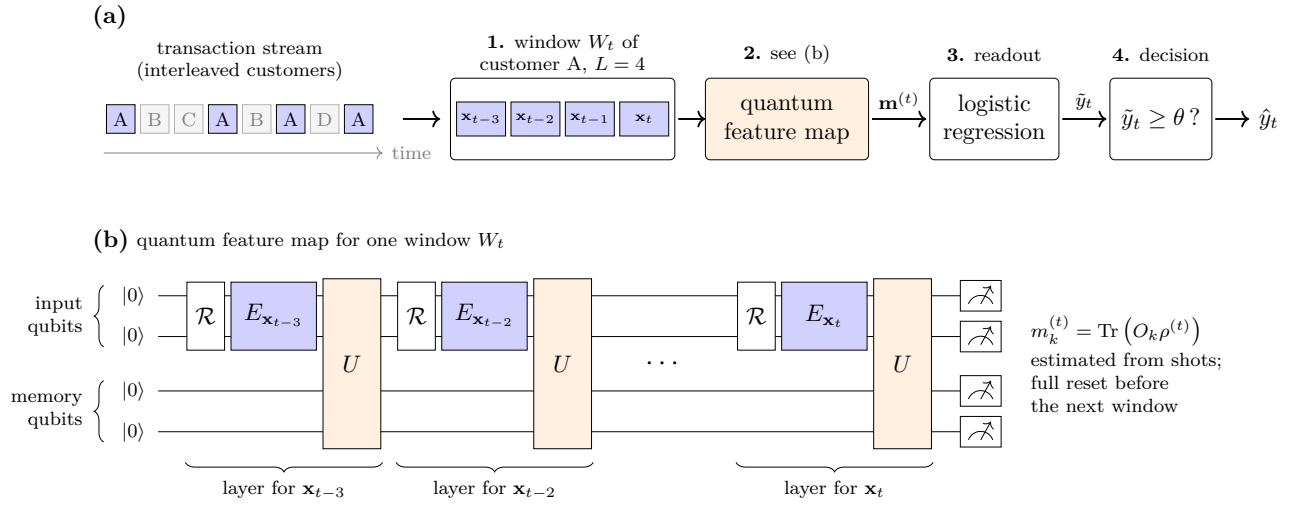


Figure 3: Our sliding-window QRC.

Fig. 3(a) illustrates the four steps our protocol. For each transaction t, the window W_t of the customer's last L transactions is mapped to a feature vector $\mathbf{m}^{(t)}$ by the quantum feature map. Then, a logistic regression turns the feature vector $\mathbf{m}^{(t)}$ into a score $\tilde{y}_t$. Finally, in the decision step, the threshold θ turns the score $\tilde{y}_t$ into a prediction $\hat{y}_t$ using the decision rule $\hat{y}_t = \mathbb{1}_{\{\tilde{y}_t \geq \theta\}}$

Fig. 3(b) illustrates what happens inside the quantum feature map: each transaction of the window is processed by one layer, made of a reset $\mathcal{R}$ of the input qubits to $|0\rangle$, the encoding $E_{\mathbf{x}}$ on the input qubits, and the evolution $U = e^{-iH\tau}$ of the whole register. The memory qubits are never reset within the window: they carry information about earlier transactions from one layer to the next. The register is measured only at the end of the window, and fully reset before the next one. For illustration, the figure uses $L = 4$ and $n_{\mathrm{in}} = n_{\mathrm{mem}} = 2$.

Two kinds of reset. The protocol involves two different resets, which should not be confused. The *full reset* of the register between windows (and between shots) is what makes each window processing independent. It is unavoidable on hardware anyway, since measuring destroys the state, and it does

not need to be fast. The *input reset* within a window, on the other hand, is a mid-circuit operation. On most platforms it means measuring the input qubits and flipping them back to $|0\rangle$ with fast classical feedback, which is costly (constraint 1). Without it, the evolution within a window is unitary: earlier inputs never fade but are only scrambled across the register.

QRC or QELM? Because the register is fully reset between windows, our protocol differs from the textbook QRC of Concept 1, in which a single persistent state is continuously driven by the incoming time series and a fading memory gradually erases the influence of past inputs. In this respect it is closer to recent time-delayed QELM proposals (Kawanabe et al., 2026). Within one window, however, it is genuinely QRC: a single quantum state is driven by L successive inputs. As in both paradigms, QRC and QELM, nothing quantum is trained. The circuits are fixed when they are built and only the classical readout is fitted.

Why this structure. The split into input and memory qubits makes the memory explicit. New data only ever overwrites the input qubits, so the memory qubits hold information about earlier transactions only through the evolution U, which spreads it across the register before the input qubits are used again (Li et al., 2026; Hamhoum et al., 2025). Keeping the readout linear (logistic) keeps training convex and cheap. It also means that any nonlinearity in the model, and any memory beyond the current transaction, comes from the quantum reservoir and not from the readout. This makes the comparison with classical baselines, and with Concept 2, meaningful.

Modularity. The protocol leaves the encoding $E_{\mathbf{x}}$, the Hamiltonian H, the evolution time τ, the register sizes, the window length L, and the observables $\{O_k\}$ open. The options we implemented, and how they were compared, are described in the next Section.

6 Methodology

The experimental methodology was designed to compare sliding-window QRC with classical alternatives under the same temporal data, selected features, customer windows, and evaluation protocol. Section 6.1 covers the data: synthetic transaction generation, chronological train-validation-test partitioning, historical feature construction, preprocessing and customer windows. Section 6.2 describes how the QRC protocol of Section 5.5 was implemented, and Section 6.3 the classical baselines and the reservoir benchmark it is compared with. Section 6.4 covers model training, validation-based model and threshold selection, and the final evaluation on a held-out future test period. Finally, Section 6.5 presents the software that runs all of these steps. Detailed data-generation rules, preprocessing definitions, and evaluation algorithms are provided in Appendix D; the rationale for dataset selection and splitting is documented in Appendix B.

6.1 Data

The evaluation of fraud detection models depends not only on the models themselves, but also on the data used to represent fraudulent behaviour. In particular, assessing QRC requires a dataset that allows us to examine how transaction patterns over time influence detection performance. This chapter establishes the data foundation for the experiments that follow.

6.1.1 Dataset choice

The dataset choice was guided by the temporal nature of the research question: detecting whether a transaction is fraudulent given the recent history of the same customer. Two initially suggested public credit-card fraud datasets were assessed, we refer to them as “The ULB credit-card fraud", and “The Credit Card Fraud Detection Dataset 2023". The ULB credit-card fraud dataset provides transaction timing and anonymized transaction-level attributes but no persistent card or customer identifier, preventing the construction of customer-specific histories. The Credit Card Fraud Detection Dataset 2023, meanwhile, does not provide transaction timestamps. Although these datasets can support transaction-level classification, neither was suitable for the intended sequential task as provided. No accessible real-world dataset meeting the combination of labelled fraud outcomes, customer identifiers, and temporal histories was identified among the sources considered. The team therefore adopted a synthetic-data approach; the dataset assessment is documented in Appendix B.

6.1.2 Transaction generation

The generated transactions used were adapted from the *Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook* (Le Borgne et al., 2022). The configuration generated transactions for 1,000 customers and 10,000 terminals over 365 days beginning on 1 January 2025, with a customer-terminal association radius of 5 and random seed 0. Individual customer profiles determine location, typical transaction amount, spending variability, and expected transaction frequency. Daily transaction counts are drawn from a Poisson distribution, and transaction amounts are sampled from each customer's spending profile. Three fraud mechanisms were introduced:

1. Scenario 1: labels transactions with an amount greater than 220 as fraudulent.
2. Scenario 2: compromises two (2) terminals per day for 28 days, labelling transactions at the affected terminals as fraud.
3. Scenario 3: compromises six (6) customers per day for 14 days, marking approximately one third of their transactions as fraudulent and multiplying their amounts by five.

The latter two scenarios create temporal customer and terminal-level dependencies relevant to reservoir computing. Because these rules are synthetic, the resulting dataset provides a reproducible benchmark for relative model comparison, not evidence of production-level detection performance.

6.1.3 Chronological split

The transaction stream was partitioned using `qrc/datasets/split_dataset.py` with its chronological boundaries: training from 1 January to 12 September 2025, validation from 13 September to 5 November 2025, and testing from 6 November to 31 December 2025. These periods span 255, 54, and 56 days, respectively (approximately 69.9%, 14.8%, and 15.3% of the calendar period). A common date-based split was chosen to evaluate generalization to future transactions while avoiding the temporal leakage associated with random splitting. The alternatives considered and the rationale for the final choice are presented in Appendix B.

6.1.4 Features, preprocessing and windows

To construct historical features, the three partitions are temporarily combined into a chronological stream and the engineered rows are then assigned back to their original periods. The fraud-scenario identifier is excluded from predictive features and retained for evaluation only. Customer rolling features exclude the current transaction, while terminal-risk features incorporate a seven-day delay before fraud labels become available. Missing values are imputed with training-set medians, and Min-Max scaling is fitted on training data only, so that every feature lies in $[0, 1]$.

Finally, transactions are grouped into ordered, customer-specific windows W_t of length L, ending with the transaction to be classified (Section 5.5). Earlier history may cross a split boundary, but future transactions cannot enter a past prediction. The same windows are given to the QRC and to every baseline of Section 6.3.

Raw-feature signal assessment. To assess the predictive information available before constructing historical features, we conducted a supplementary transaction-level ablation using logistic regression. Models were fitted on the training set and evaluated by average precision (AP) on the validation set. For each fraud category, fraudulent transactions of that category were compared only with legitimate transactions; other fraud categories were excluded. The candidate inputs were log transaction amount, timestamp-derived variables (cyclic time of day, cyclic day of week, and calendar day), and their combination. The customer category was additionally restricted to scenario 3 fraudulent transactions with a final amount equal or below 220. Training-only median imputation and standardization were applied, with class-balanced logistic regression. This analysis was used as a diagnostic of marginal raw-feature signal, not as a substitute for sequence-model evaluation. Details are given in Appendix C.

6.2 Implementation of the sliding-window QRC

The protocol of Section 5.5 fixes the structure of the model (input reset $\mathcal{R}$, encoding $E_{\mathbf{x}}$ and evolution $U = e^{-iH\tau}$ for each transaction of the window, then measurement and a logistic readout) but leaves its components open. This section describes how we implemented it and which options we implemented for each component, so that the choice could be made experimentally. Table 4 summarises them.

6.2.1 Circuit construction

The circuits are built with Qiskit. For each window W_t, we build one quantum circuit on $n = n_{\text{in}} + n_{\text{mem}}$ qubits, with n_{in} equal to the number of features by default and $n_{\text{mem}} = 2$ by default. For each of the L transactions, processed from oldest to newest, the circuit applies the following sequence: a reset of the input qubits (starting from the second transaction, as they are initially in $|0\rangle$), the transaction's encoding circuit on the input qubits, and finally the reservoir circuit on the full register. The reservoir circuit is built once, from a fixed seed, and reused unchanged for every transaction and every window. Nothing in it is trained.

The input reset can be switched off, in which case the circuit within a window is unitary. With the reset, the circuit is no longer unitary, and it is simulated with density matrices instead of state vectors, which squares the memory cost. Each experiment in Section 7 states whether the reset is used.

6.2.2 Input encoding

All encodings use single-qubit rotations, which are native gates on spin qubits. After scaling, a feature value $x_{t,i}$ is mapped to the angle $\Theta = \lambda x_{t,i}$, with a scaling factor $\lambda = \pi$ by default. We implemented three encoding methods commonly used in QML applications (IBM Quantum, 2025):

- **Angle encoding.** Each feature is written on its own input qubit as a rotation $R_Y(\lambda x_{t,i})$. If there are more input qubits than features, the features are repeated cyclically to fill them.
- **Dense angle encoding.** Two features share one qubit, through rotations about two different axes (R_Y then R_Z by default). It halves the number of input qubits needed for a given number of features.
- **Data re-uploading.** The angle encoding is applied several times to the same transaction, with fixed random R_Z rotations in between.

By default the encoding is chosen automatically: re-uploading if more than one encoding layer is requested, dense angle encoding if there are fewer input qubits than features, and angle encoding otherwise.

6.2.3 Quantum reservoir

We implemented two families of reservoirs:

The **random-circuit reservoir** repeats, for a given depth, a ring of entangling gates (CNOT, or controlled-R_Y with random angles) followed by random R_Y and R_Z rotations on every qubit. It reproduces the random reservoirs of the QRC-Lab toolbox (Dos Santos, 2026). Since it has no physical parameters, it serves as a control for whether the structure of a Hamiltonian brings anything.

The **Hamiltonian reservoirs** implement $U = e^{-iH\tau}$ for a fixed Hamiltonian H. Unlike randomly constructed gate sequences, these reservoirs are defined by a mathematical model of the interactions between qubits. Their parameters, such as coupling strengths, external fields, and evolution time, can be adjusted to investigate how different quantum dynamics affect the reservoir's ability to process information. The four ones that we implemented and are presented in Appendix F are: Transverse-Field Ising Model (TFIM), Longitudinal-Field Ising Model (LFIM), Native Ising Model (NIM) and Native Exchange Model (NEM).

Comparing these two families allows us to assess whether the choice of reservoir architecture influences fraud-detection performance. On gate-based hardware, the $e^{-iH\tau}$ has to be decomposed into gates. For that, we use a first-order Lie-Trotter product formula with a configurable number of steps (evolution time $\tau = 1$ by default). The default is two Trotter steps: the residual error does not matter here, and using more makes the circuits deeper. Further details on the implemented reservoirs and the Trotterization formulae are provided in Appendix F.

6.2.4 Observables

The observables $\{O_k\}$ are Pauli strings, built as a Qiskit `PauliList` [1] from a short specification. Single-qubit observables (X, Y and/or Z on every qubit) are the cheapest to measure, but two-body correlators (ZZ, XX, ..., on nearest neighbours or on all pairs) increase the information-processing capacity of the reservoir (see subsection 6.3.4). Any explicit list of Pauli strings can also be given (for instance all X-, Y- and Z-strings, as used in Section 7). Observables that are diagonal in the same basis are measured from the same shots, so the cost on hardware should only depend on the number of measurement bases rather than on the number of observables. In particular, all Z-type observables need a single basis, while adding X and Y observables triples the number of circuits to run.

[1] `https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.PauliList`

6.2.5 Simulation backends

The feature vectors $\mathbf{m}^{(t)}$ can be computed by several interchangeable simulators, with increasing realism:

- **Exact.** Uses `Qiskit-Aer` [2] to compute exact expectation values. To speed up our simulations, we also implemented a `NumPy` [3] simulator that reproduces the same results up to machine precision. It does so by computing the reservoir unitary once and reusing it to simulate multiple windows in parallel using dense matrix products. With systems of small size (up to 6 qubits), it helped reduce simulation time by one to two orders of magnitude.
- **Finite precision.** Each expectation value is estimated with a target standard error ε, also referred to as the precision, which corresponds to about $1/\varepsilon^2$ shots. For that, two options were implemented:
  - Adding a Gaussian noise of width ε on top of the exact values, or
  - Actually sampling the shots with `Qiskit-Aer` (much slower).
- **Hardware-aware emulation.** With SpinPulse Vermersch et al. (2026), the circuits are compiled into the control pulses of a spin-qubit chip and simulated with qubit-frequency noise (white, pink or quasi-static), a dephasing time T_2^*, optional coupling noise and dynamical decoupling, averaged over several noise realizations.

6.2.6 Classical readout

The feature vectors are standardized with statistics fitted on the training set only, and the readout is a scikit-learn logistic regression with L_2 regularization of strength α ($C = 1/\alpha$, $\alpha = 1$ by default). That is, we learn the $\mathbf{w}$ and b that minimize

$$-\sum_{t\in\text{train}}\Big[y_t\log\tilde{y}_t+(1-y_t)\log(1-\tilde{y}_t)\Big]+\frac{\alpha}{2}\|\mathbf{w}\|^2.$$

It is solved with the Newton-Cholesky solver, which converges to the optimum in a few iterations. The decision threshold is chosen on the validation set (see Section 6.4).

Axis	**Options**
Window length L	1 (QELM) up to 30
Input-qubit reset	on / off
Register	n_{in}, n_{mem}; linear, ring or all-to-all connectivity
Encoding	angle, dense, re-uploading (number of layers); scaling factor λ
Reservoir	`random-circuit` (CX or CRY entanglers, depth); TFIM; LFIM; NIM; NEX; number of Trotter steps, evolution time τ
Observables	any set of one- and two-body Pauli strings (e.g. Z; X, Y, Z; ZZ on nearest neighbours or all pairs), or an explicit list of Pauli strings
Simulation backend	exact (Qiskit Estimator or `NumPy`); finite precision (Gaussian noise or sampled shots); SpinPulse hardware-aware emulation with white, pink or quasi-static noise
Readout	logistic regression, regularization strength α

Table 4: Configuration space of the sliding-window QRC implementation.

6.3 Baselines and benchmarking methods

To assess what the quantum reservoir contributes, QRC is compared with three classical models. All of them use the same processed windows W_t, labels and chronological splits as QRC, so that differences in performance reflect the model and not the data.

[2] `https://qiskit.github.io/qiskit-aer/`
[3] `https://numpy.org/doc/stable/`

6.3.1 Direct logistic regression

The direct logistic-regression baseline flattens each window of dimensions $L \times F$ into an LF-dimensional vector. Then, it standardizes it using training-set statistics, and fits the same type of regularized logistic readout.

$$p_t = \sigma\left(\mathbf{w}^\top \mathbf{z}_t + b\right), \qquad \sigma(a) = \frac{1}{1+e^{-a}}. \tag{1}$$

Here, $\mathbf{z}_t$ is the standardized, flattened customer window, while $\mathbf{w}$ and b are weights learned from labelled training examples. The function σ converts the weighted sum into a score between zero and one. A larger score indicates that the model considers the transaction more suspicious.

This baseline tests whether the original transaction history is sufficient for fraud detection without a reservoir transformation.

6.3.2 Echo state network

The Echo State Network (ESN) is the classical reservoir-computing baseline. A fixed, randomly initialized leaky recurrent network processes the transactions sequentially. Its internal state is updated as:

$$\mathbf{h}_j = (1-a)\mathbf{h}_{j-1} + a\tanh\left(W_{\text{in}}\mathbf{x}_j + W_{\text{rec}}\mathbf{h}_{j-1} + \mathbf{b}_{\text{r}}\right), \tag{2}$$

where $j = 1, \ldots, L$ denotes the position of a transaction within the window, $\mathbf{x}_j$ represents the current transaction, and $\mathbf{h}_j$ is the network's internal memory.

The parameter a, called the leak rate, controls how much the memory changes after a new transaction. The function tanh introduces a nonlinear transformation. The matrices W_{in} and W_{rec} and the bias $\mathbf{b}_{\text{r}}$ are randomly initialized and remain fixed.

Only the logistic-regression readout applied to the final memory state $\mathbf{h}_L$ is trained. Comparison with the ESN therefore examines whether an improvement reflects reservoir-based temporal processing generally or the chosen quantum dynamics specifically.

6.3.3 Long short-term memory network

The Long Short-Term Memory (LSTM) baseline provides a trainable recurrent alternative. It processes the same customer window chronologically and produces a fraud score from the final hidden representation. Unlike QRC and ESN, its recurrent parameters are learned from training data. The memory update is represented by:

$$\mathbf{c}_j = \mathbf{f}_j \odot \mathbf{c}_{j-1} + \mathbf{i}_j \odot \widetilde{\mathbf{c}}_j, \qquad \mathbf{h}_j = \mathbf{o}_j \odot \tanh(\mathbf{c}_j). \tag{3}$$

Here, $\mathbf{c}_j$ represents the running memory, $\widetilde{\mathbf{c}}_j$ represents candidate new information, and $\mathbf{f}_j$, $\mathbf{i}_j$, and $\mathbf{o}_j$ are the forget, input, and output gates, respectively. The symbol $\odot$ denotes element-wise multiplication.

A final linear layer followed by a sigmoid transformation converts the last hidden state $\mathbf{h}_L$ into a fraud score between zero and one.

Unlike the fixed reservoirs, the LSTM's memory updates are learned by minimizing binary cross-entropy loss. Training is stopped based on validation loss to reduce overfitting.

The LSTM therefore provides a comparison with a fully trainable sequential model.

6.3.4 Information processing capacity

Information processing capacity (IPC) measures a reservoir's ability to reconstruct functions of past inputs. The IPC procedure used is an adaptation of Martínez-Peña et al. (2020). An independent input sequence s_t is sampled uniformly from $[-1, 1]$. The target functions to be learned by the QRC are constructed from products of normalized Legendre polynomials evaluated at different time delays and grouped by polynomial degree.

The input sequence is encoded into the quantum reservoir. The encoded quantum state evolves under the reservoir dynamics for a specified duration or number of circuit steps. Measurements are collected for each input window to form a feature vector describing the reservoir's response. These feature vectors are used as inputs to a linear readout, which is trained to predict each target function from the measurement data. For each target function $\bar{y}$, the capacity is computed as:

$$C(\bar{y}) = 1 - \frac{\mathrm{MSE}(\bar{y}, \hat{y})}{\mathrm{Var}(\bar{y})} \tag{4}$$

where $\bar{y}$ is the true target value, and $\hat{y}$ is the readout's predicted value. To prevent finite-sample artifacts and numerical noise from accumulating, a threshold ϵ is chosen: any capacity $C(\bar{y}) \leq \epsilon$ is omitted. The individual capacities are then aggregated by polynomial degree for the degree-wise capacity C_d:

$$C_d = \sum_{\substack{\bar{y} \in \mathcal{Y}_d \\ C(\bar{y}) > \epsilon}} C(\bar{y}) \tag{5}$$

where $\mathcal{Y}_d$ is the set of target functions of degree d, comprising all combinations of input delays whose polynomial degrees sum to $d = \sum_j d_j$. Specifically, C_1 quantifies the reservoir's linear memory depth, while $C_{d \geq 2}$ reflects its nonlinear memory and capacity to compute cross-time interactions between past inputs. The total information processing capacity of the system is the sum over all evaluated degrees:

$$C_{\mathrm{total}} = \sum_{d=1}^{d_{\max}} C_d \tag{6}$$

$$C_{\mathrm{norm}} = \frac{C_{\mathrm{total}}}{N_{\mathrm{obs}}} \tag{7}$$

The total capacity is bounded by the number of linearly independent observables, $C_{\mathrm{total}} \leq N_{\mathrm{obs}}$. For comparison across configurations with different numbers of observables, the total capacity is normalized by the number of observables (Equation 7). This gives a normalized capacity in $[0, 1]$. Detailed derivations and implementation mathematics about Legendre Polynomials and IPC are provided in Appendix E.

6.4 Evaluation methods

The training split is used to fit model parameters and preprocessing transformations; the validation split supports configuration assessment and decision-threshold selection. The test period represents future transactions held apart from these selection decisions. Average Precision (AP) is the primary threshold-independent ranking metric, supplemented by ROC-AUC. The available hyperparameter-search procedure optimizes validation metrics and can also consider circuit-resource costs; exact search budgets and selected configurations belong with the experimental results rather than being assumed to equal command-line defaults.

For each fitted classifier, the operating threshold is determined from its own validation scores by minimizing false positives subject to a minimum fraud-recall requirement:

$$\tau^* = \arg\min_{\tau} \mathrm{FP}_{\mathrm{val}}(\tau) \qquad \text{subject to} \qquad \mathrm{Recall}_{\mathrm{val}}(\tau) \geq r_{\min}. \tag{8}$$

The implementation defaults to $r_{\min} = 0.80$. The selected numerical threshold is fixed for that model and applied unchanged to the test scores. Consequently, threshold-dependent validation figures describe the chosen operating point, while test figures assess its performance on subsequent transactions. The implementation can record test metrics during automated searches; to maintain an unbiased final comparison, those values must not be used to choose the winning configuration.

Performance at the operating threshold is assessed using precision, recall, false-positive rate, F1 score, Matthews correlation coefficient and balanced accuracy. Card Precision@K additionally measures daily card-level ranking under a limited review capacity. Where fraud-scenario labels are available, AP is evaluated separately for amount fraud (TX_AMOUNT > 220), terminal- and customer-

fraud at or below that threshold. These category labels are retained only for evaluation, not as model inputs. Definitions, grouping rules and configuration defaults are detailed in Appendix D. Since the fraud mechanisms are synthetic, the results indicate relative performance within this controlled benchmark and do not alone establish operational deployment validity.

6.5 Software and reproducibility

All the experiments in this report run on one shared codebase, developed by the whole team in a single Git repository hosted on GitHub.[4] Every result in this report comes from code that is versioned, tested and can be re-run from a single command.

Our sliding-window QRC and everything around it is organised as an installable Python package, `qrc`, declared in a standard `pyproject.toml` (Python $\geq$ 3.11). One command, `pip install -e .`, installs the package and every dependency (Qiskit, Qiskit Aer, SpinPulse, scikit-learn, Optuna, etc). Each step of this section is one group of modules:

- `qrc.datasets`, `qrc.features`, `qrc.processing`, `qrc.sequences`: the data of Section 6.1 (transaction simulation, splits by date, feature engineering, scaling and per-customer windows).
- `qrc.encodings`, `qrc.reservoirs`, `qrc.hamiltonians`, `qrc.trotter`, `qrc.observables`, `qrc.backends`: the QRC of Section 6.2.
- `qrc.baselines` and `qrc.benchmarks`: the classical baselines and the reservoir benchmarks of Section 6.3 (memory capacity, parity, NARMA, IPC).
- `qrc.readout`, `qrc.decision`, `qrc.evaluation`: the classical readout, the choice of decision threshold on validation, and the metrics of Section 6.4.

Because the simulation backends share one interface, the same experiment can be run exactly, with shot noise or with hardware-level noise by changing a single option, and a new reservoir or encoding is immediately available to every experiment. Note that for this structure we were inspired by the QRC-Lab toolbox (Dos Santos, 2026), a resource that was suggested to us by the company coaches. It also helped us verify our implementation in our first steps: we checked that our pipeline could reproduce their results on three standard benchmark tasks: memory capacity, parity, NARMA.

Experiments are run through a command-line tool, `experiments/cli.py`, with one command per model (`baseline`, `lstm`, `esn`, `qrc`). Every model goes through the same data preparation, the same readout and the same metrics, so the comparisons in this report differ only in the model. Two tools build on it: `experiments/sweep.py` runs a list of configurations, and `experiments/tune.py` runs the Optuna hyperparameter searches. Each run writes its configuration and metrics to disk (`config.json`, `metrics.json`), including the seeds and the Git commit it ran on. A simulated run is fully determined by its options and seeds, so any number in this report can be traced back to the exact code and command that produced it.

Throughout the project, we tried to make the best use of Git by delivering new work through detailed commits (>150) and pull requests (>20).

The package comes with a test suite (`pytest`, about 140 tests). Besides unit tests of the individual components, it contains reference tests checking that all simulation backends produce the same results up to machine precision, as well as end-to-end tests that run the CLI tool, the sweeps and some hyperparameter search on a small simulated dataset. That way, it is unlikely that a change in one module silently breaks an experiment.

Although anyone can run the test suite locally with the simple `pytest` command, it runs automatically on GitHub Actions on every pull request and on every push to `main`, on Python 3.11 and 3.12.

We also used GitHub Actions as compute for the largest studies. A study is written as a list of CLI commands; a workflow splits the list over 20 parallel runners and collects the results into a single artifact.

[4] `https://github.com/zulee1711/cc_fraud_detection_using_QRC`

7 Results

In this section, we present some of the results we obtained when investigating our sliding-window QRC and its QELM variant, in the context of fraud detection and silicon spin-qubit hardware.

7.1 Memory retention of the QRC

These first results report the IPC (see Subsection 6.3.4) of our sliding-window QRC. Capacities are broken down into linear memory (Degree 1), quadratic memory (Degree 2), cubic memory (Degree 3), and total capacity. The experiments are tested across three backends: SpinPulse, exact estimation, and finite-shot estimation (shots). Each configuration is tested with multiple seeds to account for variability between runs.

7.1.1 Delay and window length

This first sweep includes five reservoir configurations (random-circuit plus the four Hamiltonian reservoirs presented in Appendix F), four input window lengths ($L \in \{5, 10, 15, 20\}$), and a set of maximum delays for each window. The maximum delays used are $\tau_{max} \in \{1, 2, 3, 4, 5\}$, together with all delays up to $L - 1$ for each smaller input window length L. The change in total capacity across maximum delays and window sizes is shown in Figure 4, while the full degree-wise IPC results are provided in Appendix subsection I.1. In that experiment, note that we are measuring the Z- and ZZ-Pauli strings.

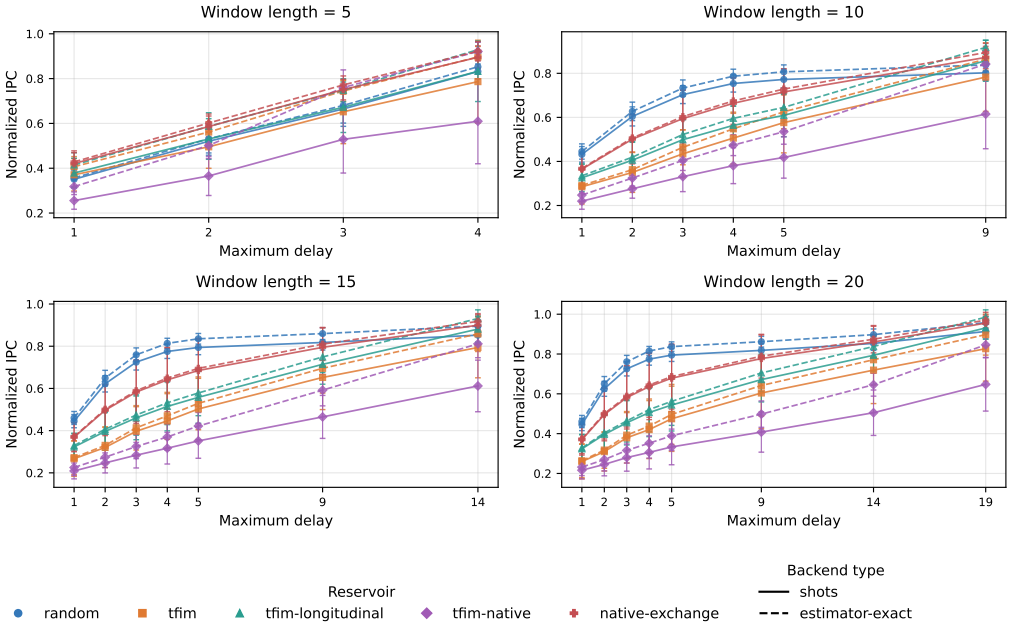


Figure 4: Normalized total information processing capacity as a function of the maximum input delay for window lengths $L \in \{5, 10, 15, 20\}$. Each panel corresponds to one window length and compares different reservoir Hamiltonians and backend types. Solid and dashed lines represent the shots and exact-estimator backends, respectively. Error bars indicate the standard deviation across seeds. All Hamiltonian models use 2 Trotterization steps and the random reservoir uses a circuit depth of 4.

Total IPC increases with maximum delay because it adds up the memory scores of each past step. At delay 1, it only counts the most recent input, so the score starts low (0.2–0.45). As the delay increases, it adds the memory of older inputs still inside the window, pushing the total score up toward 0.8–0.95. Reaching close to 1.0 means the qubits use almost all their capacity to store past inputs and their nonlinear combinations. This is directly visible in Figure 4: for the larger window lengths, the random-circuit reservoir flattens into a horizontal line after delay 4-5, meaning older inputs add zero new capacity. However, the Hamiltonian reservoir models continue to climb all the way to delay

19, proving they still retain information from 15 to 19 steps back. As shown in Figure 18, the degree-wise IPC follows the same trend, with Hamiltonian-based reservoirs retaining information from more distant past inputs.

7.1.2 Observable choice and shot precision

This sweep includes five reservoir configurations (random-circuit plus the four Hamiltonian reservoirs), three observable sets (Z, $Z + ZZ$, and XYZ), and three shot precisions ($p \in \{0.01, 0.05, 0.1\}$). The shot-based results are compared with the exact-estimator backend to investigate how the choice of observables and measurement precision affects total IPC. Multiple seeds are used to account for variation across runs.

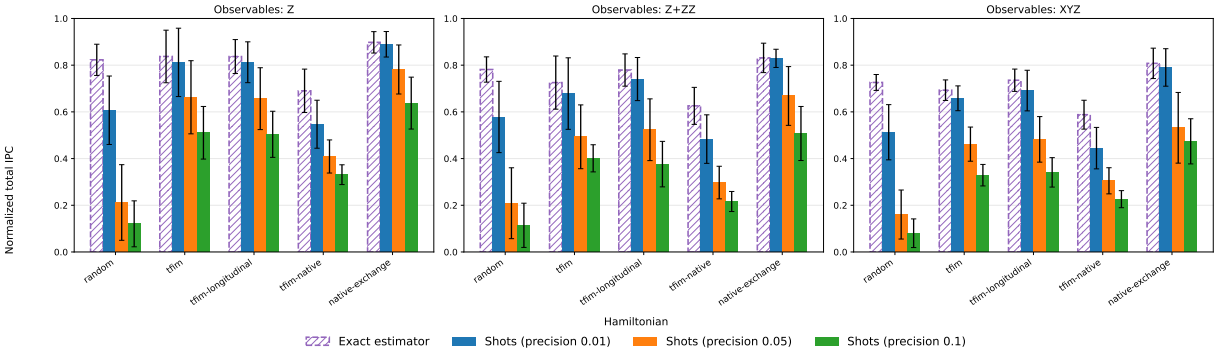


Figure 5: Normalized total information processing capacity (IPC) for different reservoir Hamiltonians using finite-shot measurements and the exact simulation results as a comparison baseline. Results are shown for Z, $Z + ZZ$, and XYZ observables across three shot precisions (precision $\sim 1/\sqrt{\text{shots}}$): $p = 0.01$ (10,000 shots), $p = 0.05$ (400 shots), and $p = 0.1$ (100 shots). Error bars represent standard deviation across seeds. All Hamiltonian models use 2 Trotterization steps and the random reservoir uses a circuit depth of 4.

Figure 5 shows that measurement shot noise drastically reduces capacity across all models. While the earlier delay curves in Figure 4 showed *how far back in time* the memory reaches, this plot shows the *total amount of memory* that actually survives measurement. A high bar here is always good: reaching 0.8+ means the reservoir is using more than 80% of its maximum capacity, while a short bar shows that shot noise destroyed most of the usable quantum information. We also observe that, with a sufficient number of shots, the NEX Hamiltonian reservoir approaches the performance of the exact estimator.

Specifically, the NEX Hamiltonian reservoir achieves the highest capacity across all observable sets and almost has the same capacity as the exact estimation in the case of 10,000 shots. The random circuit is the least resilient to shot noise, its capacity collapses to ≈ 0.20 at $p = 0.05$ and ≈ 0.10 at $p = 0.1$. Across all models, the NIM Hamiltonian reservoir is the lowest-performing reservoir architecture. In terms of observable choice, XYZ would yield the highest absolute capacity, largely due to its greater number of observables. However, to compare their performance independently of the observable count, we consider the normalized capacity. By this measure, measuring Z alone performs best, achieving the highest normalized capacity (≈ 0.85–0.90).

To investigate the effect of hardware noise, additional simulations are performed using SpinPulse with pink noise and a dephasing time of $T_2^* = 10\ \mu s$. These noisy results are compared to assess how noise affects total IPC. Again, multiple seeds are used to account for variation across runs. Figure 6 shows the results of this noisy experiment. For $p = 0.01$, the capacity shows little variation compared to the shot-based estimator, suggesting that shot noise has a greater impact on performance. At $p = 0.05$, capacity decreases across all models, although the extent of the reduction varies. The `reservoir` is the most sensitive to this increase in noise, whereas the Hamiltonian reservoirs retain approximately half of their exact-estimated capacity.

Appendices I.2 and I.4 illustrate the full degree-wise capacity per observable and precision. Appendix I.3 contains the corresponding absolute capacities, allowing comparison of the total and degree-

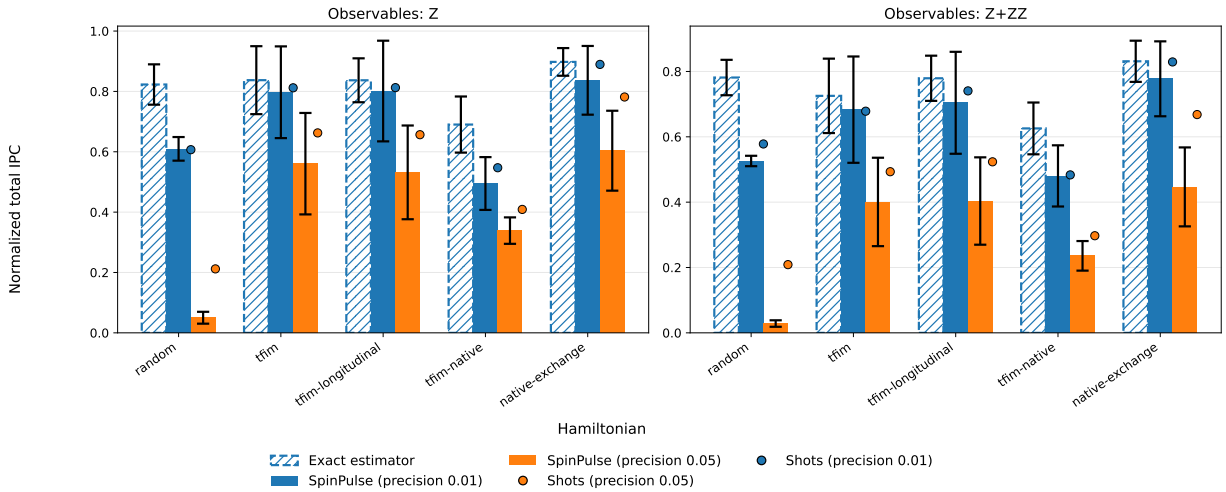


Figure 6: Normalized total information processing capacity (IPC) for different reservoir Hamiltonians simulated using SpinPulse. Results are shown for (Z) and (Z+ZZ) observables across the shot precision (precision $\sim 1/\sqrt{\text{shots}}$) $p = 0.05$ (400 shots) and 0.01 (10,000 shots). The exact and noiseless shot-based values are also shown for reference. All Hamiltonian models use 2 Trotterization steps and the random reservoir uses a circuit depth of 4.

wise contributions without normalization. Although the absolute capacity generally increases with the number of observables, the increase is not proportional to their number, indicating that adding observables does not contribute an equal amount of additional capacity (Figure 20).

7.2 Untuned QRC performance on fraud detection, against classical baselines

We now test whether the memory and nonlinear mixing measured in 7.1 carry over to fraud detection. If they do, two things should hold:

1. Customer AP should grow with the window length L
2. A linear readout on the reservoir features should beat a linear readout (logistic regression) on the raw window

The second point is the gain we can attribute to the reservoir (section 5). We therefore compare the QRC with three classical baselines that see the same window: logistic regression on the raw window, ESN with as many units as the QRC readout has observables (equal width), and an LSTM. Note that none of the models is tuned in this Subsection. The reservoirs, as well as the ESNs and the LSTM, use their default parameters. See Figure 7 for the results.

At $L = 1$, every model reaches the same customer AP (≈ 0.15). This is expected: with a single transaction there is no history to exploit, so this value is the floor that any memory has to beat. Terminal AP stays at the base rate (0.006) for every model and window length. Our windows contain the history of a single customer, while a compromised terminal shows up across many customers, so this fraud category is invisible to all our models, by construction. Amount fraud only needs the current transaction and is detected by most model (AP ≥ 0.90). We therefore focus on customer AP (leftmost column) from here on.

The two rows of Figure 7 come from the same reservoir runs; only the measured observables differ. With the Z+ZZ readout, every QRC falls below logistic regression from $L = 5$ onwards, and stays at or below the equal-width ESN-7. At $L = 10$, for example, the TFIM reaches 0.15, against 0.24 for ESN-7 and 0.33 for logistic regression. Even the current transaction is partly lost: amount AP drops to 0.91–0.95. With the 45 Pauli-string observables (XYZ-strings), the same reservoirs reach 0.35 at $L = 10$. The information is therefore present in the reservoir state, but Z and ZZ expectation values do not expose it to a linear readout. Hence we keep the XYZ-strings readout for the rest of the comparison.

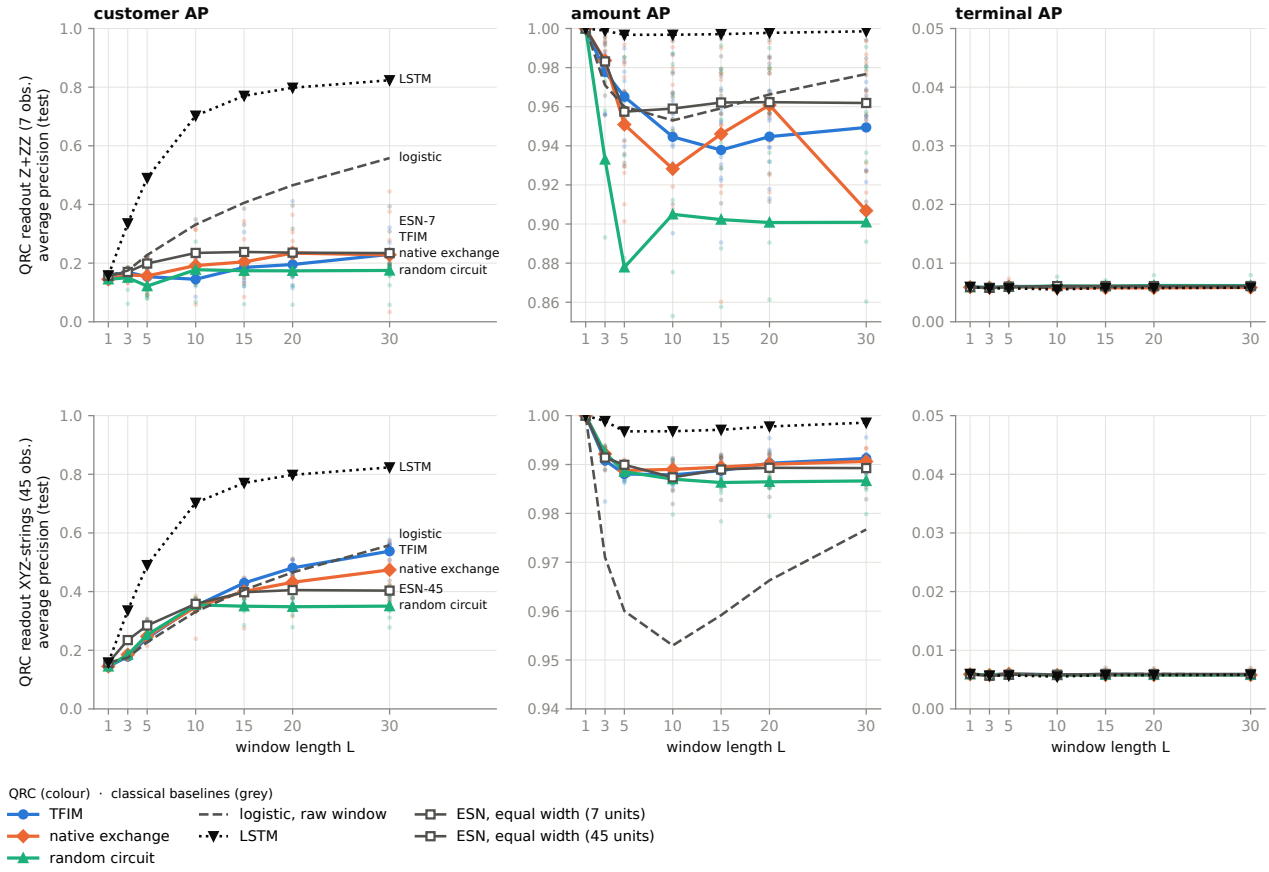


Figure 7: Average precision on the test split versus window length L, per fraud category. Top row: QRC read out with $Z{+}ZZ$ (7 observables), compared with a 7-unit ESN. Bottom row: QRC read out with all-X, all-Y and all-Z Pauli strings (45 observables), compared with a 45-unit ESN. QRC: 1 input and 3 memory qubits, input reset after each step, exact expectation values; the TFIM and NEX Hamiltonian reservoirs use their default parameters with 2 Trotter steps, the random circuit has depth 4. Classical baselines on the same window: logistic regression (deterministic), an untuned echo state network (ESN) with as many units as the QRC readout has observables, and an LSTM. Dots: seeds 42–51; lines: mean over seeds. Note that the amount and terminal panels have different y ranges.

Table 5: Test customer-fraud AP versus window length L (mean ± standard deviation over seeds 42–51). QRCs read out with the 45 Pauli-string observables, exact expectation values. All models untuned. Logistic regression has no seed. Bold: best reservoir (quantum or classical) at each L.

Model	$L = 1$	$L = 5$	$L = 10$	$L = 15$	$L = 20$	$L = 30$
TFIM	.144 ± .002	.242 ± .007	.351 ± .008	**.429 ± .011**	**.481 ± .026**	**.538 ± .039**
NEX	.145 ± .000	.247 ± .012	.347 ± .041	.401 ± .053	.432 ± .063	.474 ± .077
Random circuit	.145 ± .000	.252 ± .003	.355 ± .014	.350 ± .027	.348 ± .035	.350 ± .040
ESN-45	**.157 ± .000**	**.284 ± .015**	**.358 ± .009**	.398 ± .019	.405 ± .030	.403 ± .035
Logistic	.158	.228	.331	.406	.465	.558
LSTM	.158 ± .000	.490 ± .003	.703 ± .002	.771 ± .004	.798 ± .004	.824 ± .003

Table 5 gives the customer AP of the XYZ-strings QRCs and the baselines. Up to $L = 10$, all three reservoirs beat logistic regression by about 0.02, and the TFIM and the random circuit do so on all 10 seeds. The random circuit then plateaus at 0.35, in line with its limited memory in subsection 7.1, and falls below logistic regression from $L = 15$. The TFIM keeps improving with L. It is ahead by 0.024 at $L = 15$ and by 0.015 at $L = 20$, but passes behind by 0.020 at $L = 30$. The reservoir advantage over the logistic regression is therefore small and limited to intermediate windows.

When it comes to the equal-width ESN (ESN-45), it is better than every QRC at short windows: at $L = 5$, the TFIM is 0.043 behind on all 10 seeds. At $L = 10$ the two are similar, but from $L = 15$, the untuned ESN plateaus at ≈ 0.40 while the TFIM keeps improving. The TFIM is then ahead by 0.031 at $L = 15$ and by 0.076 at $L = 20$.

However, we observe that the random seed – which draws the reservoir, e.g. the couplings of the Trotterised Hamiltonian – matters a lot. This is especially the case for the NEX Hamiltonian reservoir: its resulting customer AP spreads with standard deviations of 0.04–0.08 from $L = 10$, as large as the gap with ESN, as illustrated in Figure 8. Consequently, we can say that the random seed acts as an hyperparameter that our experiments do not search.

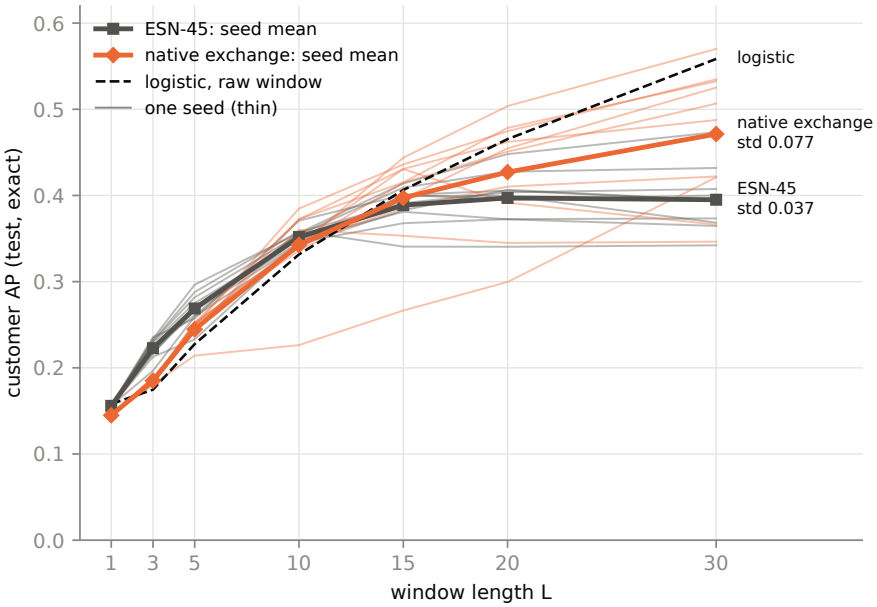


Figure 8: Same experiment as Figure 7, restricted to NEX Hamiltonian reservoir (XYZ-strings readout) and the equal-width ESN-45. Thin lines: individual seeds; bold: mean over seeds; dashed: logistic regression.

Regarding the LSTM, we observe that it reaches 0.70 at $L = 10$ and 0.77 at $L = 15$, about twice the AP of any reservoir model. However, this comparison is not like-for-like: the LSTM trains its recurrence, whereas all reservoir models only train a linear readout on fixed dynamics. What it does show is that the window contains far more information about customer fraud than any of the fixed reservoirs, quantum or classical, exposes to a linear readout.

Ultimately, with the XYZ-strings readout and exact expectation values, the TFIM reservoir is a QRC with an edge over both classical reservoir baselines for some window lengths. At $L = 15$ it beats logistic regression on all seeds and the equal-width ESN on 9 of 10 seeds. The edge is small (0.02–0.03 AP), and the results from subsection 7.6 will suggest it vanishes under realistic sampling noise and hardware fidelity.

Most importantly, and as was mentioned earlier, none of these models was tuned. The TFIM uses the default evolution time $t = 1$ and energy scale $\nu = 1$, and the ESN uses its default spectral radius and leak rate. In the next section, we therefore tune the TFIM and the ESN-45 under a fair protocol (at $L = 15$, where the untuned edge of the TFIM appears to be the clearest).

7.3 Equal-budget tuning of the models

In subsection 7.2, the untuned TFIM beat the untuned ESN-45 by 0.031 AP at $L = 15$. But an edge between two untuned models may only reflect how well each default configuration happens to suit the task. Following the protocol of Pandey (2026), we therefore give both models equal care. Their readouts already have equal width (45 features); here we also give them equal tuning budgets. We tune at $L = 15$, the window length where the untuned TFIM beats both logistic regression and the ESN-45 on at least 9 of 10 seeds. If the TFIM keeps its edge after tuning, the edge belongs to the reservoir. If it does not, it was a property of the default settings.

The protocol is the following: we run one Optuna study per model (TPE sampler, 100 trials each) over the search spaces of Table 6. Each model tunes three parameters of its dynamics plus the readout regularisation α. Each trial is scored by the validation customer AP, averaged over reservoir seeds 42–44. Trial 0 of each study is the untuned setting of subsection 7.2, so tuning can only improve the validation score. The best setting of each study is then evaluated once on the test split, on seeds 42–51; seeds 45–51 were never seen during tuning. Note that equal budget means an equal number of trials, not equal compute: a TFIM trial took much longer than an ESN trial on our runners (laptop or GitHub Actions).

Table 6: Search spaces of the two Optuna studies. All ranges are sampled on a log scale, except the spectral radius. The last column gives the untuned setting of subsection 7.2 (trial 0) and the best setting found.

Model	Parameter	Range	Untuned → tuned
TFIM	transverse-field multiplier h	$[0.01, 10]$	$1 \to 0.082$
	evolution time t	$[0.1, 20]$	$1 \to 0.52$
	encoding angle scaling	$[0.1, 2\pi]$	$\pi \to 2.91$
ESN-45	spectral radius	$[0.05, 2.5]$	$0.9 \to 0.83$
	leak rate	$[0.01, 1]$	$1 \to 0.26$
	input scaling	$[0.01, 30]$	$1 \to 2.2$
Both	readout regularisation α	$[10^{-4}, 10^{3}]$	TFIM $1 \to 3.2$; ESN $1 \to 1.5 \times 10^{-4}$

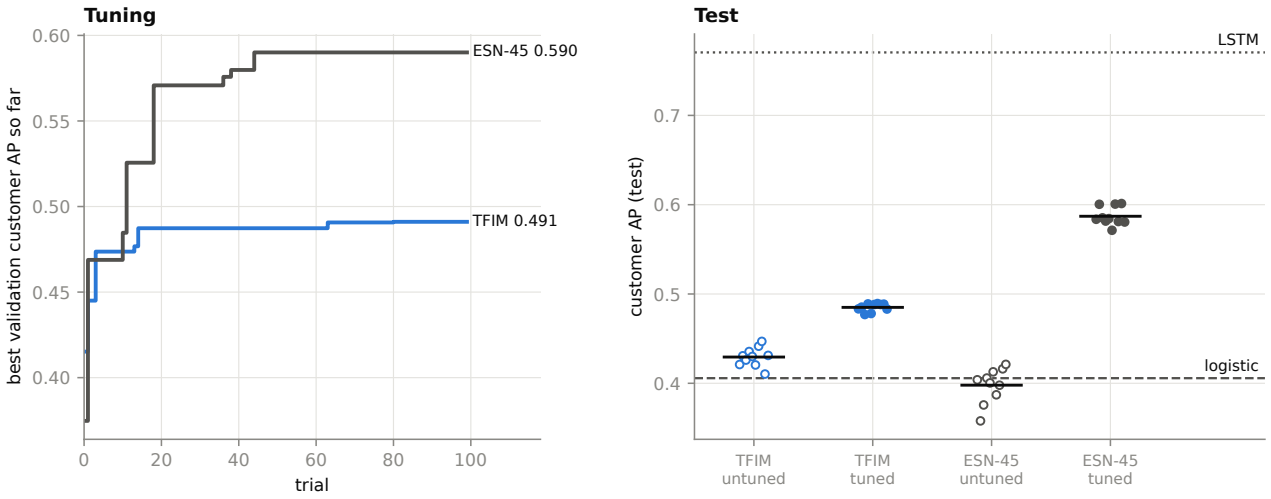


Figure 9: TFIM reservoir versus a 45-unit ESN at window length $L = 15$, with equal tuning budgets. Left: best validation customer-fraud AP found so far against the number of Optuna trials (TPE sampler, 100 trials per model). Each trial is scored as the mean over reservoir seeds 42–44; trial 0 is the untuned setting of subsection 7.2. Right: test customer-fraud AP of the untuned and tuned models on seeds 42–51 (dots; bars are means). Dashed and dotted lines: logistic regression on the raw window and the LSTM, from subsection 7.2. TFIM: 1 input and 3 memory qubits, input reset, 2 Trotter steps, XYZ-strings readout (45 observables), exact expectation values.

Figure 9 shows the outcome. Both models improve, but not by the same amount. The TFIM gains 0.056 test AP (0.429 → 0.485), while the ESN-45 gains 0.189 (0.398 → 0.587). The untuned TFIM was ahead by 0.031; the tuned ESN-45 is now ahead by 0.102, and wins on every one of the 10 seeds. The result is the same on the held-out seeds 45–51. The TFIM edge of subsection 7.2 was therefore an artefact of the default settings, not a property of the quantum reservoir.

Tuning does not change the other conclusions of subsection 7.2. The tuned TFIM is still ahead of logistic regression, so the quantum reservoir does extract useful features from the window, just fewer than a classical reservoir of the same width, and both tuned reservoirs remain far below the LSTM (0.771).

Now, AP summarises a whole ranking between models; in order to complete our study, we can continue by selecting a decision threshold for each model, and report the resulting operational metrics.

For the sake of clarity, we do so with the minimum recall rule from (Eq. 8), and we report the results in Table 7. However, we also tested using another maximum false-positive rate rule, and the results are interesting. They are reported in Appendix G.

At a fixed recall of 80%, the models mainly differ in how many alerts they raise. The tuned TFIM raises 418 alerts per day, against 244 for the tuned ESN-45, 539 for logistic regression and 138 for the LSTM. Figure 10 shows how this gap evolves as the recall target rises.

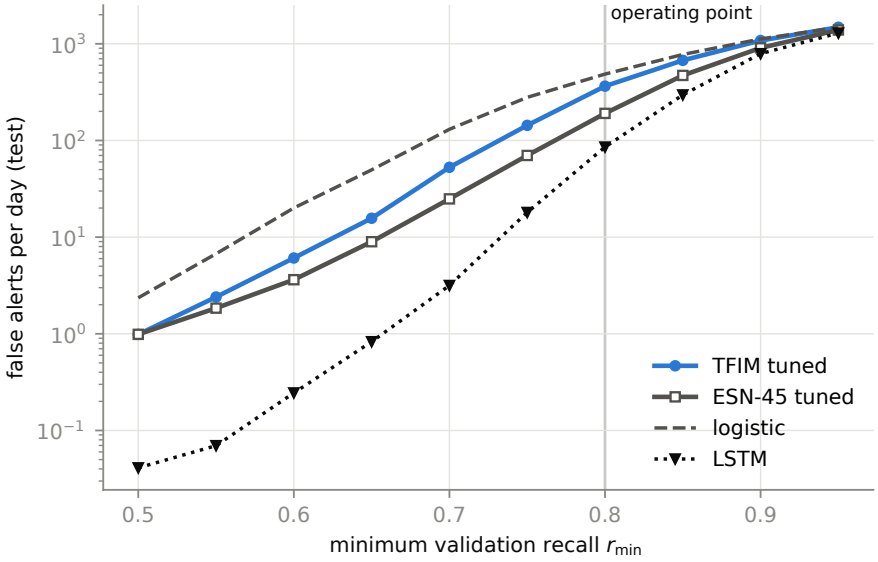


Figure 10: False alerts per day on the test split against the minimum validation recall $r_{\min}$ of the threshold rule (Eq. 8), at $L = 15$, mean over 10 seeds. Each model's threshold is chosen on its own validation scores and applied unchanged to test. The vertical line marks the operating point $r_{\min} = 0.80$.

Table 7: Operational metrics on the test split at $L = 15$. The threshold is chosen on validation and applied unchanged to test. Mean over 10 seeds (logistic regression is deterministic). Alerts per day count all flagged transactions, true and false.

Model	Precision	Recall	FPR	F1	MCC	Balanced accuracy	Recall (customer)	Alerts per day
TFIM (tuned)	0.127	0.814	19.9%	0.220	0.271	0.807	0.839	418
ESN-45 (tuned)	0.218	0.810	10.4%	0.343	0.385	0.853	0.876	244
Logistic	0.098	0.807	26.6%	0.175	0.219	0.771	0.797	539
LSTM	0.382	0.806	4.7%	0.518	0.533	0.880	0.890	138

In summary, the classical reservoir of equal width is clearly better than the TFIM reservoir under equal tuning budgets: +0.10 customer AP on every seed and about 40% fewer alerts for the same recall. The small untuned edge of subsection 7.2 does not survive a fair comparison. Also, this is in

exact-expectation simulation, the most favourable setting for the QRC; subsection 7.6 examines what happens under realistic noise and shot budgets.

7.4 QELM with engineered features

Next, we evaluate the performance of concept (2a) from Section 5.3, a quantum extreme learning machine (QELM) in which each transaction is encoded and processed independently, without recurrent quantum dynamics. Instead, historical information is captured through engineered features, while the quantum circuit provides a non-linear feature map for a classical readout. This allows us to assess whether the quantum feature map provides a benefit for fraud detection without the sequential reservoir evolution used in QRC.

We compare QELM against classical Tanh and random Fourier feature (RFF) maps using the same dataset and input features. Further details on these maps are provided in Appendix H. The inputs comprise the log-transformed transaction amount and four engineered customer-level features:

- Transaction amount deviation from the customer's 30-day average.
- Transaction amount relative to the customer's 90-day average.
- Change in average transaction amount between the 7-day and 30-day periods.
- Customer activity and transaction amounts over the previous 30 days.

The comparison is performed using the same data split and evaluation metrics. Random seeds are varied to assess the sensitivity of performance to the randomly generated feature maps. This allows us to compare the predictive value of the different transformations while keeping the input data and readout procedure consistent.

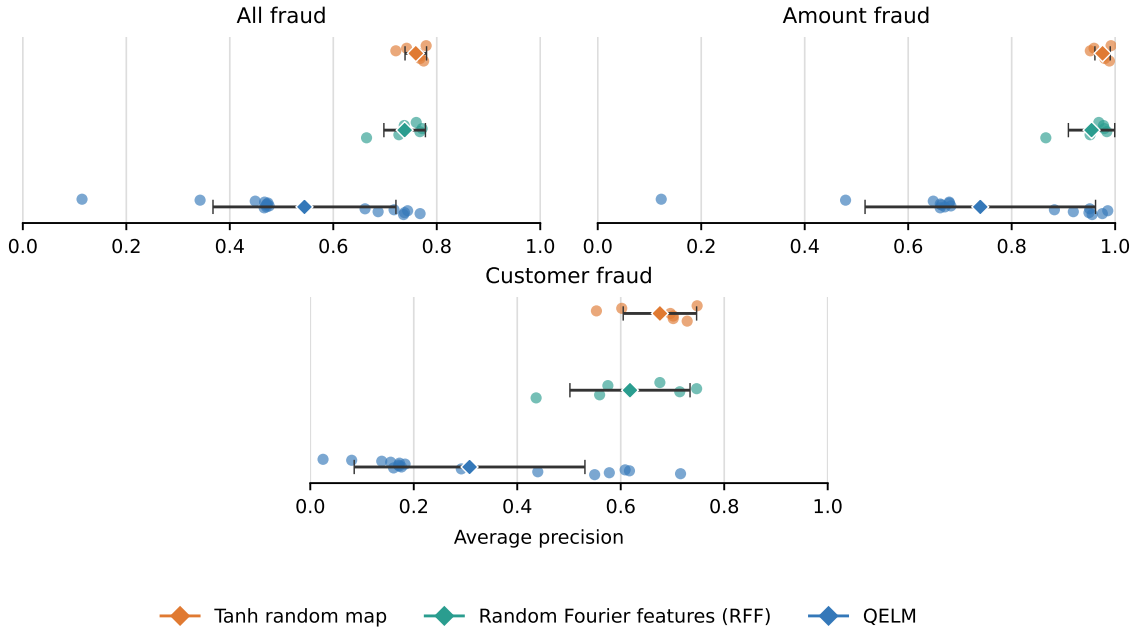


Figure 11: Test set performance per feature map for each fraud category

Figure 11 shows the results of this experiment. None of the feature maps can detect terminal fraud. Although QELM achieves comparable average precision (AP) to the classical maps in the case of amount fraud, its performance varies considerably, while the classical maps consistently achieve high AP. This suggests that applying a quantum feature map to engineered features provides no apparent benefit over classical feature maps, further indicating that the benefits of quantum reservoir computing may primarily come from its temporal memory rather than its feature mapping.

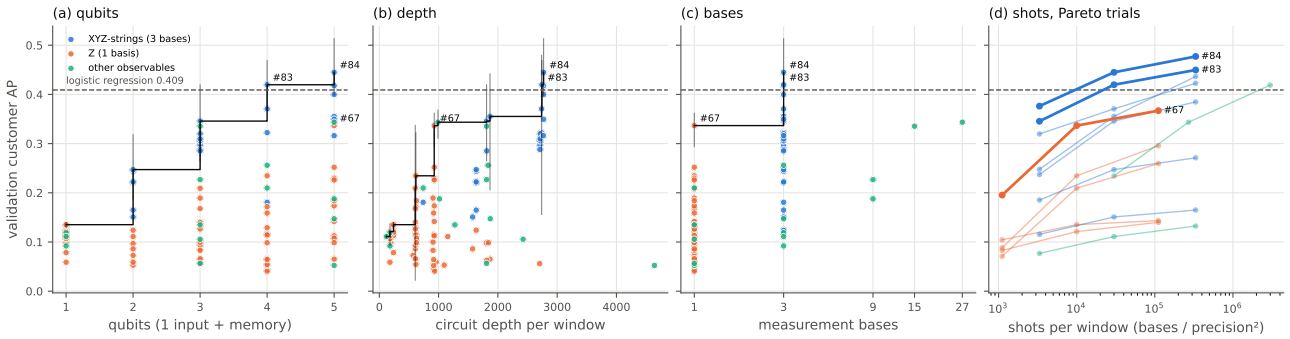


Figure 12: Validation customer AP against hardware cost, for 100 trials with $L = 15$. Each point is the mean over seeds 42–44; colour marks the observable set. In (a)–(c) the black step line is the Pareto front for that one cost, and the vertical bars show the spread over the three seeds (min–max). The dashed line is logistic regression on the raw window (0.409). (d) Trials on the four-objective Pareto front, rerun at $\varepsilon = 0.03$, 0.01 and 0.003; shots per window = bases/ε^2.

Table 8: Selected Pareto-optimal trials of the four-objective search ($\varepsilon = 0.01$). AP is the validation customer AP, averaged over seeds 42–44; brackets give the range over those seeds. Depth is for one window; 2q depth counts only two-qubit gate layers. Re-upl. (k): re-uploading encoding with k layers.

Trial	Qubits	Depth	2q depth	Bases	Encoding	Reservoir / readout	Val. AP
77	1	239	0	1	re-upl. (3)	NEX / Z	0.135 [0.128–0.141]
67	5	926	184	1	angle	NIM / Z	0.337 [0.293–0.363]
78	2	1634	270	3	re-upl. (3)	NEX / XYZ-strings	0.247 [0.185–0.319]
2	3	1809	360	3	re-upl. (2)	NEX / XYZ-strings	0.346 [0.264–0.420]
90	4	2738	546	3	re-upl. (2)	NEX / XYZ-strings	0.371 [0.155–0.481]
83	4	2742	546	3	re-upl. (3)	NEX / XYZ-strings	0.420 [0.367–0.470]
84	5	2771	552	3	re-upl. (3)	NEX / XYZ-strings	0.445 [0.397–0.514]
7	5	989	184	27	re-upl. (3)	TFIM / XYZ+XYZ-2body-all	0.343 [0.310–0.369]
Logistic regression							0.409

7.5 Multi-objective optimization (Pareto front analysis)

On hardware, a configuration costs qubits, circuit depth and shots, on top of AP. Instead of maximising AP alone, we search for the best tradeoffs between AP and hardware cost. Appendix D shows the search space used to obtain the best configuration.

We searched for configurations that show tradeoffs between customer AP and hardware cost. We use one input qubit, and a window length of $L = 15$, with the input qubit reset after every step. The qubits form a linear chain, with between zero and four memory qubits, so a configuration has at most five qubits. Dense encoding packs two features per qubit and is therefore infeasible with a single feature, which leaves the angle and re-uploading encodings. Beyond these, the search chooses between the random circuit and four Hamiltonian reservoirs and among seven observable sets, and it sets the readout regularisation $\alpha \in [10^{-1}, 10^{5}]$. Every expectation value carries Gaussian noise of standard deviation $\varepsilon = 0.01$, which corresponds to 10^4 shots per measurement basis.

Each trial is scored on four objectives. The first is the validation customer AP, averaged over reservoir seeds 42–44, which is maximised. The other three are hardware costs, which are minimised. They are the number of total (input + memory) qubits; the circuit depth of one window, that is, the L rounds of encoding and reservoir transpiled to native gates on a linear chain, excluding measurements; and the number of measurement bases, the groups of qubit-wise commuting observables, which equals the number of distinct circuits run per window. We ran 100 trials of multi-objective TPE, of which the first 10 were drawn at random. Figure 12 shows all trials, and Table 8 lists selected Pareto-optimal ones.

Each additional memory qubit raises the best AP found. Without a memory qubit, the reset input qubit carries no information from one step to the next. The readout then sees only the current

transaction, and the AP stays near the level of a memoryless model. The gains shrink with size: the fifth qubit adds 0.025, while the simulation takes about four times longer.

The front splits into two groups depth-wise. Shallow circuits, with depth between 600 and 1000, mostly use angle encoding into the NIM Hamiltonian and a Z readout, and reach an AP of 0.34. Deep circuits, with depth between 1600 and 2800, use re-uploading encoding into the NEX Hamiltonian and an XYZ-strings readout, and reach 0.45. Moving from the first group to the second triples both the total and the two-qubit depth for a gain of about 0.1 AP. We take trial 83 as the knee of the front. It reaches 0.420 with four qubits, within the seed-to-seed spread of the best trial (0.445, five qubits).

In practice the bases objective comes down to measuring one basis (Z) or three (X, Y and Z). With Z alone the best AP is 0.337 (trial 67). Every configuration above 0.35 uses XYZ-strings. This set needs only three bases but gives $3(2^n - 1)$ observables, 93 on 5 qubits. The sets with mixed X, Y and Z two-body terms need 9–27 bases and never did better than XYZ-strings, so measuring more than three bases gained no accuracy here. This differs from the observable comparison of Section 7.1, where Z alone achieved the highest normalised capacity and the larger sets were more redundant and more sensitive to shot noise per observable. Normalised capacity, however, measures the capacity per observable, whereas a linear readout benefits from the total absolute capacity, which is much larger for the XYZ-strings readout (Figure 20). Furthermore, all strings of one basis are estimated from the same shots, so the cost on hardware grows with the number of bases rather than with the number of observables.

We reran the Pareto-optimal trials at $\varepsilon = 0.03$ and 0.003 on the tuning seeds. Every trial improves with precision. At $\varepsilon = 0.03$ no configuration reaches the logistic baseline; exceeding it takes at least 3×10^4 shots per window. Although a Z-only readout needs a third of the shots, XYZ-strings achieves a higher AP for the same total number of shots. Readouts with many bases are the least efficient.

We evaluated trials 84 (best), 83 (knee) and 67 (best single-basis) on seeds 45–51, which the search never used (Table 9). On these seeds all three configurations lose more than 0.1 AP, with a large spread between seeds. They fall below logistic regression and the tuned ESN. The validation AP drops equally, so the loss comes from the new seeds. With only three draws per trial, the search favoured configurations that suited those particular couplings, the seed sensitivity already seen in Section 7.2. The tuned TFIM of Section 7.3, whose performance depends little on the draw, keeps its accuracy on the same seeds.

Model	Qubits	Tuning seeds 42–44	Held-out seeds 45–51
Trial 84 (NEX, XYZ-strings, $\varepsilon = 0.01$)	5	0.453	0.299 ± 0.107
Trial 83 (NEX, XYZ-strings, $\varepsilon = 0.01$)	4	0.414	0.298 ± 0.105
Trial 67 (NIM, Z, $\varepsilon = 0.01$)	5	0.349	0.225 ± 0.077
Tuned TFIM, exact	4	0.482	0.486 ± 0.004
Tuned ESN, 45 units	–	0.590	0.586 ± 0.011
LSTM	–	0.773	0.769 ± 0.004
Logistic regression	–	0.406	0.406

Table 9: Test customer AP (mean ± standard deviation over seeds) of the selected Pareto-optimal trials and the reference models.

The front shows that no cheap configuration is competitive. On the tuning seeds, matching logistic regression required four qubits, the deeper circuits, three measurement bases and more shots per window. Reducing any one of these lowered the AP below the baseline, whereas going beyond them gained little: a fifth qubit added only a small improvement, and more than three bases never helped. However, this configuration did not hold on new coupling draws, where it fell below logistic regression and the tuned ESN. Reservoirs with random couplings should therefore be evaluated on more draws per configuration, or replaced by reservoirs whose performance depends little on the draw, such as the tuned TFIM.

7.6 QRC performance under realistic conditions

Previous results used exact expectation values on an ideal circuit, which proves to be the most favorable setting for the QRC. On a real spin-qubit processor, each window takes physical time, during which the qubits dephase. Moreover, every gate is imperfect and lastly, an expectation value is never measured exactly. It is estimated from a finite number of shots and every measurement setting needs its own shots. We first place our operating point on a realistic chip simulated with SpinPulse, a pulse-level simulator of silicon spin qubits, to quantify the duration of a window and the quality of its gates (Subsection 7.6.1). We then measure how gate noise and finite shots together affect the memory of the reservoir (Subsection 7.6.2) and finally we compare SpinPulse's noise types with a depolarizing gate-noise model at equal gate fidelity (Subsection 7.6.3).

7.6.1 The reservoir on a spin-qubit chip

We first place the operating point (TFIM, 1 input and 3 memory qubits, 2 Trotter steps, input reset) on a realistic chip. We use the device of SpinPulse's `ParametrizingfromQPUSpecs` example: X/Y rotations of $1\,\mu$s, Z rotations of $0.1\,\mu$s, two-qubit gates with echo pulses, and $1/f$ (pink) noise with a qubit dephasing time $T_2^* = 10\,\mu$s and an exchange-coupling dephasing time $T_J^* = 5\,\mu$s, simulated with a 10 ns timestep. Table 10 reports the compiled gates and the timeline of the window.

Table 10: The QRC operating point compiled to pulses on a realistic silicon spin-qubit chip (SpinPulse, pink noise). Fidelities are average gate fidelities over 200 noise realizations.

	Duration	Fidelity (with noise)
X gate ($R_X(\pi)$)	$1.02\,\mu$s	0.99952
Z gate ($R_Z(\pi)$)	$0.11\,\mu$s	0.99997
$R_{ZZ}(\pi/2)$ with echoes	$3.08\,\mu$s	0.99565
Encoding of one transaction	$1.02\,\mu$s	
One reservoir step (12 R_{ZZ} + 37 single-qubit gates)	$12.71\,\mu$s	
Whole window, $L = 10$	$137\,\mu$s ($13.7\,T_2^*$)	≈ 0.50 (gates only)
Whole window, $L = 15$	$206\,\mu$s ($20.6\,T_2^*$)	≈ 0.35 (gates only)

The individual gates are of realistic quality, with single-qubit fidelities above 99.9% and two-qubit fidelities around 99.6%, in line with published silicon devices. The window as a whole, however, is long. At $L = 15$ it lasts about 20 times the qubit coherence time, and the product of its gate fidelities is only about 0.35. A sliding-window QRC on hardware therefore works far beyond the coherence time of a single qubit. It can only work because the input reset refreshes the input qubit at every step and the reservoir is expected to forget old transactions. It becomes therefore the question how much usable information survives in the measured features.

7.6.2 Memory under hardware noise

To answer this, we measure the short-term memory capacity $\mathrm{MC} = \sum_{d=0}^{L-1} C_d$ of the same reservoir, where $C_d \in [0, 1]$ is how well a linear readout recovers the input from d steps earlier ($L = 10$, $Z{+}ZZ$ readout). We compare an ideal circuit with the realistic chip at today's coherence (factor 1) and at a three times shorter coherence time (factor 0.3), with exact expectation values and with 400 shots per observable. We also report the signal, the standard deviation of the features across windows.

Table 11 shows that gate noise does not destroy the information in the reservoir. With exact expectation values, memory capacity even rises slightly, as dephasing adds forgetting on top of the reset. What noise does change is the size of the features. The signal drops by about a quarter on today's chip and by more than half on the studied noisier one. As soon as the features are estimated from shots, this smaller signal is what matters. At 400 shots, today's chip (1.96) is still close to the ideal circuit (2.12) but the noisier chip would lose almost half of its memory (1.12). Gate noise

Table 11: Total memory capacity (sum over 10 delays) of the TFIM reservoir (1 input + 3 memory qubits, input reset, Z+ZZ readout) on SpinPulse hardware. Coherence factor 1 is today's chip ($T_2^* = 10\,\mu$s, $T_J^* = 5\,\mu$s), 0.3 is a three times noisier chip. Signal: mean standard deviation of features across windows. 20 noise realizations per window. With 400 shots, the ideal column is the realistic chip without noise, which matches the ideal circuit in exact mode (2.92).

	Ideal	Today's chip (1)	3 x Noisier chip
MC, exact expectation values	2.92	3.07	3.68
MC, 400 shots per observable	2.12	1.96	1.12
Signal (feature spread)	0.163	0.125	0.070

therefore hurts the QRC mainly through the shot budget: it shrinks the features until shot noise drowns them.

7.6.3 Effect of different noise types on memory

SpinPulse can generate three kinds of dephasing noise: slow drifts that stay consistent during a shot (quasistatic), fast fluctuations (white), and $1/f$ noise in between (pink), the spectrum observed on real devices. To compare them fairly, we calibrate each one so that the $R_{ZZ}(\pi/2)$ gate of the realistic chip reaches the same average fidelity of 99% (qubit dephasing only). We add a depolarizing model at the same fidelity, in which a depolarizing channel acts after every native gate with a single qubit error equal to one fifth of the two-qubit error. We then measure the memory capacity of the reservoir of Subsection 7.6.2(TFIM, 1 input and 3 memory qubits, input reset, $L = 10$, Z+ZZ readout) for each model, with exact expectation values and with 400 shots per observable.

Equal gate fidelity requires very different dephasing time: $T_2^* = 3.4\,\mu$s for quasistatic noise, $5.0\,\mu$s for pink noise and $249\,\mu$s for white noise. The echo pulses of the two-qubit gate cancel slow noise but not fast noise, so T_2^* alone is not a meaningful figure of merit across noise types, gate fidelity is. The single qubit gates, which have no echoes, then differ: 99.55% for quasistatic noise against about 99.86% for pink and white noise.

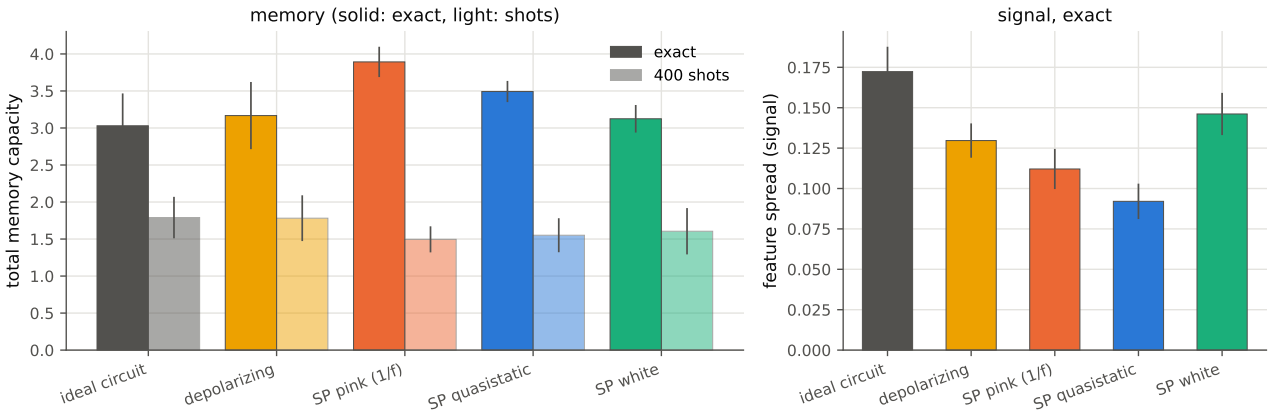


Figure 13: Total memory capacity (left; solid: exact expectation values, light: 400 shots per observable) and feature spread (right) of the TFIM reservoir at a two-qubit gate fidelity of 99%, for the ideal circuit, the depolarizing model, and SpinPulse pink, quasistatic and white noise on the realistic chip. Mean ± standard error over seeds 42–44; 300 time steps, 40 noise realizations per window.

Figure 13 and Table 12 show two regimes. With exact expectation values, none of the noise models reduces the memory of the reservoir at this gate fidelity. The depolarizing model is slightly above the ideal circuit on every seed (+0.1 to +0.2), and so is pink noise, by 0.9 on average. Quasistatic and white noise are on average equal to or above the ideal circuit, with differences of either sign between seeds. As in Subsection 7.6.2, dephasing adds forgetting on top of the input reset, which can even

Table 12: Total memory capacity at a two-qubit fidelity of 99% (mean ± standard error over seeds 42–44), with exact expectation values and with 400 shots per observable. SpinPulse features are averaged over 40 noise realizations per window.

Model	MC, exact	MC, 400 shots	Signal (exact)
Ideal circuit	3.03 ± 0.44	1.79 ± 0.28	0.172
Depolarizing	3.17 ± 0.45	1.78 ± 0.31	0.130
SpinPulse, pink	3.89 ± 0.20	1.50 ± 0.18	0.112
SpinPulse, quasistatic	3.49 ± 0.14	1.55 ± 0.23	0.092
SpinPulse, white	3.12 ± 0.19	1.61 ± 0.31	0.146

increase the memory capacity. What all noise models do change is the size of the features: the signal drops from 0.17 for the ideal circuit to 0.09–0.15.

With 400 shots, this smaller signal is what matters. The depolarizing model stays equal to the ideal circuit, while the SpinPulse models are 0.2 to 0.3 below it on average, white noise on every seed. These differences are smaller than the spread between seeds: once shot noise is included, the shot budget dominates the result, and the type of gate noise is a secondary effect. Slow noise, which adds the most memory with exact readout, also shrinks the features the most (signal 0.09–0.11 against 0.15 for white noise), and loses its advantage once shots are included.

In summary, at equal two-qubit fidelity, the type of dephasing noise does not change the picture of Subsection 7.6.2: noise does not remove the information stored in the reservoir, but shrinks the measured features until shot noise drowns them. Gate fidelity is therefore a reasonable figure of merit for the QRC, provided it is compared at a realistic shot budget. Note that SpinPulse features in exact mode are averages over 40 noise realizations per window, repeating with 160 realizations changes the memory capacity by up to about 0.45, less than the spread between reservoir seeds.

7.6.4 Fraud detection on the chip

Finally, we run the fraud task itself on the realistic chip of Subsection 7.6.1 (TFIM, 1 input and 3 memory qubits, input reset, $L = 15$, 39-observable XYZ + two-body readout), on a subsample of 10 000 windows per split. Finite shots are modelled as Gaussian readout noise of width $1/\sqrt{N_{\text{shots}}/9}$ on each observable, since the 39 observables require nine measurement settings among which the shots are divided. Table 13 compares the ideal circuit, the realistic chip without noise, and today's chip with logistic regression on the raw window, evaluated on the same windows.

Table 13: Test customer-fraud AP on a subsample of 10 000 windows per split ($L = 15$, TFIM, 39-observable XYZ + two-body readout, reservoir seed 42). Today's chip: SpinPulse realistic chip with pink noise, $T_2^* = 10\,\mu$s and $T_J^* = 5\,\mu$s, 20 noise realizations per window. Logistic regression on the raw window reaches 0.364 on the same windows.

Readout	Ideal circuit	Chip without noise	Today's chip
Exact	0.333	0.336	0.311
$N_{\text{shots}} = 10^5$	0.331	0.332	0.267
$N_{\text{shots}} = 10^4$	0.206	0.207	0.171
$N_{\text{shots}} = 10^3$	0.087	0.087	0.079

With exact expectation values, today's chip performs like the ideal circuit: 0.311 against 0.333 on the test split, but 0.327 against 0.291 on the validation split, a difference within the statistical scatter. Gate noise alone therefore costs the fraud task little, as it cost the memory task little in Subsections 7.6.2 and 7.6.3. Finite shots then separate the two. At 10^5 shots per window, the ideal circuit loses nothing (0.331), while today's chip falls to 0.267: the same shot budget is harmless for full-size features but costly for features that the noise has shrunk. At 10^4 and 10^3 shots, both fall sharply, to 0.206 and 0.171, then to below 0.09, close to the fraud base rate. At every budget, including

exact readout, the QRC remains below logistic regression (0.364). The pattern of the memory study therefore holds for the task itself: gate noise costs little on its own, and finite shots are what push the QRC below the classical baseline. With about 130 customer frauds per split, the AP values are uncertain by a few hundredths; the gap to logistic regression is not.

7.7 Cost of running the QRC: time and energy

We now estimate what one fraud decision costs for each implementation: time per transaction, energy per transaction and the time needed to build the training set. On hardware, every transaction requires its own window circuit (section 5.5), repeated N_{shots} times to estimate the features. One shot takes L steps of encoding, reservoir evolution and input reset, followed by a final readout:

$$T_{\text{tx}} = N_{\text{shots}} \left[L \left(t_{\text{enc}} + t_{\text{res}} + t_{\text{reset}} \right) + t_{\text{ro}} \right], \tag{9}$$

with $t_{\text{enc}} = 1.02\,\mu$s and $t_{\text{res}} = 12.71\,\mu$s from Table 10, consistent with the microsecond timescale of unitary operations in silicon spin qubits. Our simulations do not model reset and readout times, which depend strongly on the readout technology: about $1\,\mu$s is projected for future architectures (Santos et al., 2026), while single-shot in situ readout demonstrated on a foundry-fabricated unit cell needs about $100\,\mu$s of integration and $100\,\mu$s of initialisation (Hamonic et al., 2026). We assume $t_{\text{reset}} \approx t_{\text{ro}} \approx 10\,\mu$s, as an intermediate range. At $L = 15$, one shot then takes about 0.37 ms. Figure 14 shows how this time is spent.

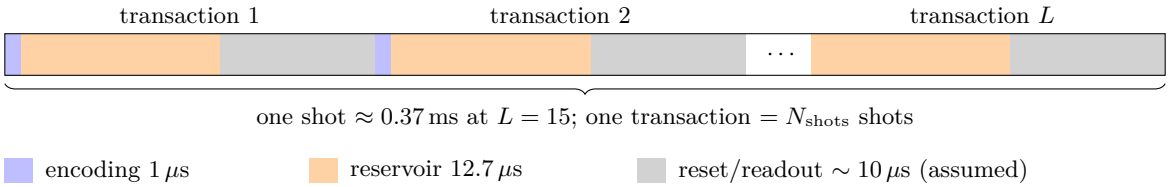


Figure 14: Time budget of one shot of the sliding-window QRC on a spin-qubit chip (Equation 9). Gate durations from Table 10. A fraud decision needs N_{shots} such shots, one full window per shot.

For the energy consumption, we take Quobly's smallest commercial system, the Alloy Pioneer (Quobly, 2026), designed for data-centre operation: it draws 20.5 kW, has a footprint of 12 m^2 with clearance and an uptime of 75–90%. Most of this power goes into cryogenic cooling, which alone requires 5–10 kW (Santos et al., 2026), even though silicon spin qubits operate at about 1 K rather than the millikelvin temperatures of superconducting qubits. The control electronics of a 4-qubit chip add only tens of watts. For the classical models and the exact simulation, Table 14 uses the run times calculated in sections 7.2 and 7.3 ($L = 15$, 691 963 windows over all splits, on a laptop or a GitHub Actions runner), divided by the number of windows, and assumes 30 W for one CPU.

Two main conclusions follow from Table 14. First, the QRC on hardware would be slow. Even at 10^3 shots, one decision takes 0.37 s, beyond the 100 ms budget of real-time authorization. Only the asynchronous, post authorization use case remains possible. Even there, an issuer processing 10^6 transactions per day would need about $3.7 \cdot 10^6$ s of processor time per day at 10^4 shots, i.e. about 50 Alloy Pioneer systems running in parallel once their 75-90% uptime is accounted for, drawing about 21 MWh per day and occupying about 600 m^2. Second, a 4-qubit reservoir is cheaper to simulate exactly on a laptop than to run on a quantum processor, by three to four orders of magnitude in time and six to seven in energy (10^3–10^4 shots). Hardware becomes worth considering for registers too large to simulate (tens of qubits for exact state-vector simulation). At that size, our results suggest that the shot budget, not the register size, would be the limiting factor. Finally, energy-wise, we note that, in principle, spin-qubit hardware can become energetically competitive with classical computing before it becomes faster, since its power draw is dominated by a constant cooling cost; this crossover, however, is expected only at tens of qubits (Santos et al., 2026), far beyond our four-qubit reservoir, whose energy per decision is dominated by the cryostat running through thousands of shots.

Table 14: Cost of one fraud decision ($L = 15$, 4 qubits) per implementation. Classical and simulated times are measured; chip times follow Equation 9 with reset and readout times of 10 μs. Energy assumes 30 W per CPU and 20.5 kW for a Quobly Alloy Pioneer system (Quobly, 2026). Training set: 483 807 windows. AP: test customer-fraud AP from Subsections 7.2 and 7.3. *Training time of the LSTM. ‡With 20 noise realizations per window. †Measured on a subsample of 10000 windows per split (TFIM, 39-observable XYZ + two-body readout, today's chip), where logistic regression reaches 0.364; finite shots are modelled as Gaussian readout noise of width $1/\sqrt{N_{\text{shots}}/9}$ on each observable.

Implementation	Time per tx	Energy per tx	Training-set features	Customer AP
Logistic regression (CPU)	$\sim 1\,\mu$s	~ 0.03 mJ	< 1 s	0.406
ESN-45 (CPU)	15 μs	0.4 mJ	7 s	0.587 (tuned)
LSTM (CPU)	not timed	–	4.6 min*	0.771
QRC, exact simulation (CPU)	0.25 ms	7.6 mJ	2 min	0.485 (tuned)
QRC, pulse-level simulation‡ (CPU)	30–60 ms	1–2 J	4–8 h	0.311†
QRC on chip, $N_{\text{shots}} = 10^3$	0.37 s	7.5 kJ	2 days	0.079†
QRC on chip, $N_{\text{shots}} = 10^4$	3.7 s	75 kJ	21 days	0.171†

8 Discussion

In this section, we discuss the results that were presented in Section 7. In summary, the implemented quantum reservoirs meet the properties that motivate the use of QRC: non-linear mixing and retaining memory, particularly when using Hamiltonian architectures. However, this does not imply any advantage for fraud detection. We found that, under current realistic conditions, classical models are better suited for this task. In this section we provide an interpretation of the results found, the limitations identified, and an assessment of the achievement of the minimum viable product.

8.1 Interpretation

The reservoirs have a short memory and mix inputs nonlinearly. The IPC experiments (Section 7.1) show that all reservoirs retain information about past inputs and combine them nonlinearly, but in different ways. The random circuit concentrates its capacity on the most recent inputs: its linear memory saturates after about four steps, and it has the largest quadratic and cubic capacities at short delays. The Hamiltonian reservoirs spread their capacity over longer delays: their linear memory keeps growing slowly up to the end of the window, by about 0.05 per additional delay. This matches the fraud results, where the random circuit stops improving beyond $L = 10$ while the TFIM keeps improving up to $L = 30$ (Section 7.2).

The readout observables matter, and their cost does too. The observable sweep (also Section 7.1) also reveals a trade-off between normalized efficiency and classification expressiveness. While sets like $Z + ZZ$ and XYZ provide more total features, measuring Z alone achieves the highest normalized capacity because it provides the most information with the least observables. However, on a 4-qubit reservoir, Z yields only 4 features, which proved too low-dimensional for a linear readout to separate complex customer fraud patterns (subsection 7.5). In contrast, the 45-observable XYZ-string set has a slightly lower normalized capacity, but provides a larger absolute capacity (≈ 10 vs. 3 as seen in Figure 20). This higher-dimensional feature space gives the linear readout the degrees of freedom needed to detect fraud, even though it requires more measurement settings.

Hamiltonian reservoirs are sensitive to the random seed. The performance of the Hamiltonian reservoirs varies with the random seed because the coupling and local field strengths are drawn randomly upon initialization (7.2). For our NEX Hamiltonian reservoir, this spread is particularly important. Averaging across random seeds ensures our capacity benchmarks reflect the typical performance of physical hardware that is subject to disorder, rather than a single best-case device.

A dissipation mechanism is necessary, and the input reset provides it. Without the input reset, the evolution within a window is unitary: earlier inputs are never forgotten, only scrambled across the register. In an ablation reported in Appendix Appendix J, the QRC without reset collapsed from $L = 5$. Hardware dephasing provides some forgetting of its own: with exact readout, the memory capacity on a realistic chip is equal to or slightly above that of the ideal circuit, the gain being concentrated on the last one to three transactions (7.6.2 and 7.6.3). It cannot replace the input reset, however: pure dephasing erases phases but preserves populations, and its benefit disappears once features are estimated from finite shots. On hardware, the input reset is a mid-circuit operation, which costs time and fidelity.

Comparison between models needs equal budget tuning. In Section 7.3, we saw that the edge of a quantum model over its classical counterpart was nothing but a tuning artifact. This confirms the warning of Pandey (2026): claims of a quantum advantage are only meaningful against classical baselines of equal width that received the same tuning effort.

The shot budget, not gate noise, is the bottleneck. The three hardware studies of Subsection 7.6 point to the same conclusion. With exact expectation values, realistic gate noise has little cost for the reservoir: its memory capacity is virtually unchanged (Tables 11 and 12) and its fraud-detection performance on today's chip is close to that of the ideal circuit (test customer AP 0.311 against 0.333). Noise does not erase the information stored in the reservoir. It shrinks the measured features towards zero. With exact readout, a linear readout simply rescales them. With finite shots, however, every feature carries a statistical error of $1/\sqrt{N_{\text{shots}}}$ that does not shrink with the features, and the smaller signal is progressively lost in it. On the realistic chip, even 10^5 shots per window leave the QRC below logistic regression (0.267 against 0.364) at a cost of 37 s per transaction. In that sense, the quantum reservoir keeps its information but cannot express it within a realistic shot budget.

8.2 Quantum vs classical, under realistic conditions

In exact simulation, the QRC extracts useful features from the transaction window: the tuned TFIM beats logistic regression on the raw window. But it extracts fewer than a classical reservoir of the same width, and far fewer than a trained recurrent network, which shows that the window contains much more information than any fixed reservoir, quantum or classical, exposes to a linear readout.

Under realistic conditions, the comparison becomes less favourable still. Classical models read their state exactly, in a single pass, while the QRC must repeat every window thousands of times to estimate its features, which makes it slower, more energy-consuming and less accurate than the classical benchmarks at every shot budget we tested (7.6.4 and 7.7). Finally, a four-qubit reservoir is much cheaper to simulate exactly on a laptop than to run on hardware. Hardware only becomes relevant for registers too large to simulate classically, and nothing in our results suggests that larger registers would close the gap with classical models.

8.3 Limitations

First of all, we think that a major limitation lies in the task itself and in the data available for it. Among the public datasets we assessed, none combined fraud labels, card identifiers and timestamps (see Appendix B). We therefore generated synthetic transactions and simulated fraud with hand-made rules, inspired from Le Borgne et al. (2022): amounts above a fixed threshold, compromised terminals for 28 days, and compromised customers whose amounts are multiplied by five for 14 days. But these rules are much simpler than real fraud patterns, and they shape which kind of memory is useful. The customer's history does matter, but it can arguably already be fully captured by classical rolling statistics. Moreover, in the simulator a customer is compromised at random, independently of their further-in-the-past history. This differs from the benchmarks on which QRC has been extensively tested, such as NARMA or Santa Fe laser data prediction, where the target depends genuinely non-linearly on the trajectory of past inputs. Lastly, all results come from a single dataset and a single chronological split, with evaluation periods of only 54 days (validation) and 56 days (test).

Another limitation concerns the scope of our model. Most sliding-window QRC experiments of section 7 encode a single input feature, the log amount of the transaction, on one input qubit. This kept simulations tractable, and was also motivated by our data: timestamp features add no measurable signal over the log amount (see Appendix C). As a consequence, no results on higher-dimensional encodings on the fraud task could be presented, although the company coaches asked for them, and all fraud experiments use a register of only four qubits. Moreover, our choice of building windows from each customer's history makes terminal fraud undetectable by construction: its AP stays at the base rate for every model. Overall metrics, which mix this undetectable category with trivially detected amount fraud, are therefore hard to interpret, and we based our conclusions on per-category AP instead.

When it comes to the hardware results, we remark that they rely on several simplifications. SpinPulse models pure dephasing only: energy relaxation (T_1), readout errors and reset errors are not included. T_1 is typically much longer than T_2^* in silicon spin qubits, but relaxation is dissipative: amplitude damping noise has been shown to improve the performance of quantum reservoir computing (Domingo et al., 2023), while phase damping, the noise modelled here, should rather be corrected. Including T_1 could therefore provide part of the fading memory that the input reset currently supplies.

The chip is assumed stable between the collection of training and test data, whereas real devices drift over time. Reset and readout times are not simulated but taken from the literature, ranging from about $1\,\mu$s (projected) to $100\,\mu$s (demonstrated), which changes the cost estimates by a factor of five, however without affecting the conclusions. The hardware results also have limited statistics: the fraud evaluation on the chip uses a subsample of 10 000 windows per split and models shots as Gaussian readout noise, and the memory studies average over 20–40 noise realizations and one to three reservoir seeds.

8.4 Validation of the minimum viable product

The minimum viable product (MVP) that we defined at the start of the project was mostly achieved: we delivered an installable and documented Python package that implements a sliding-window QRC pipeline for credit-card fraud detection, along with a command-line tool, classical baselines and a thorough hardware-aware benchmark.

The code we produced was continuously checked via automated tests, and we made sure it could reproduce the results from independent work on QRC (Dos Santos, 2026). Moreover, we traced every run which contributed to the results presented in this report. We versioned their configuration, seeds, and metrics in our Git repository – hosted on GitHub – for reproducibility.

This provided us with a valuable research tool for investigating QRC, including its different configurations and its applicability under realistic hardware conditions. On fraud detection, it produced negative results (outperformed by classical baselines, much higher energy cost, etc.), but we expect the rigor and honesty of our process to be valuable.

9 Impact assessment

This section assesses the potential impacts and risks that a fraud detection system based on quantum computing technologies could raise. First, impacts and risks for Quobly and financial institution customers are assessed, focusing on regulatory risks and explainability. This relates to the profit aspect of the 3 P's. Next, we discuss ethical implications with impacts on society, related to the people aspect from the 3 P's. This includes data privacy, bias and potential harm from false positives. Finally, we examine the planet aspect, and the impacts on the environment and contribution to the Sustainable Development Goals (SDGs) proposed by the United Nations. In addition, we propose measures to mitigate the risks identified.

Several of the risks mentioned in this section are relevant for AI systems. This is because although QRC is leveraged by quantum computing, its underlying model is based on machine learning techniques. For quantum computing and quantum technologies there are not as many regulations compared to those for AI systems. This is because of the early stage and maturity of quantum computing implementation on the industry. Government oversight on quantum is mainly concerning post-quantum cryptography (PQC). For example, the "Roadmap for the Transition to Post-Quantum Cryptography" published on June 2025, by the NIS Cooperation Group from the EU (NIS Cooperation Group, 2025). This roadmap does not impact directly the use case of this project, still it is important to stay informed on updates and potential new regulations.

Regulations such as the EU AI Act (Regulation EU 2024/1689) can pose a risk both for Quobly and for financial institutions customers. Under (European Parliament and Council of the European Union, 2024, Annex III, point 5(b)), the AI Act flags credit scoring and creditworthiness assessments as high-risk. It explicitly excludes fraud detection systems from this category.

> **ANNEX III: High-risk AI systems referred to in Article 6(2)**
>
> High-risk AI systems pursuant to Article 6(2) are the AI systems listed in any of the following areas:
>
> ...
>
> **5. Access to and enjoyment of essential private services and essential public services and benefits**
>
> ...
>
> (b) AI systems intended to be used to evaluate the creditworthiness of natural persons or establish their credit score, with the exception of AI systems used for the purpose of detecting financial fraud.
>
> (Regulation (EU) 2024/1689)

The exemption applies when the system is used for fraud detection, however, if the outputs are used to inform creditworthiness assessments, the fraud detector could fall into the high-risk category. This is a potential risk for Quobly as the provider of the algorithm and for financial institutions as users of the algorithm, since it could impose additional legal obligations. To mitigate this risk, both stakeholders must have clarity on the end purpose of the detector and comply with obligations in the event that the output is used to inform high-risk systems.

Another potential risk for Quobly and for financial institutions as customers is explainability. On the Ethics Guidelines for Trustworthy AI, presented by the High-Level Expert Group on Artificial Intelligence (2019), explicability is proposed within the ethical principles in the context of AI systems. According to this guidelines, explicability means that "processes need to be transparent, the capabilities and purpose of AI systems openly communicated, and decisions - to the extent possible - explainable to those directly and indirectly affected". As previously detailed, QRC transforms data into a high-dimensional space and processes it through non-linear dynamics. Then, a classical readout uses quantum observables as input, instead of the original features. This complicates the possibility of explaining why a prediction was made and how the original inputs influenced it.

To mitigate this risk, we propose implementing explainable AI (XAI) methods such as local interpretable model-agnostic explanations (LIME), shapley additive explanations (SHAP), counterfactual explanations (Materassi et al., 2024), feature importance, permutation importance, and partial dependence plots (Deole, 2025). XAI methods were not implemented in this project since it falls out the scope of work.

Moving on to ethical implications with an impact on society, we put attention to data privacy. Credit card fraud detectors rely on large volumes of card transaction data to operate. Transaction history can reveal sensitive information such as political opinions, religious beliefs, health data, or data concerning a persons sexual orientation. Regulations that protect data privacy, most notably the General Data Protection Regulation (GDPR) (Regulation EU 2016/679) mitigate this impact. However, in the context of card transactions, clients have limited control over the processing of their data. According to GDPR, Recital 47 “The processing of personal data strictly necessary for the purposes of preventing fraud also constitutes a legitimate interest". For the credit card user, this means having the right of access and right of rectification, but cannot oppose to data treatment, or exercise the right to erasure. The fraud detector implemented for our project was designed using anonymized data, and does not depend on sensitive information features. We encourage any future user of our implementation to omit sensitive information features from the datasets used as input.

Harm from false positives is another ethical implication with impact on society. In the context of credit card fraud detection, a false positive means the detector flags a client positive for commiting fraud, when in reality no fraud was carried out. For the credit card user this could imply a blocked or frozen account, or a declined payment. It is also a risk for financial institutions, because of the loss of trust and loyalty from a client from being wrongly flagged. As described in the Section 6, Average Precision (AP) was chosen as the primary threshold-independent metric for comparing the models' ability to distinguish fraudulent from legitimate transactions. However, because of the high cost of a false positive, the decision threshold was selected on the validation set to minimize false positives while maintaining a minimum fraud recall. Precision and other threshold-dependent metrics were then used to assess the resulting classification performance. If the classification threshold is adjusted, we recommend maintaining this trade-off between minimizing false positives and ensuring sufficient fraud detection. Additionally, the detector is suggested as a tool to aid decision making, rather than an automated decision maker.

The risk with the most material implication on the environment is the energy demand associated with quantum computing. Cooling energy is the dominant contributor to energy expenditure on a spin-qubit quantum processor: the cryostat alone requires 5–10 kW of input power, and a complete commercial system, such as Quobly's Alloy Pioneer draws about 20.5 kW. Therefore, the energy for a single fraud decision grows with the number of shots needed to estimate the features. For details and figures on the energy consumed, see Subsection 7.7). To mitigate this impact, we recommend that reservoirs with a small number of qubits be simulated classically instead of on quantum hardware.

Initially, on our problem statement review, we used the Sustainable Development Goals (SDGs) as a guideline to adress sustainability. We commited to adressing SDG 9 “Industry, Innovation and Infrastructure" and SDG 16 “Peace, Justice and Strong Institutions". In retrospect, the project did not contribute to SDG 16 as anticipated. However, we believe there is a contribution to SDG 9, particularly in the innovation aspect, by investigating a QRC approach to fraud detection.

10 Recommendations and future work

Considering the results obtained from our QRC implementation and comparing them with both the benchmark model presented in Section 4 and the classical baselines developed in this project, it is evident that – at the scale of our project – this quantum technology is not yet mature enough for market adoption or to support a viable business case. As discussed in Section 8, QRC currently does not consistently outperform the classical benchmarks in terms of processing speed, resource efficiency, or predictive accuracy.

It is important to recognize that, as noted by Ortt (2010), not all technologies should undergo large-scale adoption immediately after their invention. QRC, in particular, remains in the innovation phase, and rushing its initial market introduction could hinder its long-term potential. In parallel with its work on Quantum Processing Units development, we consider that Quobly should continue exploring such applied algorithms with the objective of being able to offer business value once the hardware is ready.

On a more practical note, specific to the context of our project, below are some recommendations we would like to make.

Hardware

- Shorten each shot by developing a faster active qubit reset method and increase the shot rate, since the run time scales with the number of shots times the duration of each window.
- Run many small reservoirs in parallel on one chip, since the cryostat and the control electronics are what dominate energy consumption, not the gates.

SpinPulse

- Natively support X- and Y- basis measurements as well as active qubit reset in SpinPulse, so that our protocol can be simulated more faithfully.

Methodology

- Give quantum and classical models an equal chance: the same inputs, the same tuning budget, equal readout width and realistic shot counts.
- Choose the observables to measure from the available shot budget, not from exact-simulation performance.

Future work

- Model credit-card fraud more realistically: real or realistic data and fraud rates, more than one input feature, and the business-relevant operating constraints.
- Explore QRC on other use cases, since results from the literature on other tasks are promising.
- Re-run our experiments under even more realistic hardware conditions to test whether noise can act as a genuine dissipation mechanism.

11 Team reflection

11.1 Team process during project

To undertake the challenge proposed by Quobly, we rapidly chose a collaboration framework understanding that, further than the complexity of the technology we were going to work with, we were facing a software development project. Given the project's exploratory nature and the short time available to build a functioning QRC circuit capable of predicting financial fraud, we decided that using scrum was the most effective method to kick-off the project. Building a modular MVP at the project's early stages with constant feedback from our company coaches allowed us to test different circuit architectures while, in parallel, we implemented other necessary but non-critical components like the spinpulse emulator, the optuna optimizer and some classical machine learning methods to set our own baseline. This approach helped us to pick an optimal architecture and to implement the expected technologies through incremental iterations.

This scrum implementation included short meetings every other day through the week to discuss, backlog control, two weeks sprints and weekly meetings with our main stakeholder, Quobly. The roles needed by this agile approach were taken organically by each of us. Moreover, we divided the team into two sub-teams, a data/product team and a quantum team. This division helped us to thrive on our interdisciplinarity while we complied with the different needs of the project. We used the scrum meetings as a checkpoint to assure constant communication between both teams.

Our team dynamics were in general good, from the first moment all the members showed interest to participate actively in the development of the project, and it evolved positively week by week as we started to know each other better. This was confirmed by the buddy check instances and the peer review session. The feedback given was positive or constructive and it helped us to strengthen our good relationship.

11.2 Interdisciplinary work

The problem proposed by Quobly sat at the intersection of three disciplines: quantum computing, machine learning, and finance. Our team combines backgrounds in applied physics, electrical engineering, computer science, quantum computing, applied mathematics, finance, industrial engineering and management of technology. Spliting the group into a quantum sub-team, and a data/product sub-team allowed each member to better put their skills into practice.



All of the members of the team were indispensable and needed to complete the project. Our experts in quantum computing: Ioanna, Prisha and Jules, were an essential element to understand the problem at hand, and implement the quantum reservoir computing. Their expertise on quantum computing was shared with enthusiasm, and the rest of the learned a lot from it. Radu, our data scientist, made significant contributions, specially when implementing the classical machine learning baselines. Linh and Lola, the applied mathematicians played key roles in the project. Linh, with a solid background in finance and computing, contributed by implementing the dataset generator, and by offering insights into the financial industry. Lola's experience in working in a corporate environment was meaningful in her role as technical contact, and to carry out the impact analysis. Last but not least, the team management skills of our project manager, Julian, were essential for ensuring cohesion among the different parts of the project and for providing the necessary follow-up to keep the project on track.

Our different backgrounds did not operate in isolation; instead, they complemented each other like pieces of a puzzle. For example, when establishing the MVP: without the skills of a project manager, the framework for defining it and the steps to achieve it would not have been very well defined; at the same time, the skills of the quantum computing experts were necessary to define what we wanted and what we could achieve as the MVP.

Having different backgrounds and working styles presented some initial challenges. For instance, with language and vocabulary. Words like noise, optimization, or memory can have different meanings for someone with a background in business, or computer science, or applied physics. This challenge emerged and was resolved throughout the project thanks to good communication. In the end, this challenge enriches us because we learned to understand the same vocabulary from different perspectives.

The most important lesson that the JIP teaches us is not that interdisciplinary work is better because tasks are divided. Rather, the greatest benefit comes from learning from other disciplines and being able to share our discipline with others.

12 Conclusion

The aim of this project was to evaluate if a gate-based, sliding window, quantum reservoir computing algorithm could detect credit-card fraud, and to compare the performance to established classical methods. We built a reproducible and documented QRC pipeline, benchmarked it against classical baselines, and simulated the pipeline under realistic conditions using SpinPulse.

First, Information Processing Capacity benchmarks confirmed that the quantum reservoirs satisfy the fading memory property and generate nonlinear feature combinations across time. The Hamiltonian architectures demonstrated better memory depth, whereas the random circuit's memory decayed in fewer steps. An input reset operation between consecutive time steps was found to be necessary to prevent unitary scrambling and maintain stable fading memory.

Second, regarding readout design, measuring single-Z observables achieved the highest normalized capacity per measurement line. While measuring two-qubit correlators (ZZ) and multi-axis strings (XYZ) generated a larger total number of features, the corresponding increase in IPC was not proportional. This suggests that the additional features provided by ZZ and XYZ measurements were partly redundant and more susceptible to noise.

Third, the predictive performance on credit-card fraud detection showed that QRC provides no practical advantage over classical models. Although the untuned TFIM reservoir initially showed a slight edge over an equal-width classical Echo State Network (ESN-45), this gap disappeared entirely once both models received equal hyperparameter tuning budgets (≈ 0.5 test AP for TFIM versus ≈ 0.6 for ESN-45). Both reservoir approaches fell far short of a trained LSTM. Furthermore, testing a feed-forward Quantum Extreme Learning Machine (QELM) on engineered features revealed no benefit over classical random feature maps, confirming that any value in QRC comes strictly from its temporal processing rather than static quantum embeddings.

Fourth, hardware-level simulations via SpinPulse demonstrated that the shot budget, rather than physical gate dephasing, is the critical failure point for QRC. Physical dephasing ($T_2^* = 10\ \mu$s) shrank feature amplitudes but preserved the underlying memory under exact statevector readout. However, under finite-shot sampling, statistical noise drowned out these small signals, dropping customer fraud detection below that of a simple logistic regression on the raw window at practical budgets ($\leq 10^4$ shots).

Finally, runtime and energy cost analyses indicated that physical spin-qubit hardware is currently ill-suited for this application. Re-preparing the quantum state across thousands of shots per window results in latency between 0.37 s and 3.7 s per transaction, violating real-time authorization deadlines (< 100 ms). Due to continuous cryogenic cooling power, running small 4-qubit reservoirs on physical hardware requires six to seven orders of magnitude more energy than exact simulation on a classical CPU.

In conclusion, while QRC successfully implements temporal memory and nonlinear transformations, it is not competitive with classical baselines for financial fraud detection under current hardware limitations. The modular, validated pipeline delivered in this project provides a reproducible benchmark to evaluate QRC on other dynamical tasks where quantum representations may be better suited.

Declaration of AI use

During this project, generative AI was used in the following ways:

- Refine text redaction and grammar.
- Find and understand the legal documents mentioned in Section 9.
- Generate LaTeXcode for reporting and referencing
- Brainstorm support
- Code debugging and documentation

References

Abemon Engineering (2026). Real-time fraud detection architecture: A production guide. https://www.abemon.es/en/insights/real-time-fraud-detection-architecture-guide/.

Branco, B., Abreu, P., Gomes, A. S., Almeida, M. S. C., Ascensão, J. T., and Bizarro, P. (2020). Interleaved sequence rnns for fraud detection. *arXiv preprint arXiv:2002.05988*.

Bureau of Justice Statistics (2017). Financial fraud in the united states, 2017. Technical report, Office of Justice Programs, U.S. Department of Justice.

Dataintelo (2026). E-commerce fraud analytics market research report 2034: Global industry analysis, growth, share, size, trends, and forecast 2025–2034. Dataintelo Market Research. https://dataintelo.com/report/e-commerce-fraud-analytics-market.

De Nederlandsche Bank (2026). More fraud in payments in 2025. Statistical News, De Nederlandsche Bank. Available at https://www.dnb.nl/en/general-news/statistical-news/2026/more-fraud-in-payments-in-2025/.

Deole, S. (2025). Model Explainability in Machine Learning: Understanding the 'Why' behind predictions.

Domingo, L., Carlo, G., and Borondo, F. (2023). Taking advantage of noise in quantum reservoir computing. arXiv preprint arXiv:2301.06814. https://doi.org/10.48550/arXiv.2301.06814.

Dos Santos, A. F. P. (2026). QRC-Lab: An educational toolbox for quantum reservoir computing. arXiv preprint arXiv:2602.03522. https://doi.org/10.48550/arXiv.2602.03522.

Egger, D. J., Gambella, C., Marecek, J., McFaddin, S., Mevissen, M., Raymond, R., Simonetto, A., Woerner, S., and Yndurain, E. (2020). Quantum Computing for Finance: State-of-the-Art and Future Prospects. *IEEE Transactions on Quantum Engineering*, 1:1–24.

European Parliament and Council of the European Union (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L 2024/1689. Available at https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Ewig, K. (2024). Tools of the TOGAF: Stakeholder map. Medium. https://medium.com/@enivek/tools-of-the-togaf-stakeholder-map-4517981f50d7.

Google for Developers (2026). Thresholds and the confusion matrix. Machine Learning Crash Course.

Hamhoum, W., Cherkaoui, S., Laprade, J.-F., Ahmad, O., and Wang, S. (2025). Multivariate time series forecasting with gate-based quantum reservoir computing on NISQ hardware.

Hamonic, P., Toubeix, M., Haas, G., Nath, J., Dartiailh, M. C., Martinez, B., Bertrand, B., Niebojewski, H., Vinet, M., Bäuerle, C., Balestro, F., Meunier, T., and Urdampilleta, M. (2026). Single-shot in situ readout of a spin qubit unit cell. *Nature Electronics*, 9:868–875.

High-Level Expert Group on Artificial Intelligence (2019). Ethics guidelines for trustworthy AI. European Commission. Available at https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai.

IBM Quantum (2025). Data encoding. IBM Quantum Learning. Accessed: 2026-10-05.

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., and Gambetta, J. M. (2024). Quantum computing with Qiskit.

Jiang, X. (2026). Real-time fraud detection architecture: Where coherence breaks. Tacnode. https://tacnode.io/post/real-time-fraud-detection-architecture.

Kawanabe, M., Cindrak, S., Ludge, K., Shirakashi, J.-i., Shibuya, T., and Imai, H. (2026). Efficient time-series prediction on NISQ devices via time-delayed quantum extreme learning machine.

Le Borgne, Y.-A., Siblini, W., Lebichot, B., and Bontempi, G. (2022). *Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook*. Université Libre de Bruxelles.

Li, Q., Mukhopadhyay, C., Bayat, A., and Habibnia, A. (2026). Quantum reservoir computing for realized volatility forecasting. *Physical Review Research*, 8(2).

Ma, C., Zhang, L., Xing, Z., and Su, J. (2026). Credit card fraud detection under extreme class imbalance using leakage-safe feature selection and ga-based hyperparameter optimization. *Applied Sciences*, 16(13).

Martínez-Peña, R., Nokkala, J., Giorgi, G. L., Zambrini, R., and Soriano, M. C. (2020). Information processing capacity of spin-based quantum reservoir computing systems. *Cognitive Computation*, 15(5):1440–1451. https://doi.org/10.1007/s12559-020-09772-y.

Materassi, D., Warnick, S., Rojas, C., Schoukens, M., and Cross, E. (2024). Explaining complex systems: a tutorial on transparency and interpretability in machine learning models (part I). *IFAC-PapersOnLine*, 58(15):492–496.

Mironowicz, P., Shenoy, A., Mandarino, A., Yilmaz, A. E., and Ankenbrand, T. (2026). Quantum machine learning for quantitative finance. *Quantum Science and Technology*. In press.

Mohanty, A. (2026). False declines: The authorization optimization gap costing $3 for every $1 in processing fees. Optimus. https://optimus.tech/blog/false-declines-the-authorization-optimization-gap-costing-usd3-for-every-usd1-in-processing-fees.

Mujal, P., Martínez-Peña, R., Giorgi, G. L., Soriano, M. C., and Zambrini, R. (2023). Time-series quantum reservoir computing with weak and projective measurements. *npj Quantum Information*, 9:16.

Mujal, P., Martínez-Peña, R., Nokkala, J., García-Beni, J., Giorgi, G. L., Soriano, M. C., and Zambrini, R. (2021). Opportunities in quantum reservoir computing and extreme learning machines. *Advanced Quantum Technologies*, 4(8):2100027.

Nakajima, K. and Fischer, I., editors (2021). *Reservoir Computing: Theory, Physical Implementations, and Applications*. Natural Computing Series. Springer Singapore.

NIS Cooperation Group (2025). A coordinated implementation roadmap for the transition to post-quantum cryptography. Report, European Commission.

Nowak, M. (2026). Rule-based fraud detection in banking: A 2026 cross-vertical guide. Higson Blog. https://www.higson.io/blog/enhancing-fraud-detection-in-banking-with-rule-based-decision-engines.

Ortt, J. R. (2010). Understanding the pre-diffusion phases. In *Understanding the Pre-diffusion Phases*, chapter 2, pages 1–34. Imperial College Press.

Orús, R., Mugel, S., and Lizaso, E. (2019). Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028.

Otieno, W., Zagoskin, A., Balanov, A. G., Gongora, J. T., and Savel'ev, S. E. (2026). A quantum reservoir computing approach to quantum stock price forecasting in technology-markets. *EPJ Quantum Technology*, 13(1):97.

Ozaki, Y., Watanabe, S., and Yanase, T. (2026). Optunahub: A platform for black-box optimization. *Journal of Machine Learning Research*, 27(203):1–10.

Pan, E. (2024). Machine learning in financial transaction fraud detection and prevention. *Transactions on Economics, Business and Management Research*, 5:243–249.

Pandey, T. (2026). When classical baselines are tuned as carefully as the quantum model, does quantum reservoir computing still win? arXiv preprint arXiv:2607.09905v1. https://doi.org/10.48550/arXiv.2607.09905.

Patel, R. B. (2026). Who pays for credit card fraud? liability explained. MoneyLion. https://www.moneylion.com/learn/credit/basics/who-pays-for-credit-card-fraud.

PayCompass (2025). Temporary authorization hold secrets that cost your business thousands (and how to fix them). PayCompass Blog. https://paycompass.com/blog/temporary-hold.

Popova, I. and Gardi, H. A. A. (2025). Credit card fraud detection. *arXiv preprint arXiv:2509.15044*.

Quobly (2026). Alloy quantum computers: technical details. Company presentation, provided to the project team. Includes a comparison of experimental platforms for large-scale quantum computing.

Rainio, O., Teuho, J., and Klén, R. (2024). Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14:6086.

Ravelin (2024). How to reduce false positives and false declines. https://www.ravelin.com/blog/reduce-false-positives-fraud.

Rees, B. and Liu, S. (2025). Supercharging fraud detection in financial services with graph neural networks. NVIDIA Technical Blog. https://developer.nvidia.com/blog/supercharging-fraud-detection-in-financial-services-with-graph-neural-networks.

Santos, J., Nath, J., Pezzutto, M., Savin, V., Meunier, T., and Omar, Y. (2026). Energetics of fault-tolerant spin-qubit quantum computation. arXiv preprint arXiv:2610.22199. https://doi.org/10.48550/arXiv.2610.22199.

Scikit-learn Developers (2026). Precision-recall. Scikit-learn 1.9.1 documentation.

Smith, L. W. (2000). Stakeholder analysis: A pivotal practice of successful projects. In *Project Management Institute Annual Seminars & Symposium*, Houston, Texas. Project Management Institute.

Vermersch, B., Gravier, O., Miscopein, N., Guignon, J., Ramos Marimón, C., Durandau, J., Dartiailh, M., Meunier, T., and Savin, V. (2026). The spinpulse library for transpilation and noise-accurate simulation of spin qubit quantum computers.

Visa Contributor (2025). Fraud detection vs. fraud prevention: Why both matter more than ever. Forbes BrandVoice. https://www.forbes.com/sites/visa/2025/12/12/fraud-detection-vs-fraud-prevention-why-both-matter-more-than-ever/.

Wu, X., Lin, Z., Deng, J., Li, J., and Feng, Y. (2024). Nonmasking-based reservoir computing with a single dynamic memristor for image recognition. *Nonlinear Dynamics*, 112(8):6663–6678.

Appendix

A Model performance metrics

In this appendix we list a few standard performance metrics for binary prediction models.

Most of the following definitions are inspired from a resource that we found online (Google for Developers, 2026).

- True positive (TP): correctly predicted positive outcome. (e.g., transaction was fraudulent and model classified it as fraudulent).
- True negative (TN): correctly predicted negative outcome (e.g., transaction was not fraudulent and model classified it as not fraudulent).
- False positive (FP): incorrectly predicted positive outcome. (e.g., transaction was innocent and model classified it as fraudulent).
- False negative (FN): incorrectly predicted negative outcome (e.g., transaction was fraudulent and model classified it as not fraudulent).

	Actually Positive (1)	**Actually Negative (0)**
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Table 15: Confusion Matrix

- Matthews Correlation Coefficient (MCC): can be interpreted as the Pearson correlation coefficient for binary classification. Similar to accuracy, it considers all metrics derived from the confusion matrix. However, unlike accuracy, it remains reliable even when the data is imbalanced (Rainio et al., 2024).

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- Balanced accuracy: is the arithmetic mean of sensitivity and specificity. It is a useful metric when classes are imbalanced, since accuracy can be a misleading metric in such scenarios.

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}$$

- Recall (or Sensitivity): measure from all positive values, how many had positive predictions. It is also known as true positive rate (TPR). Useful if the cost of a false negative is high (i.e. cancer detection) (Rainio et al., 2024).

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Specificity: measure from all negative values, how many had negative prediction (Rainio et al., 2024).

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- Precision: measures out of all positive predictions, how many had positive values. Useful if the cost of a false positive is high (i.e. spam filters) (Rainio et al., 2024).

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Precision-recall curve: when classes are unbalanced, the precision-recall curve (PRC) is a useful measure of prediction success across all thresholds. The curve is drawn by plotting precision over recall at all possible thresholds. In an ideal model, as illustrated in the figure 15, the plot is a point at (1,1), meaning that both precision and recall are high (Scikit-learn Developers, 2026).

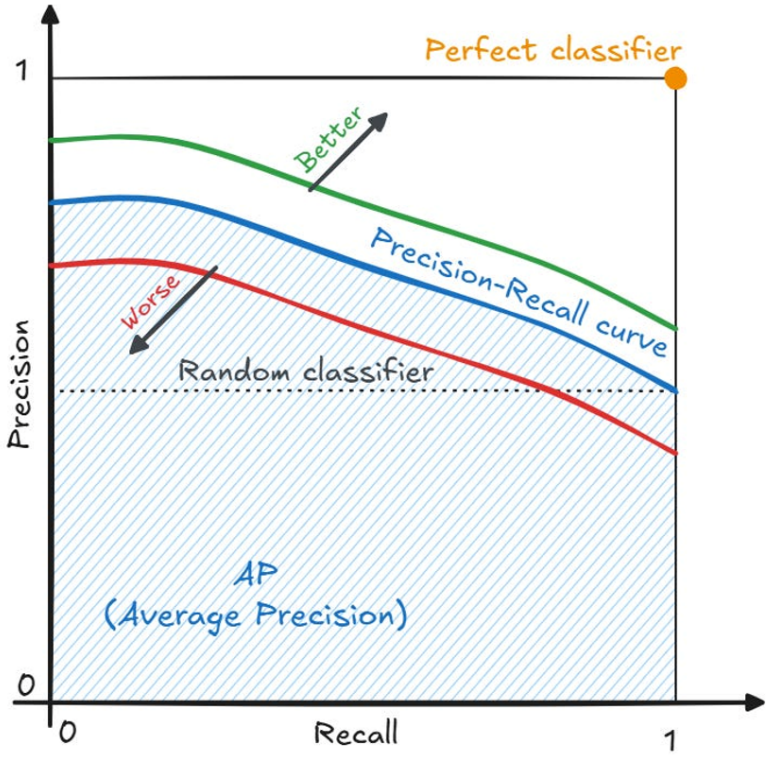


Figure 15: Precision-Recall curve and Average Precision.

The curve does not consider false negatives and only focuses on how well the model handles the positive class. That is why this metric is preferred in the setting of an imbalanced class, where the positive class is the minority.

- Average precision: As shown in figure 15, using the PRC we can compute the average precision; an approximation of the area under the PRC. It is computed as the sum of precisions achieved at each threshold, weighted by the increase in recall from previous threshold (Scikit-learn Developers, 2026):

$$AP = \sum_n (R_n - R_{n-1}) P_n$$

B Dataset selection and chronological partitioning

B.1 Assessment of candidate datasets

The research objective requires a persistent customer or card identifier, transaction timestamps, and fraud labels, enabling sequential windows to be constructed from the same customer's historical activity. The team's initial dataset assessment (documented in the project's *Design choices* notes) identified two suggested public datasets that did not jointly satisfy these requirements:

ULB credit-card fraud dataset. The publicly available dataset includes a transaction-time variable and anonymized principal-component features, but not a persistent card or customer identifier. Consequently, one cannot reconstruct the recent transaction history of a particular card for customer-specific temporal classification. It remains useful for transaction-level, non-contextual benchmarks. `https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`

Credit Card Fraud Detection Dataset 2023. The suggested dataset supplies anonymized transaction-level attributes and fraud labels, but no transaction timestamp. Without a temporal ordering, it cannot support the same customer-history experiment. `https://www.kaggle.com/datasets/nelgiriyewithana/credit-card-fraud-detection-dataset-2023`

Other potential sources were considered, but no accessible real-world dataset satisfying all of the project's temporal and customer-linkage requirements was identified among the candidates reviewed. We therefore generated a controlled synthetic benchmark following the openly documented simulator of Le Borgne et al. (Le Borgne et al., 2022). This addresses the requirements of the chosen experiment while limiting the external validity of the findings.

B.2 Split strategies considered

The team compared four chronological strategies. The key criteria were avoidance of temporal leakage, availability of customer history, preservation of seasonal variation, and evaluation on genuinely later observations.

1. **Monthly global split.** Within each month, all transactions are ordered by time and a percentage division is made. Although each month's local order is preserved, training data from later months may coexist with test data from earlier months, so the pooled evaluation does not represent a single future holdout period.

2. **Monthly customer-specific split.** Within each month and customer, transactions are ordered and split by percentage. This increases the likelihood that a customer's tested transactions have earlier customer history, but it fragments time across customers and months and complicates leakage-safe evaluation.

3. **Customer-specific chronological split.** All transactions for each customer are ordered across the whole year and then split by percentage. This preserves customer-level chronology, but different customers can have different cut-off dates, so the resulting pooled train/test sets do not share a single temporal boundary.

4. **Global chronological split.** All transactions are separated by the same calendar boundaries. Every training observation precedes every validation observation, which in turn precedes every test observation. Customers occurring in multiple periods retain their earlier history, while previously unseen customers constitute realistic cold-start cases.

The final decision is to use a modified version of Strategy 3, replacing customer-specific percentage cut-offs with fixed calendar dates. In terms of assigning transactions to the train, validation, and test partitions, this produces the global chronological split of Strategy 4; customer-specific grouping occurs later, when the input windows and historical features are constructed. It is therefore inaccurate to describe the implemented partitions as a separate percentage split performed within each customer.

B.3 Implemented split and leakage control

The reported experiments use `qrc/datasets/split_dataset.py` with specific cut-off dates. For a full calendar year beginning on 1 January 2025, the partitions are

$$\text{split}(t) = \begin{cases} \text{train}, & t \leq 12 \text{ September } 2025, \\ \text{validation}, & 12 \text{ September } 2025 < t \leq 5 \text{ November } 2025, \\ \text{test}, & t > 5 \text{ November } 2025. \end{cases} \tag{10}$$

The respective intervals contain 255, 54, and 56 calendar days, corresponding to approximately 69.9%, 14.8%, and 15.3% of the full year. These percentages are calendar-day shares, not measured proportions of transaction rows or fraud labels.

Rolling features and customer windows may use transactions from an earlier partition, provided that those transactions occurred before the prediction timestamp. They cannot use future observations, and imputation and scaling parameters are fitted only on training data. This construction preserves the information that would be available to an operational fraud detector at each prediction time. It does not remove temporal distribution shift, seasonal differences between the periods, or the challenge of customers with insufficient previous history; these remain limitations of the evaluation design.

C Raw-feature signal by fraud category

C.1 Objective and procedure

This supplementary analysis tests whether transaction-level time information adds measurable predictive signal beyond log transaction amount for the synthetic fraud categories. The analysis is particularly relevant to the single-feature input used by the customer-fraud sliding-window QRC experiments. It is a lightweight classical diagnostic, rather than a quantum ablation or a measurement of the value of customer history.

The script `experiments/analyze_raw_features.py` loads the existing train and validation set; the test set is not accessed. The feature transformation and fraud categorization reuse the project's `qrc` functions. Each category is evaluated as a separate binary task against legitimate transactions, excluding transactions from the other two fraud categories. The positive customer category is restricted further to scenario 3 fraud with a post-modification transaction amount equal or below 220. This restriction is important because amounts above the threshold belong to the amount-fraud category in the project evaluation.

For each feature group, a separate pipeline imputes missing values with the training median, standardizes features using training statistics, and fits class-balanced logistic regression. The non-historical input groups are (i) log transaction amount, (ii) time of day represented by sine and cosine, (iii) day of week represented by sine and cosine, (iv) calendar day, (v) all five time variables, and (vi) log amount together with the five time variables. No customer identifier, terminal identifier, fraud-scenario identifier, or historical rolling statistic is used as a predictor. The reported metric is Average Precision (AP) on the validation subset for each task. The random-ranking AP reference is approximately that task's validation fraud prevalence.

C.2 Results

Category	Fraud n	Prevalence	Log amount AP	Time AP	Combined AP	ΔAP
Amount	1,536	0.01535	1.00000	0.01520	1.00000	0.00000
Terminal	575	0.00580	0.00558	0.00573	0.00574	0.00016
Customer	1,336	0.01337	0.14662	0.01309	0.13720	-0.00942

Table 16: Validation-set predictive signal from raw transaction attributes. Each fraud category is compared separately with legitimate transactions; the category prevalences are therefore conditional on this filtering. The change in AP is combined minus log amount.

Category	Time of day	Day of week	Calendar day	All time
Amount	0.01550	0.01525	0.01658	0.01520
Terminal	0.00564	0.00573	0.00552	0.00573
Customer	0.01349	0.01345	0.01301	0.01309

Table 17: Average precision of the individual non-historical time-feature groups on validation.

Amount fraud. The amount-category model achieves AP 1.00000 using log amount alone because the generator deterministically labels transactions with `TX_AMOUNT`$>$ 220 as fraudulent. Its result mainly validates the analysis against this known rule; it is not evidence that the classifier discovered a complex behavioral pattern. Adding time features leaves AP unchanged at the reported precision.

Terminal fraud. Log amount achieves AP 0.00558 compared with category prevalence 0.00580 (ROC-AUC 0.49073). Adding timestamp features yields AP 0.00574, still close to random-ranking performance. Neither timestamp features nor their combination with amount yields a meaningful improvement. Since the fraud-generating mechanism compromises specific terminals, its main signal

is expected in terminal identity and its recent history, neither of which is present in this ablation. Thus the result must not be interpreted as an upper bound on terminal-fraud detectability.

Customer fraud. For the scenario 3 customer category restricted to a final amount at or below 220, log amount alone gives AP 0.14662 (ROC-AUC 0.77633) at prevalence 0.01337, roughly eleven times the random-ranking AP. This is consistent with the synthetic scenario's multiplication of selected transaction amounts by five. The absolute AP nevertheless remains low, reflecting the difficulty of identifying rare fraudulent transactions from a single amount. Time features alone yield AP 0.01309, near the random baseline; adding all five time variables to log amount yields AP 0.13720, a change of -0.00942. Consequently, in this particular transaction-level linear diagnostic, the extra timestamp inputs provide no measured AP gain.

C.3 Implications and limitations

The evidence supports log amount as a parsimonious raw input when the study aims to focus its limited quantum-simulation resources on temporal reservoir dynamics. Compared with log amount plus the five time variables, amount-only input reduces the number of directly encoded features from six to one; the resulting computational saving depends on the chosen encoding and circuit implementation and was not measured by this ablation.

The finding is restricted to the observed synthetic dataset, chronological validation period, tested variables, and class-balanced logistic-regression scoring. No confidence intervals or repeated data replicates were computed, so the observed decrease in AP from adding time features does not establish that those features are universally helpful. Neither customer identity nor engineered historical behavior is included; the test cannot establish that these are uninformative. In particular, the ability of a QRC to extract information from sequences of transaction amounts must be assessed separately with the sliding-window experiments. Finally, the AP values reported here use validation data and are not directly comparable with sequence-model test AP unless the data population, prediction task, and split are verified to match.

D Data generation, preprocessing and evaluation implementation

D.1 Generation procedure and transaction schema

Parameter	Value
Number of customers	1,000
Number of terminals	10,000
Simulation period	365 days from 1 January 2025
Customer-terminal association radius	5 spatial units
Profile-generation random seed	0
Customer compromise rate	6 customers per day
Terminal compromise rate	2 terminals per day

Table 18: Generation settings for the synthetic transaction dataset

The transaction generator is adapted from the *Reproducible Machine Learning for Credit Card Fraud Detection - Practical Handbook* (Le Borgne et al., 2022). The experiment dataset was generated with `create_dataset.py` and then partitioned with `split_dataset.py`, both located in `qrc/datasets/`. Table 18 summarizes the generation parameters used.

Customer and terminal positions are sampled in a two-dimensional synthetic space. For each customer c, a mean transaction amount $\mu_c \sim \mathcal{U}(5, 100)$, an amount standard deviation $\sigma_c = \mu_c/2$, and an expected daily transaction count $\lambda_c \sim \mathcal{U}(0, 4)$ are generated. Eligible terminals are those located within the specified radius.

Daily transaction counts follow a Poisson distribution,

$$N_{c,d} \sim \text{Poisson}(\lambda_c)$$

Transaction times, expressed in seconds after midnight, are sampled from a normal distribution:

$$T \sim \mathcal{N}\left(\frac{86400}{2}, 20000^2\right)$$

The mean corresponds to noon, while the standard deviation of 20,000 seconds produces a concentration of transactions during daytime. Transaction attempts with generated times outside the interval are discarded.

Transaction amounts are sampled according to

$$A_c \sim \mathcal{N}(\mu_c, \sigma_c^2)$$

If a sampled amount is negative, it is replaced by a draw from $\mathcal{U}(0, 2\mu_c)$. Amounts are rounded to two decimal places. The resulting transactions are sorted chronologically and assigned consecutive transaction identifiers.

The saved table contains a transaction key (`TRANSACTION_ID`), a timestamp (`TX_DATETIME`), a customer key (`CUSTOMER_ID`), a terminal key (`TERMINAL_ID`), and an amount (`TX_AMOUNT`). Two additional time columns record elapsed seconds and days (`TX_TIME_SECONDS`, `TX_TIME_DAYS`). Finally, `TX_FRAUD` is the binary target and `TX_FRAUD_SCENARIO` is evaluation metadata; neither is a direct model predictor.

The fraud mechanisms are applied to the generated legitimate transaction stream in the following order:

1. **High-amount fraud (scenario 1):** transactions exceeding an amount of 220 when this rule is applied are labelled fraudulent.

2. **Compromised terminals (scenario 2):** two (2) terminals are selected on each simulated day; transactions at those terminals over the subsequent 28-day interval receive a fraud label.

3. **Compromised customers (scenario 3):** six (6) customers are selected on each simulated day; approximately one third of their transactions in the subsequent 14-day interval are labelled fraudulent and have their amounts multiplied by five.

D.2 Chronological partitioning and configuration precedence

The experiment uses a calendar boundaries in `split_dataset.py` to split the dataset: training transactions have $t \leq$ 12 September 2025; validation transactions satisfy 12 September 2025 $< t \leq$ 5 November 2025; and test transactions have $t >$ 5 November 2025. The periods comprise 255, 54, and 56 days (approximately 69.9%, 14.8%, and 15.3% of the calendar year); the percentages are not assertions about transaction counts or class frequencies. The comparison of alternative partitioning strategies are documented in B.

Split	Transactions	Amount	Terminal	Customer	Total fraud
Train	483,807	7,122	2,495	6,176	15,793
Validation	102,003	1,536	575	1,336	3,447
Test	106,153	1,680	581	1,401	3,662
Total	691,963	10,338	3,651	8,913	22,902

Table 19: Transaction counts and fraud-category composition of the dataset

D.3 Historical feature construction

The shared preparation pipeline first loads the three raw partitions and combines them into a chronological transaction stream. It sets aside fraud-scenario labels for subsequent category evaluation, then calls `FeatureEngineer` to construct the predictors. The engineered rows are subsequently returned to their original date partitions. Thus feature construction has access to past observations across split boundaries, without using future observations for an earlier prediction.

Family	Definition and purpose
Transaction amount	Cleaned amount, $\log(1 + \max\{0, A_t\})$, and an amount-missingness indicator.
Transaction time	Hour, weekday, month, weekend/night indicators, and cyclic sine/cosine time-of-day variables.
Customer history	Prior customer transaction counts and average amounts in rolling 1-, 7-, 30-, 90- and 180-day windows.
Terminal history	Historical terminal counts and historical confirmed-fraud rates over the same rolling windows, with a seven-day label-availability delay.
Behavioral comparisons	Time since the customer's prior transaction, current amount relative to a past average, and combinations/deviations comparing short- and long-term activity or spending.

Table 20: Main predictor families constructed by `qrc.features.FeatureEngineer`

Customer rolling features are explicitly left-closed. With $\mathcal{H}_c(t; w) = \{i : c_i = c,\ t - w \leq t_i < t\}$ denoting the same-customer transactions preceding the current time, a representative count and average amount are

$$N_c^{(w)}(t) = |\mathcal{H}_c(t; w)|, \tag{11}$$

$$\overline{A}_c^{(w)}(t) = \frac{\sum_{i \in \mathcal{H}_c(t;w)} A_i}{|\mathcal{H}_c(t; w)|}, \qquad \text{when } |\mathcal{H}_c(t; w)| > 0. \tag{12}$$

The implementation fills missing initial rolling aggregates with zero. A transaction does not contribute to its own historical count or mean. These features make deviations from an individual customer's usual spending and transaction rate explicit.

Terminal-risk features represent a simulated verification delay: a fraud label becomes available for use as historical information only after seven days. For terminal m and window width w, define

$$\mathcal{H}_m^{\text{known}}(t; w) = \{i : m_i = m,\ t - (w + 7\text{d}) \leq t_i < t - 7\text{d}\}. \tag{13}$$

The corresponding terminal count is the number of transactions in this interval, and terminal risk is the proportion with a confirmed fraud label; the implementation uses zero when the denominator is zero. In particular, the seven most recent days are excluded from the terminal-risk numerator and denominator. This prevents the model from using a fraud-confirmation outcome before its assumed availability.

D.4 Preprocessing and customer-specific windows

`DataProcessor` selects a feature set, replaces infinite values with missing values, imputes missing entries using the training-set median for each feature, and then fits a `MinMaxScaler` on the training split. For a non-constant training feature j, the transformation can be written as

$$x'_{ij} = \frac{x_{ij} - a_j^{\text{train}}}{b_j^{\text{train}} - a_j^{\text{train}}}, \tag{14}$$

where a_j^{train} and b_j^{train} are its training minimum and maximum. The fitted imputation and scaling parameters are applied unchanged to validation and test data.

After preprocessing, each split has a flat input matrix of shape (N_s, F). Customer-specific windows are constructed using the `window_by_customer_id` function, which converts these matrices to (N_s, L, F). Each window contains the customer's most recent L transactions, ordered oldest to newest, and ends with the transaction being classified. Historical rows may come from an earlier split because the sequence builder processes training, validation and test in chronological order, but each window contains no later customer transaction. A customer with fewer than L available observations is padded on the left by repetitions of that customer's earliest available transaction. A `has_full_history` indicator is produced to identify complete windows; the reviewed experimental fitting path does not exclude padded windows solely on that basis.

D.5 Evaluation categories and ranking metrics

The project retains one binary target (fraud versus legitimate) for model fitting. For diagnostic evaluation, `qrc.evaluation.categorize` defines categories from the final transaction amount A_t, fraud label y_t, and stored scenario s_t:

$$\text{category}(t) = \begin{cases} \text{legitimate}, & y_t = 0, \\ \text{amount}, & y_t = 1 \text{ and } A_t > 220, \\ \text{terminal}, & y_t = 1,\ A_t \le 220,\ s_t = 2, \\ \text{customer}, & y_t = 1,\ A_t \le 220,\ s_t = 3. \end{cases} \tag{15}$$

This reclassification is important: a scenario-3 transaction multiplied above 220 is evaluated as amount fraud, not as the harder low-amount customer category. For category-specific Average Precision (AP), only legitimate transactions and positives of the selected category are retained; frauds belonging to other categories are excluded. Therefore, each category has its own positive-class prevalence, which is also its approximate random-ranking AP reference. Category AP values should not be interpreted without this prevalence context.

AP and ROC-AUC measure ranking without choosing a particular binary threshold. The full fraud-detection study also reports threshold-dependent precision, recall, false-positive rate, F1, Matthews correlation coefficient and balanced accuracy. Their conceptual definitions are collected in A. The category-based raw-feature ablation is a separate validation-only, single-transaction logistic-regression diagnostic, documented in C; it is not numerically interchangeable with customer-window test AP from Section 7.2.

D.6 Validation decision threshold

A fitted model returns a continuous score s_i for transaction i. For each model configuration, the operating threshold is determined solely from validation scores and labels. The implemented rule is

$$\tau^* = \arg\min_{\tau} \text{FP}_{\text{val}}(\tau) \quad \text{subject to} \quad \text{Recall}_{\text{val}}(\tau) \ge r_{\min}, \tag{16}$$

with the default $r_{\min} = 0.80$. A transaction is flagged when $s_i \geq \tau$. The algorithm sorts the distinct validation scores in descending order, computes cumulative true- and false-positive counts at each candidate threshold, and selects a feasible threshold with the fewest false positives. If false-positive counts tie, it prefers greater recall; any remaining tie is resolved in favor of the higher threshold. The resulting numerical τ^* is frozen before the corresponding model is evaluated on the future test period. Different models can have different thresholds despite sharing the same selection rule.

Threshold-dependent metrics on the validation set are operating-point selection diagnostics: the validation labels have already been used to choose τ^*. Test metrics at the frozen threshold measure subsequent-period performance, provided the test results have not influenced the choice of model configuration or recall constraint. The reported numerical thresholds belong with the experimental results, not with the predefined methodology.

D.7 Daily Card Precision@K

The implementation provides a separate operational ranking metric for a limited daily investigation budget. On each day d, transactions are grouped by customer/card c. The card score and label are the maximum transaction score and maximum fraud label for that customer on that day:

$$S_{c,d} = \max_{i \in (c,d)} s_i, \qquad Y_{c,d} = \max_{i \in (c,d)} y_i. \tag{17}$$

Cards are ranked by decreasing $S_{c,d}$. After removing cards already detected on earlier days, the first K eligible cards form the day's review queue $\mathcal{T}_{d,K}$. Daily precision and the final score are

$$\mathrm{CP@}K(d) = \frac{1}{K} \sum_{c \in \mathcal{T}_{d,K}} Y_{c,d}, \qquad \mathrm{CP@}K = \frac{1}{D} \sum_{d=1}^{D} \mathrm{CP@}K(d). \tag{18}$$

The denominator is the fixed capacity K, even if fewer than K eligible cards remain on a particular day. By default, a fraudulent card found within the top K is removed from all subsequent daily rankings so that it cannot be counted repeatedly. The experiment CLI defaults to $K = 100$. Card Precision@K is a ranking and review-capacity measure; it does not use the binary threshold of equation (16).

D.8 Reproducibility, validity and interpretation

The controlled synthetic benchmark allows model comparisons on consistent labels, feature sets, sequence windows, date partitions and evaluation rules. Reproducibility is supported by explicit generation parameters, chronological split dates, training-only preprocessing, versioned experiment configurations and fixed random seeds.

D.9 Configuration search space for multi-objective optimization

Each trial is evaluated on reservoir seeds 42–44. The reported AP is the mean of the three validation customer APs. The three hardware costs are computed from the circuit of a single window, which does not depend on the data. The circuit depth is that of the L rounds of encoding and reservoir after transpilation to the gate set on a linear chain of qubits, using Qiskit's optimisation level 1, so two-qubit gates between distant qubits pay for the SWAPs they require. Measurements, and the single-qubit rotations into each measurement basis, are not counted. The number of measurement bases is the number of groups into which the observables split when grouped by qubit-wise commutation. Each group corresponds to one circuit configuration that must be executed separately. The sampler is Optuna's multi-objective TPE with multivariate, grouped modelling of the conditional parameters. Its first 10 trials are drawn at random.

Parameter	Applies to	Scale	Range/values
Encoding	all	categorical	angle, re-uploading
Memory qubits	all	integer	0 to min(5, $Q_{\max} - n_{\text{in}}$); here 0–4
Re-uploading layers	re-uploading	integer	2–3
Encoding scaling	all	linear	$[\pi/4, \pi]$
Reservoir	all	categorical	random circuit, TFIM, TFIM with longitudinal field, native TFIM, native exchange
Circuit depth	random circuit	integer	1–6
Entangling gate	random circuit	categorical	CX, CRY
Coupling topology	Hamiltonians	categorical	linear (fixed in this study)
Evolution time t	Hamiltonians	logarithmic	[0.1, 10]
Trotter steps	Hamiltonians	integer	2–3
Observables	all	categorical	Z, Z+ZZ, Z+ZZ-all, XYZ, XYZ-strings, XYZ+XYZ-2body, XYZ+XYZ-2body-all
Readout regularisation α	all	logarithmic	$[10^{-1}, 10^{5}]$

Table 21: Hyperparameters searched by the multi-objective study. $Q_{\max} = 5$ is the qubit cap and $n_{\text{in}} = 1$ the number of input qubits.

E Mathematical formulation of IPC implementation

This appendix summarizes the exact formulas used to construct targets and evaluate the Information Processing Capacity.

E.1 Normalized Legendre polynomials

For uniform input $s \sim \mathcal{U}[-1, 1]$, standard Legendre polynomials $P_d(s)$ are normalized to have unit variance:

$$\bar{P}_d(s) = \sqrt{2d+1}\, P_d(s). \tag{19}$$

To match the $[0, 1]$ input range required by quantum angle encoding, inputs are rescaled as:

$$u_t = \frac{s_t + 1}{2}. \tag{20}$$

E.2 Target functions by degree

Target functions of total degree d are constructed using products of polynomials across time delays $\tau \in [0, \tau_{\max}]$:

$$\bar{y}_t = \prod_j \bar{P}_{d_j}(s_{t-\tau_j}) \quad \text{where} \quad \sum_j d_j = d. \tag{21}$$

The code generates targets up to degree $d = 3$:

- **Degree 1 (Linear):**

$$\bar{P}_1(s_{t-\tau}) \quad \text{for } 0 \le \tau \le \tau_{\max} \tag{22}$$

- **Degree 2 (Quadratic):**

$$\bar{P}_2(s_{t-\tau}) \quad \text{and} \quad \bar{P}_1(s_{t-\tau_1})\bar{P}_1(s_{t-\tau_2}) \quad (0 \le \tau_1 < \tau_2 \le \tau_{\max}) \tag{23}$$

- **Degree 3 (Cubic):**

$$\bar{P}_3(s_{t-\tau}), \tag{24}$$
$$\bar{P}_2(s_{t-\tau_1})\bar{P}_1(s_{t-\tau_2}) \quad (\tau_1 \neq \tau_2), \tag{25}$$
$$\bar{P}_1(s_{t-\tau_1})\bar{P}_1(s_{t-\tau_2})\bar{P}_1(s_{t-\tau_3}) \quad (0 \le \tau_1 < \tau_2 < \tau_3 \le \tau_{\max}). \tag{26}$$

E.3 Capacity calculation and aggregation

Given true target values $\bar{\mathbf{y}}$ and readout predictions $\hat{\mathbf{y}}$, the capacity $C(\bar{\mathbf{y}})$ is computed as:

$$C(\bar{\mathbf{y}}) = 1 - \frac{\mathrm{MSE}(\bar{\mathbf{y}}, \hat{\mathbf{y}})}{\mathrm{Var}(\bar{\mathbf{y}})} = 1 - \frac{\frac{1}{M}\sum_{t=1}^{M}(\bar{y}_t - \hat{y}_t)^2}{\frac{1}{M}\sum_{t=1}^{M}(\bar{y}_t - \langle\bar{\mathbf{y}}\rangle)^2}. \tag{27}$$

Values at or below a noise threshold ϵ (set to 10^{-3}) are discarded. The capacities are then summed by degree:

$$C_d = \sum_{\substack{\bar{\mathbf{y}}\in\mathcal{Y}_d \\ C(\bar{\mathbf{y}})>\epsilon}} C(\bar{\mathbf{y}}). \tag{28}$$

The total capacity is the sum over all degrees:

$$C_{\mathrm{total}} = \sum_{d=1}^{d_{\max}} C_d. \tag{29}$$

Finally, normalizing by the number of measured observables N_{obs} gives:

$$C_{\mathrm{norm}} = \frac{C_{\mathrm{total}}}{N_{\mathrm{obs}}}, \tag{30}$$

where $C_{\mathrm{total}} \leq N_{\mathrm{obs}}$ represents the theoretical maximum.

F Reservoir architectures and Hamiltonians

This appendix details the mathematical formulation and circuit implementations of the quantum reservoir architectures.

Each reservoir operates on an n-qubit register consisting of n_{in} input qubits and n_{mem} memory qubits ($n = n_{\text{in}} + n_{\text{mem}}$). Parameters are drawn once upon initialization from a fixed random seed and held constant throughout all experiments.

F.1 Hamiltonian models

1. **Transverse-Field Ising Model (TFIM):** The standard textbook Ising model with random ferromagnetic coupling and uniform transverse field ν:

$$\hat{H} = \sum_{(i,j)\in\mathcal{E}} J_{ij}\hat{\sigma}_i^x\hat{\sigma}_j^x + \nu\sum_{i=0}^{n-1}\hat{\sigma}_i^z, \tag{31}$$

   where $J_{ij} \sim \mathcal{U}[0,\nu]$ and default $\nu = 1.0$.

2. **Longitudinal-Field Ising Model (LFIM):** Adds site-dependent longitudinal fields g_i to break integrability and induce chaotic dynamics:

$$\hat{H} = \sum_{(i,j)\in\mathcal{E}} J_{ij}\hat{\sigma}_i^x\hat{\sigma}_j^x + \nu\sum_{i=0}^{n-1}\hat{\sigma}_i^z + \sum_{i=0}^{n-1} g_i\hat{\sigma}_i^x, \tag{32}$$

   where $J_{ij} \sim \mathcal{U}[0,\nu]$ and $g_i \sim \mathcal{U}[-1,1]$.

3. **Native Ising Model (NIM):** Rotates the interaction term to $\hat{\sigma}_i^z\hat{\sigma}_j^z$ to match the native R_{ZZ} gate basis of superconducting/transmon processors:

$$\hat{H} = \sum_{(i,j)\in\mathcal{E}} J_{ij}\hat{\sigma}_i^z\hat{\sigma}_j^z + \nu\sum_{i=0}^{n-1}\hat{\sigma}_i^z + \sum_{i=0}^{n-1} g_i\hat{\sigma}_i^x, \tag{33}$$

   where $J_{ij} \sim \mathcal{U}[0,\nu]$, $\nu = 1.0$, and $g_i \sim \mathcal{U}[-1,1]$.

4. **Native Exchange Model (NEX):** Extends the native Ising model with tunable Heisenberg XY exchange interactions controlled by scale parameter η:

$$\hat{H} = \sum_{(i,j)\in\mathcal{E}} J_{ij}\left(\hat{\sigma}_i^z\hat{\sigma}_j^z + \eta\hat{\sigma}_i^x\hat{\sigma}_j^x + \eta\hat{\sigma}_i^y\hat{\sigma}_j^y\right) + \nu\sum_{i=0}^{n-1}\hat{\sigma}_i^z + \sum_{i=0}^{n-1} g_i\hat{\sigma}_i^x, \tag{34}$$

   where $J_{ij} \sim \mathcal{U}[0,\nu]$, $g_i \sim \mathcal{U}[-1,1]$, and default parameters are $\nu = 1.0$ and $\eta = 1.0$.

F.2 Random circuit reservoir

The random circuit reservoir uses discrete quantum gates rather than Hamiltonian evolution. It consists of D layers, each containing an entangling layer followed by local rotations:

1. **Entangling layer:** Each qubit i interacts with its next neighbour, $(i+1) \bmod n$, forming a ring. The entangler is either a CNOT gate or a controlled-R_y gate with a randomly sampled angle: $\text{CR}_y(\theta_{d,i})$, $\theta_{d,i} \sim \mathcal{U}[0, 2\pi)$.

2. **Rotation layer:** Each qubit undergoes two single-qubit rotations:

$$R_z(\phi_{d,i,1})R_y(\phi_{d,i,0}), \qquad \phi_{d,i,k} \sim \mathcal{U}[0, 2\pi).$$

The rotation angles are sampled independently for each layer and qubit. The resulting circuit combines entanglement with randomly parameterized local operations.

Figure 16 illustrates one layer of a three-qubit reservoir using CNOT entanglers. The final entangler wraps around from the last qubit to the first.

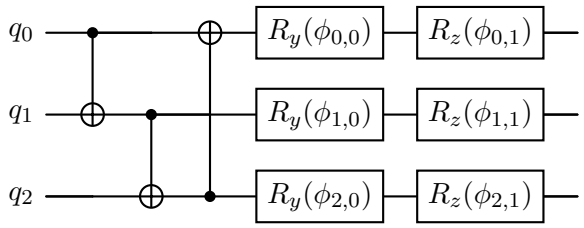


Figure 16: One random circuit reservoir layer. Three CNOT gates connect neighbouring qubits in a ring, followed by independent local R_y and R_z rotations. Alternatively, all CNOT gates can be replaced by controlled-R_y gates with randomly sampled angles.

F.3 Hamiltonian evolution

Hamiltonian-based reservoirs evolve the quantum state under a Hamiltonian $\hat{H}$ for a fixed duration t, using the unitary

$$U(t) = e^{-i\hat{H}t}, \tag{35}$$

where $\hbar = 1$. Since the Hamiltonian is decomposed into single- and two-qubit Pauli terms,

$$\hat{H} = \sum_k \hat{H}_k, \tag{36}$$

the evolution is approximated using first-order Trotterization with S steps:

$$U(t) \approx \left(\prod_k e^{-i\hat{H}_k t/S} \right)^S. \tag{37}$$

Each step applies the evolution associated with each term separately, approximating the combined Hamiltonian evolution.

G Operational metrics under a false-positive budget

In this appendix, we present the resulting operational metrics for the models discussed in subsection 7.3, when another decision threshold selection rule than that presented in subsection 6.4 (Eq. 8) is used. Namely, we here use a false-positive budget rule that can be mathematically formulated as follows:

$$\tau = \arg\max_{t} \ \mathrm{Recall}_{\mathrm{val}}(t) \quad \text{subject to} \quad \mathrm{FPR}_{\mathrm{val}}(t) \leq f_{\max}, \tag{38}$$

with $f_{\max} = 1\%$. As before, the threshold is applied unchanged to the test split.

Table 22 reports the resulting metrics. Under this rule, all models raise about the same number of alerts (60–70 per day, of which 19 are false), so the comparison is now about how much fraud each model catches. The ranking is the same as under the minimum-recall rule. At $f_{\max} = 1\%$, the tuned TFIM catches 54% of customer fraud, against 66% for the tuned ESN-45, 44% for logistic regression and 76% for the LSTM. Figure 17 shows that the LSTM and the ESN-45 lead at every budget from 0.1% to 5%. The TFIM gives the same recall as the logistic regression at $f_{\max} = 0.1\%$, but it is ahead from $f_{\max} = 0.2\%$ onwards.

Table 22: Operating point on the test split at $L = 15$: the threshold maximises validation recall subject to a validation FPR $\leq 1\%$ (Eq. 38) and is applied unchanged to test. Mean over 10 seeds (logistic regression is deterministic). Alerts per day count all flagged transactions, true and false.

Model	Precision	Recall	FPR	F1	MCC	Balanced accuracy	Recall (customer)	Alerts per day
TFIM (tuned)	0.697	0.665	1.03%	0.681	0.670	0.827	0.535	62
ESN-45 (tuned)	0.708	0.707	1.04%	0.708	0.697	0.848	0.657	65
Logistic	0.676	0.611	1.05%	0.642	0.631	0.800	0.435	59
LSTM	0.724	0.751	1.02%	0.737	0.728	0.870	0.762	68

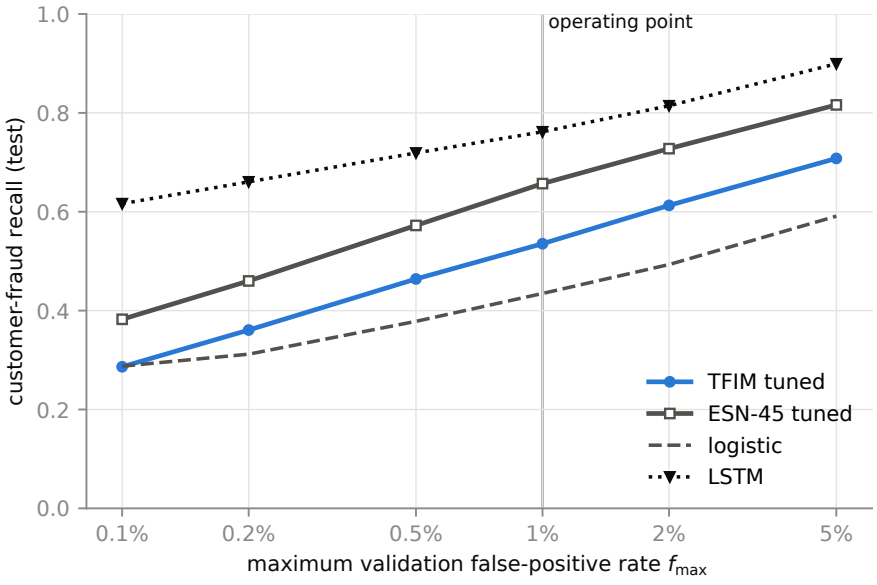


Figure 17: Customer-fraud recall on the test split against the maximum validation false-positive rate $f_{\max}$ of the threshold rule (Eq. 38), at $L = 15$, mean over 10 seeds. Each model's threshold is chosen on its own validation scores and applied unchanged to test. The vertical line marks the operating point $f_{\max} = 1\%$.

H Classical fixed random feature maps

The classical baselines use fixed random feature maps to transform the input features before training the classical readout. Let $\mathbf{x} \in \mathbb{R}^d$ denote an input vector and let m denote the number of random features. Both maps first apply a random affine transformation,

$$\mathbf{z}(\mathbf{x}) = \mathbf{x}W + \mathbf{b}, \tag{39}$$

where $W \in \mathbb{R}^{d\times m}$ is a random weight matrix and $\mathbf{b} \in \mathbb{R}^m$ is a random bias vector. The weights and biases are initialized using a fixed random seed and remain unchanged during training. The feature maps do not use class labels; only the subsequent classical readout is trained.

H.1 Tanh random feature map

The Tanh map applies the hyperbolic tangent element-wise to the random affine transformation:

$$\mathbf{h}(\mathbf{x}) = \tanh(\mathbf{x}W + \mathbf{b}). \tag{40}$$

The entries of the weight matrix and bias vector are sampled as

$$W_{ij} \sim \mathcal{N}\left(0, \frac{s^2}{d}\right), \tag{41}$$

$$b_j \sim \mathcal{U}[-1, 1], \tag{42}$$

where $s > 0$ is the map scale, d is the input width, and $\mathcal{U}$ denotes the uniform distribution. In the implementation, the weights are generated with standard deviation $s/\sqrt{d}$. The default scale is $s = 1$.

The element-wise Tanh activation maps the transformed values to $(-1, 1)$, producing an m-dimensional nonlinear feature vector. The weights and biases are sampled independently using NumPy's random number generator initialized with the specified seed.

H.2 Random Fourier features

The Random Fourier Feature (RFF) map approximates the Gaussian radial basis function (RBF) kernel,

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma\|\mathbf{x} - \mathbf{y}\|^2\right), \tag{43}$$

where $\gamma > 0$ controls the kernel length scale.

The entries of the random weight matrix and bias vector are sampled as

$$W_{ij} \sim \mathcal{N}(0, 2\gamma), \tag{44}$$

$$b_j \sim \mathcal{U}[0, 2\pi]. \tag{45}$$

The feature map is

$$\boldsymbol{\phi}(\mathbf{x}) = \sqrt{\frac{2}{m}}\cos(\mathbf{x}W + \mathbf{b}), \tag{46}$$

where the cosine is applied element-wise. The normalization factor $\sqrt{2/m}$ scales the resulting m-dimensional feature vector.

The corresponding feature inner product approximates the RBF kernel:

$$\boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{y}) \approx k(\mathbf{x}, \mathbf{y}). \tag{47}$$

I Full IPC results across varying parameters

I.1 Window lengths and maximum delays

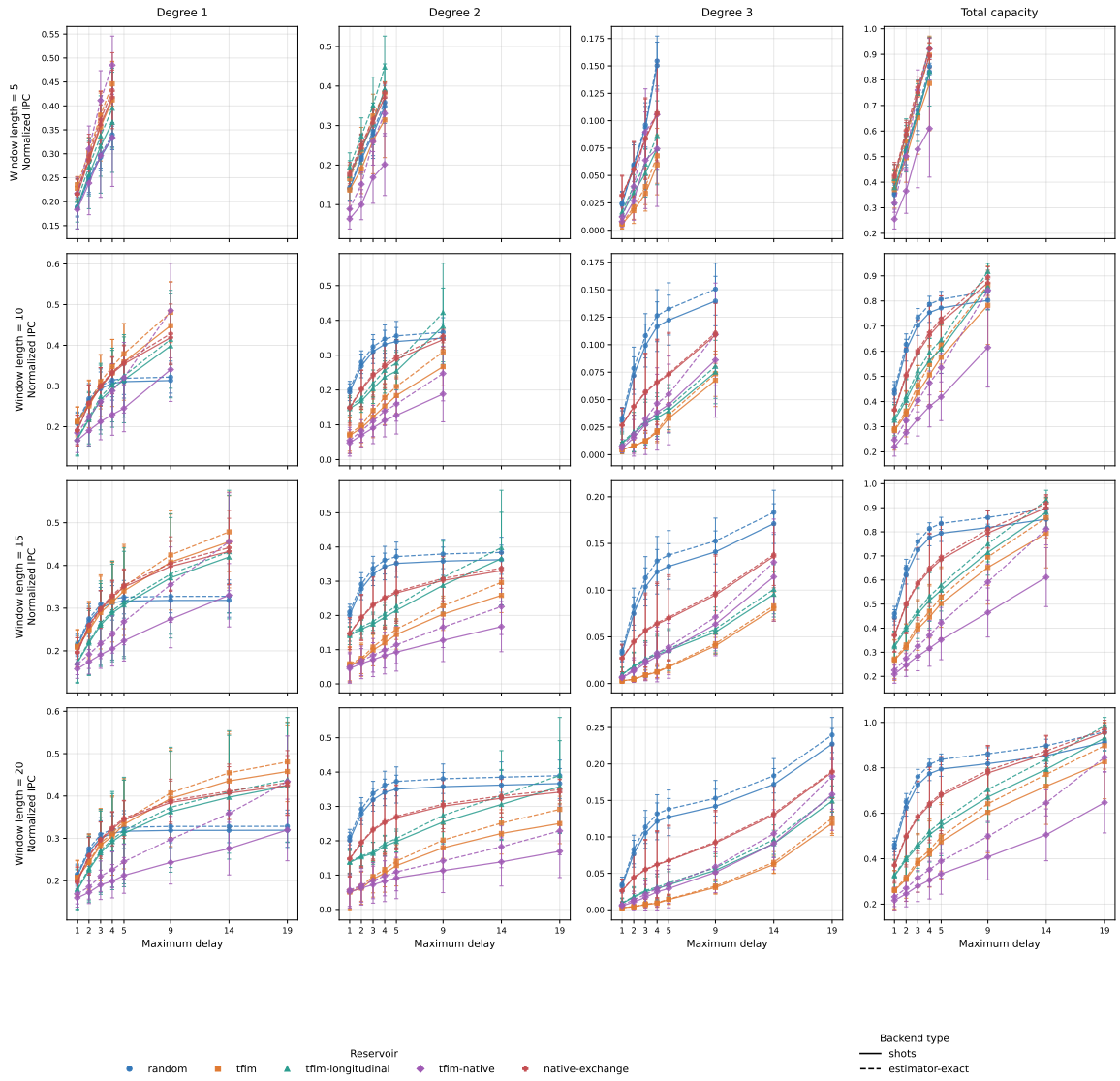


Figure 18: Normalized information processing capacity (IPC) as a function of the maximum input delay for different reservoir Hamiltonians and backend types. Rows correspond to window lengths ($L \in 5, 10, 15, 20$), while columns show the normalized degree-1, degree-2, degree-3, and total capacities. Solid and dashed lines represent the shots and exact-estimator backends, respectively. Markers distinguish the reservoir Hamiltonians. Error bars indicate the standard deviation across seeds. 2 Trotterization steps for Hamiltonian-based reservoirs and circuit depth 4 for the random reservoir.

I.2 Observables and precision

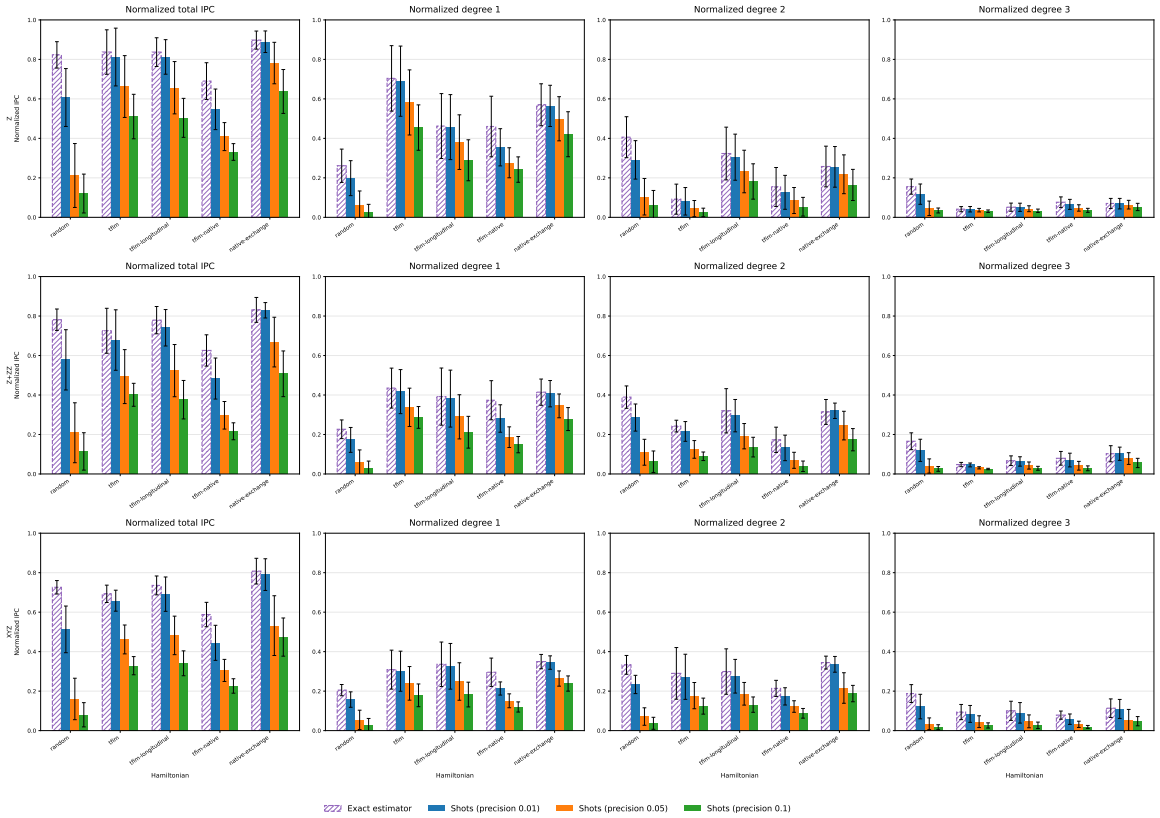


Figure 19: Normalized information processing capacity (IPC) across different reservoir Hamiltonians, observable sets, and shot precisions. Rows correspond to the measured observable sets (Z, $Z + ZZ$, and XYZ), while columns display the normalized total, degree-1, degree-2, and degree-3 capacities. Bar patterns and colors distinguish the exact-estimator baseline (purple hatched) from finite-shot measurements at precisions $p \in \{0.01, 0.05, 0.1\}$ (precision $\sim 1/\sqrt{\text{shots}}$). Error bars indicate the standard deviation across seeds. All Hamiltonian models use 2 Trotterization steps and the random reservoir uses a circuit depth of 4.

I.3 Observables and precision: absolute IPC

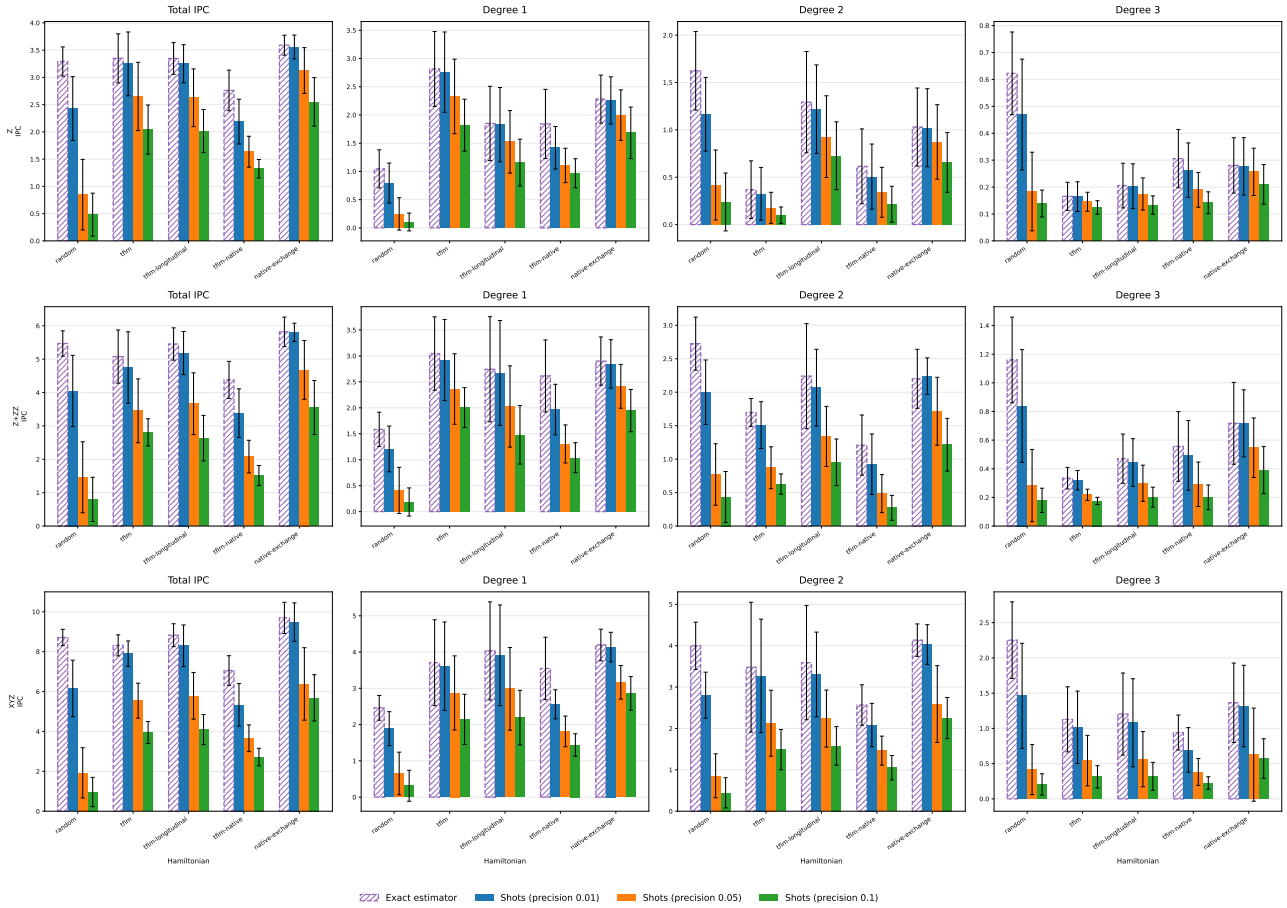


Figure 20: Absolute information processing capacity (IPC) across different reservoir Hamiltonians and observable sets. Rows correspond to the measured observable sets (Z, $Z + ZZ$, and XYZ), while columns show the absolute total, degree-1, degree-2, and degree-3 capacities. Bar patterns and colors distinguish the exact-estimator baseline (unfilled, purple hatched) from finite-shot measurements at precisions $p \in \{0.01, 0.05, 0.1\}$ (precision $\sim 1/\sqrt{\text{shots}}$). Error bars indicate the standard deviation across seeds. Note that the y-axis scales are independent across subplots. All Hamiltonian models use 2 Trotterization steps, while the random reservoir uses a circuit depth of 4.

I.4 Observables and precision under noise

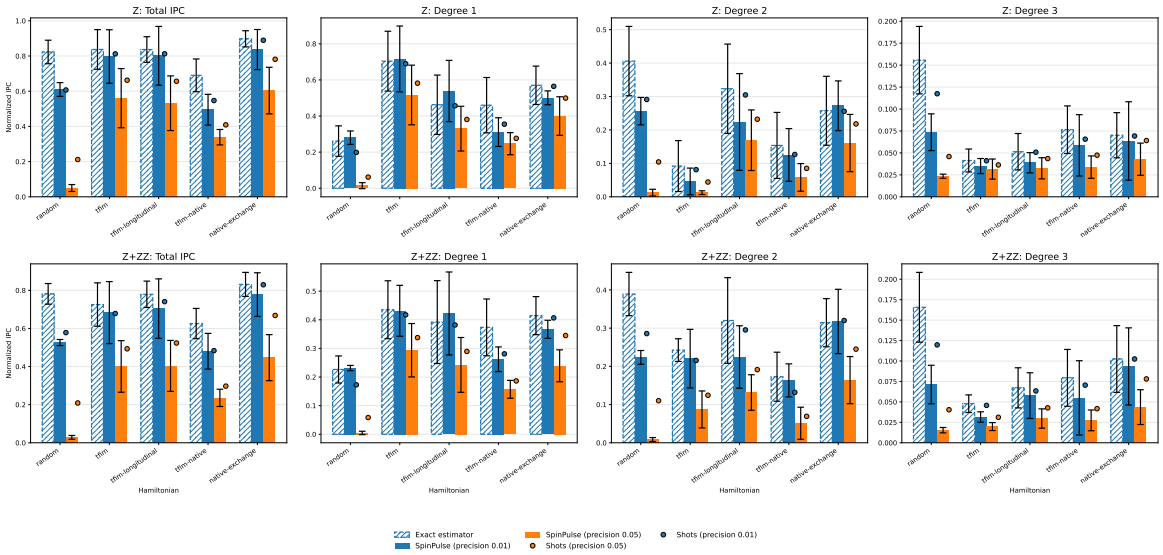


Figure 21: Normalized information processing capacity (IPC) across different reservoir Hamiltonians and observable sets. Rows correspond to the measured observable sets (Z and $Z+ZZ$), while columns show the normalized total, degree-1, degree-2, and degree-3 capacities. Bar patterns and colors distinguish the exact-estimator baseline (blue hatched) from spinpulse backend estimation runs at precisions $p \in \{0.01, 0.05\}$ (precision $\sim 1/\sqrt{\text{shots}}$). The points represent the shot-based IPC from Figure 19 for comparison. Error bars indicate the standard deviation across seeds. All Hamiltonian models use 2 Trotterization steps, while the random reservoir uses a circuit depth of 4.

J Input qubits reset ablation

The short ablation presented in Figure 22 shows how input qubits reset plays the role of an essential dissipation mechanism under ideal conditions. We see that without reset, the QRC matches the reset QRC at $L = 5$, but from $L = 8$ it falls below the linear baseline and loses the current transaction.

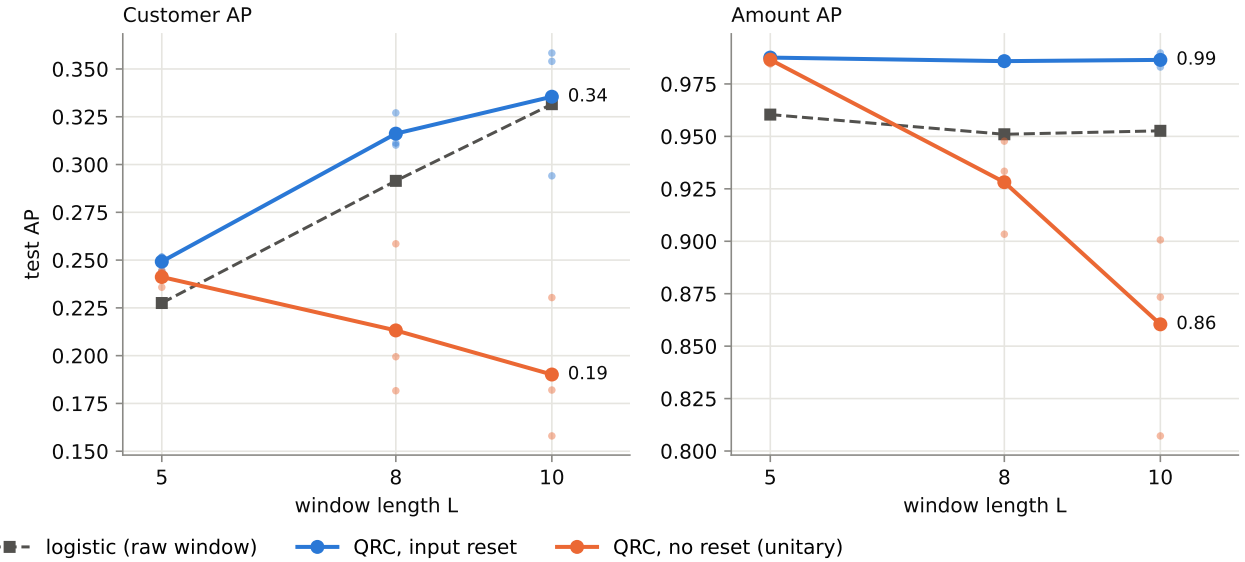


Figure 22: Input qubits reset ablation for the sliding-window QRC (1 input + 3 memory qubits, random circuit of depth 4, XYZ + two-body XYZ observables, exact simulation). Test average precision against window length L, for customer-compromise fraud (left, needs memory of past transactions) and amount fraud (right, needs the current transaction). Large markers: mean over seeds 42–44; small dots: individual seeds. Dashed: logistic regression on the raw window.